Project:
Analyze Data in a Model Car Database with MySQL Workbench

Step1: Required Files & Set-up

I started with setting up MySQL Workbench and importing the MySQL database in Power BI for visualizations:

MySQL set-up:

First think I need to do was to install the MySQL Workbench in order to have a proper environment to work with. After MySQL Workbench is installed, configured, launched and connected to a local MySQL server instance, I had to download the “mintclassicsDB.sql” file, which is the Mint Classics relational database, and then import it to use the script (the script includes commands to create the schema) to create the Mint Classic database on my MySQL Workbench platform.

Importing the Database into Power BI

During this step, you might encounter a message stating, **"This connector requires one or more additional components to be installed before it can be used."** This means an extra component is needed for the connection. Simply follow the provided link, which will likely direct you to:

🔗 MySQL Connector Download

Download the required component and, before attempting to import the database again, try refreshing Power BI or restarting the application. Once that’s done, you can follow the detailed instructions in this video on **How to Connect MySQL Database to Power BI**, where the process is explained thoroughly:

📽️ YouTube Tutorial

Project Scenario & Business Problem

Project Scenario:

Mint Classics Company, a retailer of classic model cars and other vehicles, is looking at closing one of their storage facilities. The main question is which one to close? Closing which one is more beneficial or less costly for the business?

To support a data-based business decision, they are looking for suggestions and recommendations for reorganizing or reducing inventory, while still maintaining timely service to their customers. For example, they would like to be able to ship a product to a customer within 24 hours of the order being placed.

Business problem:

It is important to understand the business problem before starting the analysis. Because it is not really possible to ask questions related to the data (which is the first step of analysis, at least for me) without knowing what problem we are trying to solve.

Understanding the Business Problem

Mint Classics Company sells classic model cars and other vehicles. They are considering closing one of their storage facilities (warehouses). However, they want to ensure that:

- Customers still receive their orders quickly (within **24 hours** of ordering). This is very important because we don't want to lose customers, which => losing sales.
- Inventory is **organized efficiently** to reduce storage costs.
- The company **does not lose sales** due to stock shortages or delays.

Understanding the Database

Understanding the Data

Another important step before analysis is the data we are working with. We need to be familiar with the database, what tables contain what information, which tables are more important for solving the business problem, how the tables are related (in the case of relational database, which is our case), etc.

After understanding the database, it is time to focus on solving the business problem. What can be analysed and what kind of insight might be interesting (ask questions, analyse, get answers, give business recommendations).

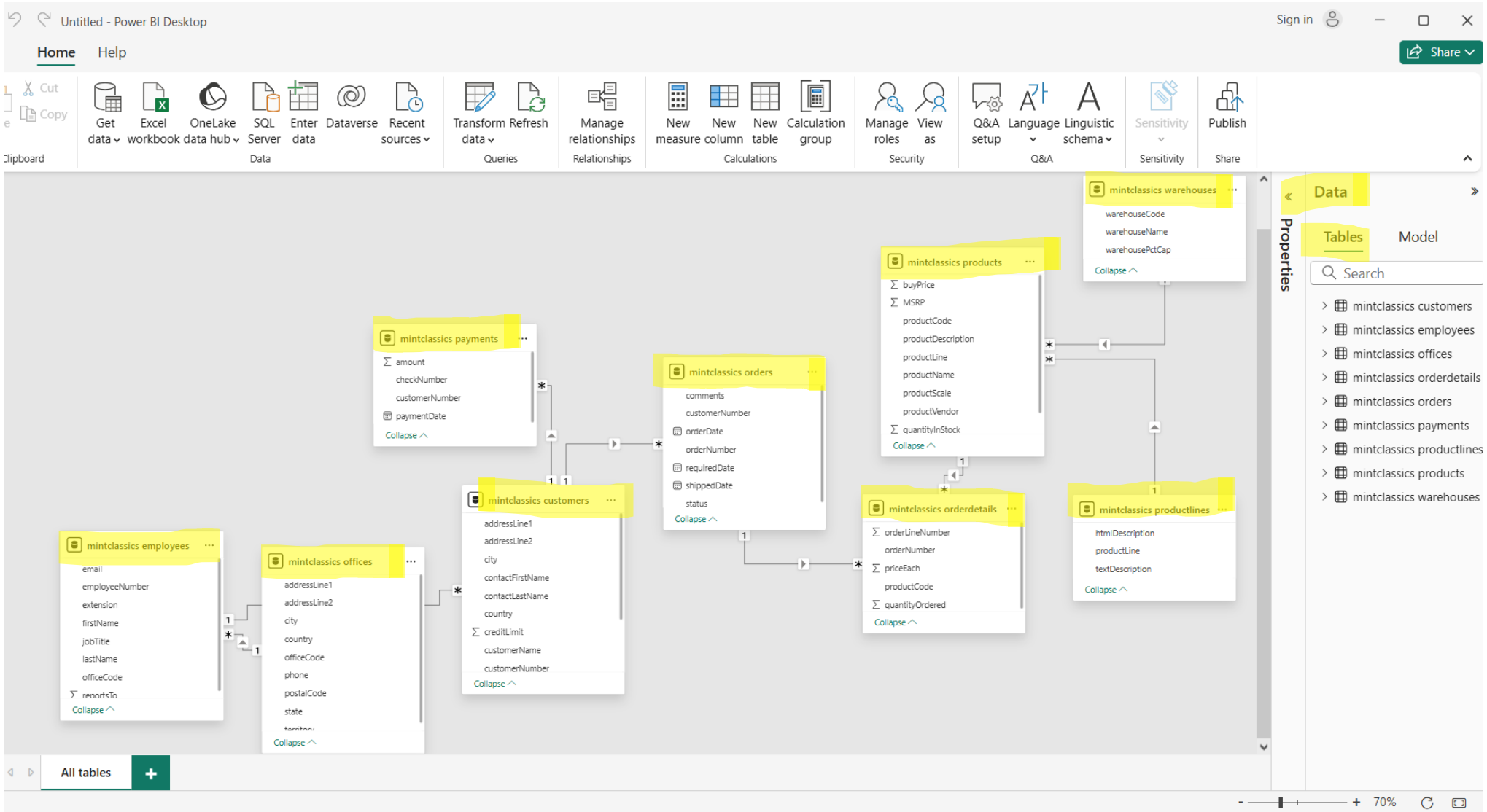
This step will continue with asking questions. Questions always help me go deeper, and this is helpful for analysing the data - seeking answers leads to continuous learning and deeper understanding.

Since we have 9 tables and an **ER diagram(check below)**, let's start by:

- Checking **how they are related** (primary keys, foreign keys)
- Understanding **what each table represents**
- Identifying **which tables are most relevant** to solving the problem

My goal -> Familiarize myself with the database and the business process: Before focusing on the business problem, I will focus on analysing the dataset, finding relationships between tables, answer some questions. This is, I will do general EDA (exploratory data analysis), answer general questions that might pop up for the dataset we have. This will help me to get familiar with the mintclassics dataset and to find relationships. Later, I can focus on solving the business problem.

Before moving into Step 2, I want to provide the snapshot of how our relational database or in other words the **EER (Extended Entity-Relationship)** diagram looks like on **Power BI** tool (table names are highlighted):



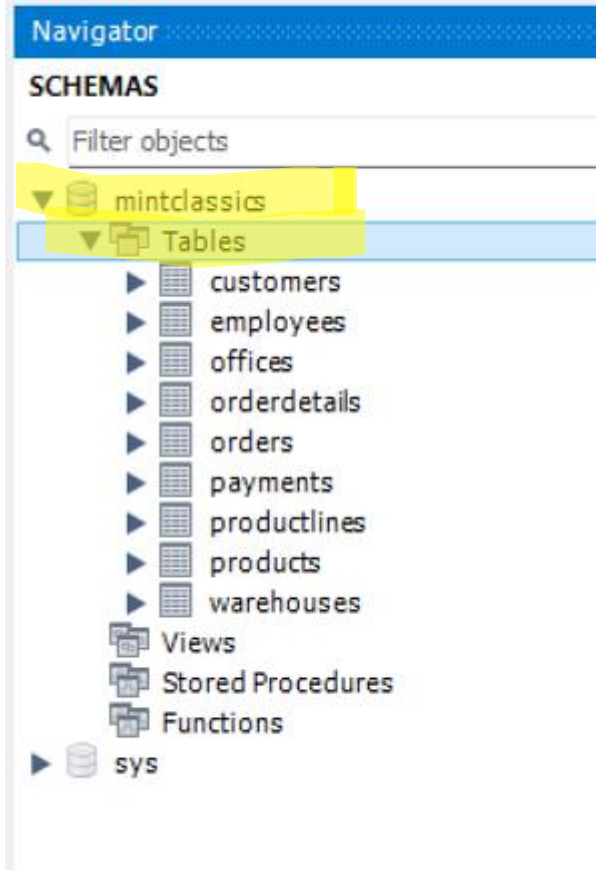
Step 2: Understanding Table Relationships

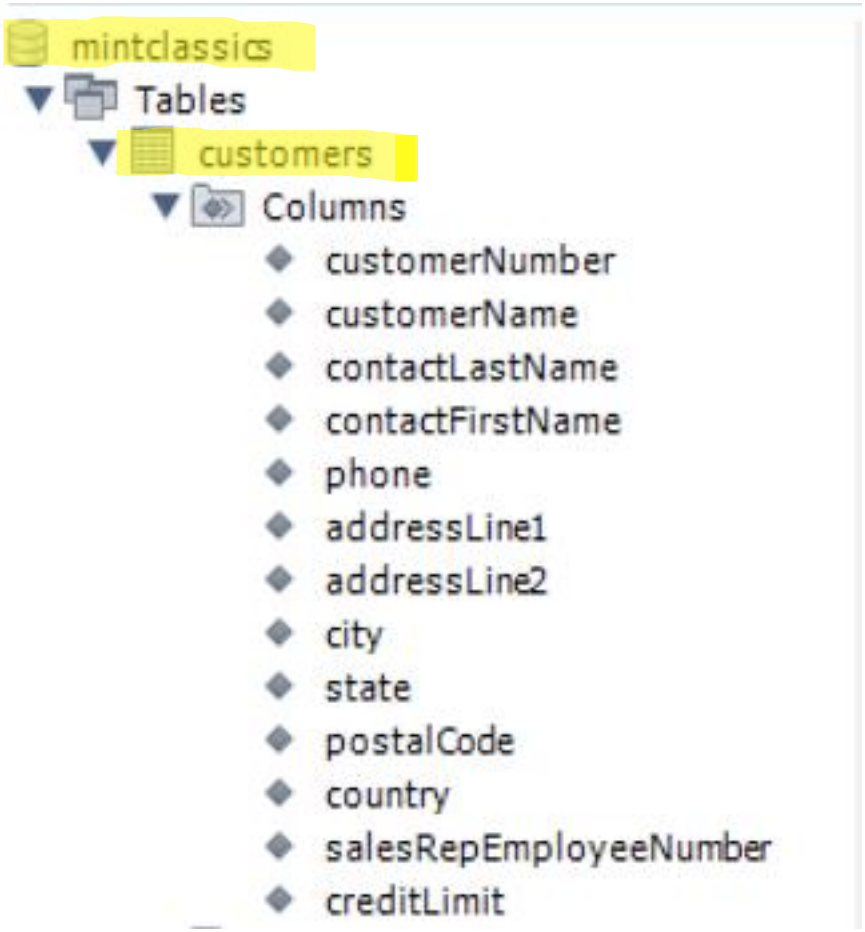
In this step, before starting the analysis, I usually take time to understand the project database, what kind of data we are working on, what tables do I have in this database, how they are related, what data they contain, are there any common information (column names might be different but the data it covers can be the same).

Our database is called **mintclassics (**it comes from the hypothetical company name **Mint Classics),** which contains 9 tables (customers, employees, offices, orderdetails, orders, payments, products, productlines, warehouses).

I will go through all the tables to understand what information they contain.

The first one is the **customers** table: this table contains data about the customer demographics. Note that our customers are companies but not individual person, but our contacts are people, of course. This table has 13 columns, which are given in the snapshot on the left:





customerNumber	customerName	contactLastName	contactFirstName	phone	addressLine1
475	West Coast Collectables ...	Thompson	Steve	3105553722	3675 Furth Circle
459	Warburg Exchange	Ottlieb	Sven	0241-039123	Walserweg 21
144	Volvo Model Replicas, Co	Berglund	Christina	0921-12 3555	Berguvsvägen 8
181	Vitachrome Inc.	Frick	Michael	2125551500	2678 Kingston Rd.
298	Vida Sport, Ltd	Holz	Mihael	0897-034555	Grenzacherweg 237
201	UK Collectables, Ltd.	Devon	Elizabeth	(171) 555-2282	12, Berkeley Gardens Blvd
205	Toys4GrownUps.com	Young	Julie	6265557265	78934 Hillside Dr.
186	Toys of Finland, Co.	Karttunen	Matti	90-224 8555	Keskuskatu 45
259	Toms Spezialitäten, Ltd	Pfalzheim	Henriette	0221-5554327	Mehrheimerstr. 369
398	Tokyo Collectables, Ltd	Shimamura	Akiko	+81 3 3584 0555	2-2-8 Roppongi
450	The Sharp Gifts Warehouse	Frick	Sue	4085553659	3086 Ingle Ln.
328	Tekni Collectables Inc.	Brown	William	2015559350	7476 Moss Rd.
161	Technics Stores Inc.	Hashimoto	Juri	6505556809	9408 Furth Circle

When checking the data in the columns of the **customers** table, the first 6 of them is all about the customer names, addresses, contacts and contact persons, and no Null values exist in these columns.

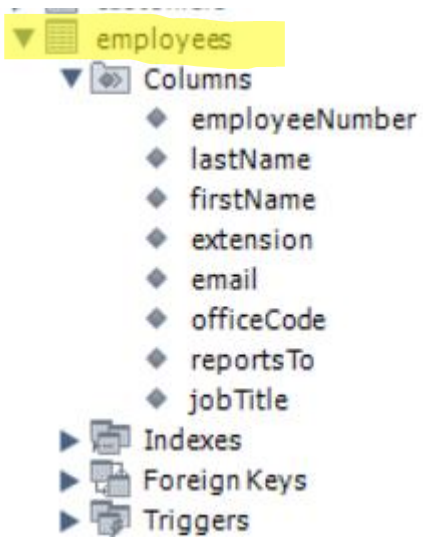
Whereas the other 7 columns contains some Null values. I will see later if these has any impact on my analysis.

About the information it contains, they are countries, cities and states where the customers/companies locate, their postal codes, companies credit limits (I need to check what this actually refers to), and salesRepEmployeeNumber (I guess this means Representative Employee in Sales (?))

addressLine2	city	state	postalCode	country	salesRepEmployeeNumber	creditLimit
NULL	Nantes	NULL	44000	France	1370	21000.00
NULL	Las Vegas	NV	83030	USA	1166	71800.00
Level 3	Melbourne	Victoria	3004	Australia	1611	117300.00
NULL	Nantes	NULL	44000	France	1370	118200.00
NULL	Stavern	NULL	4110	Norway	1504	81700.00
NULL	San Rafael	CA	97562	USA	1165	210500.00
NULL	Warszawa	NULL	01-012	Poland	NULL	0.00
NULL	Frankfurt	NULL	60528	Germany	1504	59700.00
NULL	San Francisco	CA	94217	USA	1165	64600.00
NULL	NYC	NY	10022	USA	1323	114900.00
NULL	Madrid	NULL	28034	Spain	1370	227600.00
NULL	Luleå	NULL	S-958 22	Sweden	1504	53100.00
NULL	Kobenhavn	NULL	1734	Denmark	1401	83400.00

This table contains information about employee demographics, their email addresses, offices, job titles and the employee number of whom they report to. For example, Murphy Diane is the President with employee number 1002, and he has a NULL values in reportsTo column. This is because he reports to nobody, but others report to him, apparently, since he is a President of the company. I am not sure about the **extension** column for the moment.

I need to do the same check for other 8 tables (okaaay). The second table is the **employees** table, and contains 8 columns:

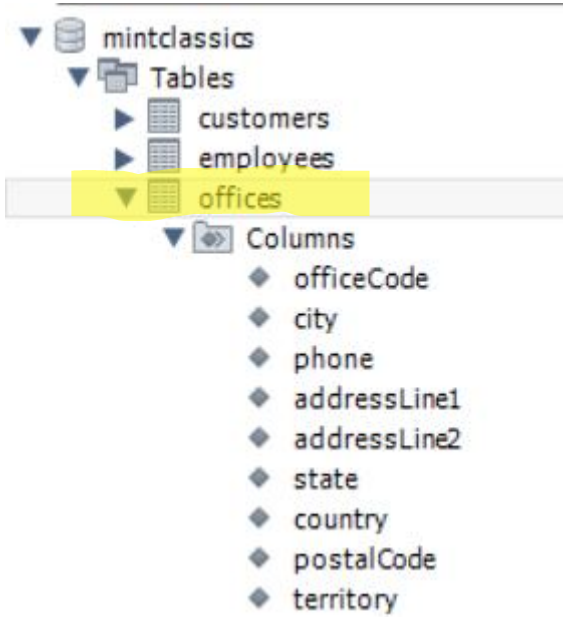



```
4 •  SELECT * FROM mintclassics.employees;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

employeeNumber	lastName	firstName	extension	email	officeCode	reportsTo	jobTitle
1002	Murphy	Diane	x5800	dmurphy@classicmodelcars.com	1	NULL	President
1056	Patterson	Mary	x4611	mpatterso@classicmodelcars.com	1	1002	VP Sales
1076	Firrelli	Jeff	x9273	jfirrelli@classicmodelcars.com	1	1002	VP Marketing
1088	Patterson	William	x4871	wpatterson@classicmodelcars.com	6	1056	Sales Manager (APAC)
1102	Bondur	Gerard	x5408	gbondur@classicmodelcars.com	4	1056	Sale Manager (EMEA)
1143	Bow	Anthony	x5428	abow@classicmodelcars.com	1	1056	Sales Manager (NA)
1621	Nishi	Mami	x101	mnishi@classicmodelcars.com	5	1056	Sales Rep
1611	Fixter	Andy	x101	afixter@classicmodelcars.com	6	1088	Sales Rep
1612	Marsh	Peter	x102	pmarsh@classicmodelcars.com	6	1088	Sales Rep
1619	King	Tom	x103	tking@classicmodelcars.com	6	1088	Sales Rep
1337	Bondur	Loui	x6493	lbondur@classicmodelcars.com	4	1102	Sales Rep
1370	Hernandez	Gerard	x2028	ghernande@classicmodelcars.com	4	1102	Sales Rep
1401	Castillo	Pamela	x2759	pcastillo@classicmodelcars.com	4	1102	Sales Rep
1501	Bott	Larry	x2311	lbott@classicmodelcars.com	7	1102	Sales Rep
1504	Jones	Barry	x102	bjones@classicmodelcars.com	7	1102	Sales Rep
1702	Gerard	Martin	x2312	mgerard@classicmodelcars.com	4	1102	Sales Rep
1165	Jennings	Leslie	x3291	ljennings@classicmodelcars.com	1	1143	Sales Rep

employees 1 ×



Offices table **:** contains 9 columns and 8 rows.
When we check the table, we can see that the business has total 7 offices in different countries; 3 offices in USA, the others in France, Japan, Australia, UK, 1 office for each.

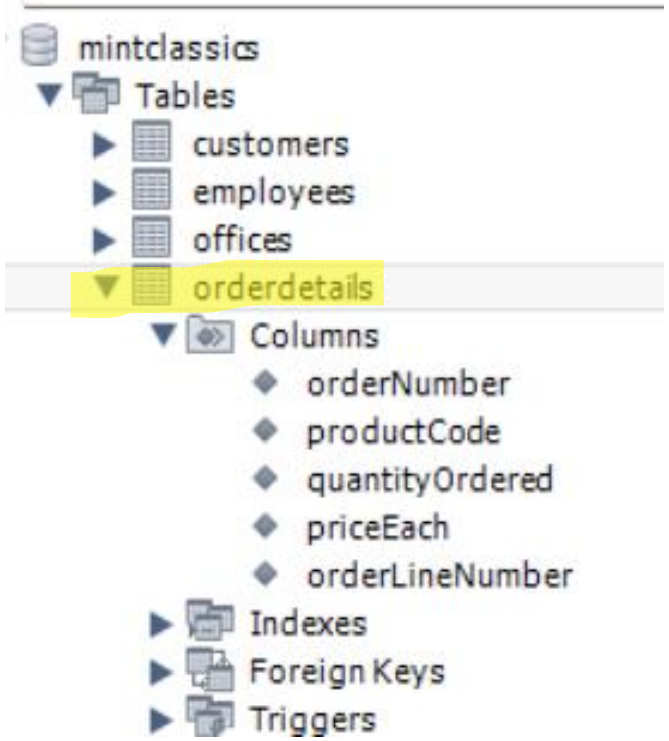
By having information from the **employees** and the **offices** table, we can find out in which **country** or **city** each employee works in.

```
4 •  SELECT * FROM mintclassics.offices;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

officeCode	city	phone	addressLine1	addressLine2	state	country	postalCode	territory
1	San Francisco	+1 650 219 4782	100 Market Street	Suite 300	CA	USA	94080	NA
2	Boston	+1 215 837 0825	1550 Court Place	Suite 102	MA	USA	02107	NA
3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A	NY	USA	10022	NA
4	Paris	+33 14 723 4404	43 Rue Jouffroy D'abbans	NULL	NULL	France	75017	EMEA
5	Tokyo	+81 33 224 5000	4-1 Kioicho	NULL	Chiyoda-Ku	Japan	102-8578	Japan
6	Sydney	+61 2 9264 2451	5-11 Wentworth Avenue	Floor #2	NULL	Australia	NSW 2010	APAC
7	London	+44 20 7877 2041	25 Old Broad Street	Level 7	NULL	UK	EC2N 1HN	EMEA
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

orderdetails table covers the information about the product code, quantity ordered, price for each product/productCode, order number, which seems like not unique. Need to check this out. Also, what orderLineNumber is about?
It turned out that the orderNumber is actually unique. The reason why it appears multiple times in the table is because multiple products can be ordered at the same time under one order number.



Under one order number **10100** multiple products, meaning that 4 different products are ordered with one order.

```
4 •  SELECT * FROM mintclassics.orderdetails;
```

Result Grid | Filter Rows: | Edit: | Export/I

orderNumber	productCode	quantityOrdered	priceEach	orderLineNumber
10100	S18_1749	30	136.00	3
10100	S18_2248	50	55.09	2
10100	S18_4409	22	75.46	4
10100	S24_3969	49	35.29	1
10101	S18_2325	25	108.06	4
10101	S18_2795	26	167.06	1
10101	S24_1937	45	32.53	3
10101	S24_2022	46	44.35	2
10102	S18_1342	39	95.55	2
10102	S18_1367	41	43.13	1
10103	S10_1949	26	214.30	11
10103	S10_4962	42	119.67	4
10103	S12_1666	27	121.64	8
10103	S18_1097	35	94.50	10
10103	S18_2432	22	58.34	2
10103	S18_2949	27	92.19	12
10103	S18_2957	35	61.84	14

orderdetails 1 ×

Check:

- **orderdetails** table:

a. **orderNumber** column – what it actually represents?

b. **productCode** – is this unique for each product?

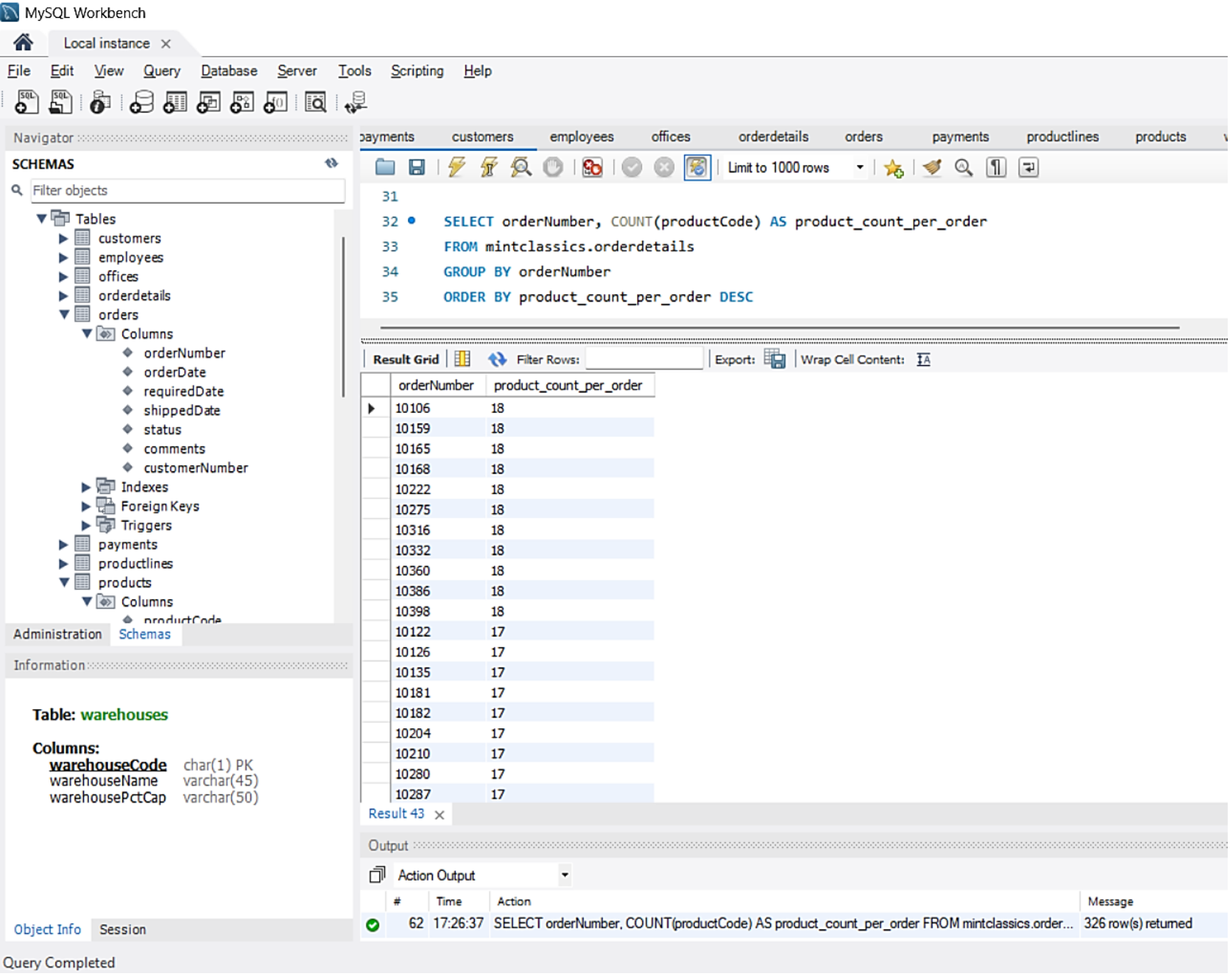
Q1: Under same orderNumber different productCode (i.e., products) appear in the table. This might mean that different products are sold to the same customer under the same orderNumber? => meaning that one order by a customer can contain multiple products:

orderNumber	productCode
10100	S18_1749
10100	S18_2248
10100	S18_4409

Q2: In the photo below, same products (productCode) have different orderNumber, which might mean the same product is ordered by different customers under different order number?

orderNumber	productCode
10107	S10_1678
10121	S10_1678
10134	S10_1678

c. orderLineNumber column – what is this column about?



I will try to answer these questions with SQL query in order to have a meaningful result.

A1: an **order (orderNumber)** can contain multiple **products (productCode),** as I noticed in the orderdetails table. An order represents a **single transaction** where a customer buys one or more products. Let's check how many products each order contains using SQL query.

Observations:

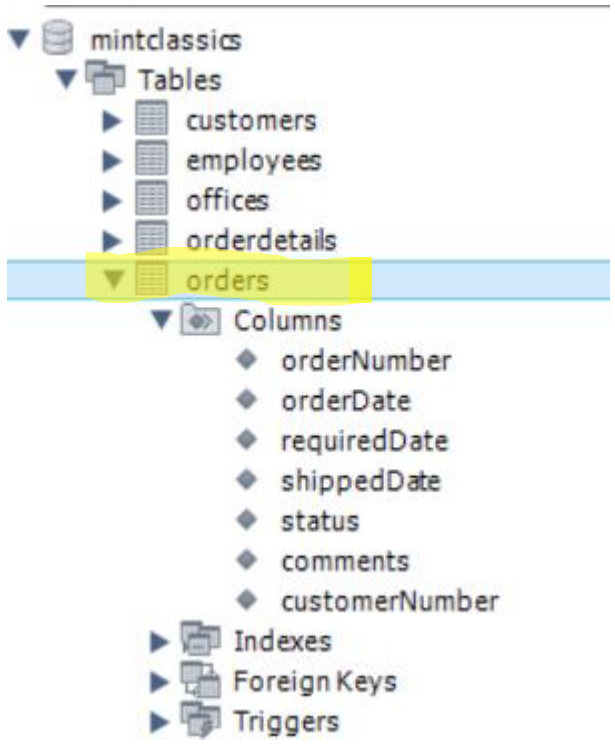
Order number 10106 contains 18 products, which means a customer purchased 18 products under a single transaction/order. Q: Are all these products ordered on the same day? My intuition says yes. We need **orderDate** column to check that. This column is in other table called **orders**. We can use JOIN to merge the tables (Needs to be checked).

Key Takeaway:

When a customer places **one order**, they can buy **one or more products**.

We can find out **when** the **orders** were placed by a **customer**, i.e., which order placed when. This way we can know when each customer made the corresponding orders.

a. And we know that **a customer can place multiple orders (a single transaction with 1 or more products) over time.**
b. It looks like **multiple orders can be placed (most probably by different customers) on each date**. By grouping by the orderDates I can find out the number of orders (total_orders) placed on each date.



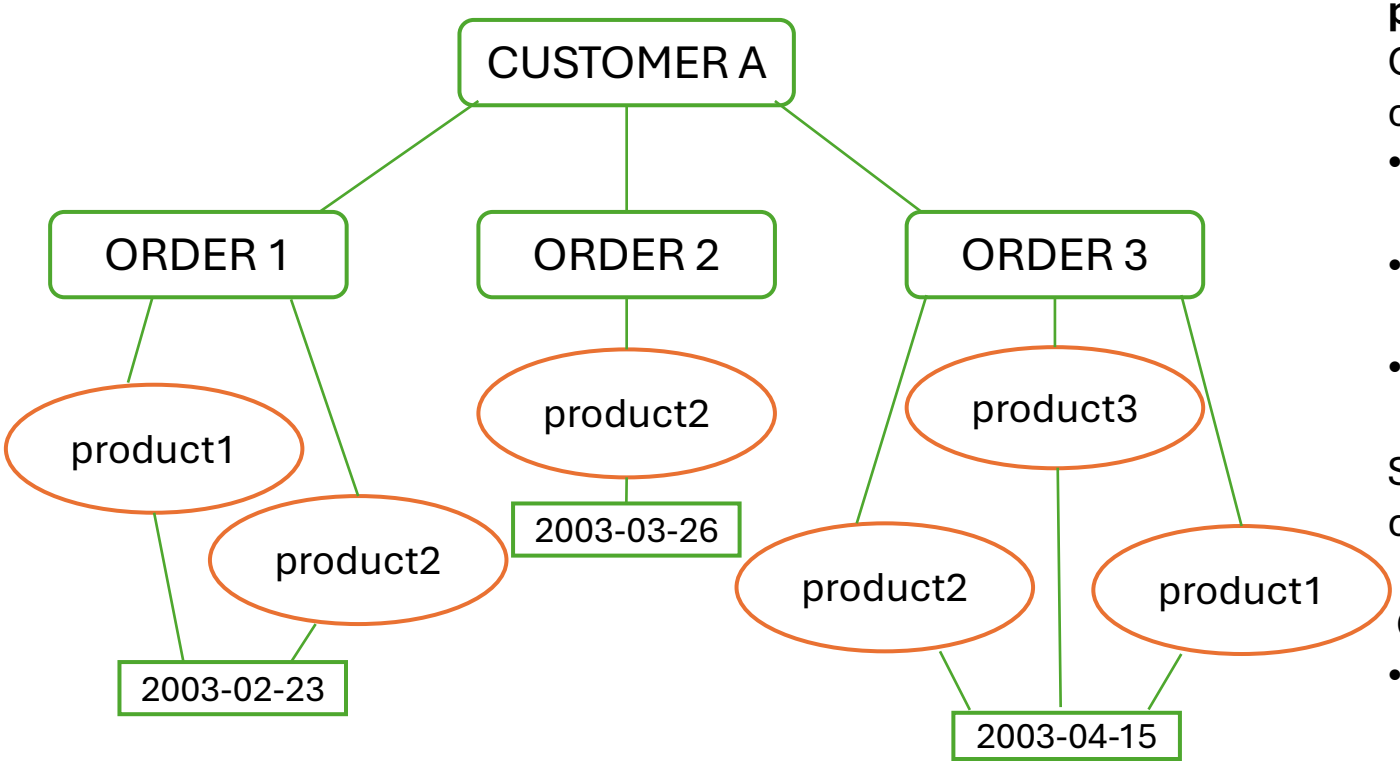

```
4 •   SELECT * FROM mintclassics.orders;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

orderNumber	orderDate	requiredDate	shippedDate	status	comments	customerNumber
10138	2003-07-07	2003-07-16	2003-07-13	Shipped	NULL	496
10179	2003-11-11	2003-11-17	2003-11-13	Cancelled	Customer cancelled due to urgent budgeting iss...	496
10360	2004-12-16	2004-12-22	2004-12-18	Shipped	NULL	496
10399	2005-04-01	2005-04-12	2005-04-03	Shipped	NULL	496
10207	2003-12-09	2003-12-17	2003-12-11	Shipped	Check on availability.	495
10243	2004-04-26	2004-05-03	2004-04-28	Shipped	NULL	495
10186	2003-11-14	2003-11-20	2003-11-18	Shipped	They want to reevaluate their terms agreement...	489
10213	2004-01-22	2004-01-28	2004-01-27	Shipped	Difficult to negotiate with customer. We need m...	489
10149	2003-09-12	2003-09-18	2003-09-17	Shipped	NULL	487
10219	2004-02-10	2004-02-17	2004-02-12	Shipped	NULL	487
10109	2003-03-10	2003-03-19	2003-03-11	Shipped	Customer requested that FedEx Ground is used...	486
10236	2004-04-03	2004-04-11	2004-04-08	Shipped	NULL	486
10331	2004-11-17	2004-11-23	2004-11-23	Shipped	Customer requested special shippment. The inst...	486
10184	2003-11-14	2003-11-22	2003-11-20	Shipped	NULL	484
10303	2004-10-06	2004-10-14	2004-10-09	Shipped	Customer inquired about remote controlled mod...	484
10199	2003-12-01	2003-12-10	2003-12-06	Shipped	NULL	475
10215	2004-01-29	2004-02-08	2004-02-01	Shipped	Customer requested that FedEx Ground is used	475

orders 1

How can we explain the relationship between **a. a customer can place multiple orders (a single transaction with 1 or more products) over time** and **b. multiple orders can be placed (most probably by different customers) on each date** ? I will try to explain it with a simple tree example with my observations.



a. a customer can place multiple orders (a single transaction with 1 or more products) over time.

E.g. on each given dates in the tree, only one order is placed => one order at the exact same time.
I showed this claim with SQL query, see next slide.

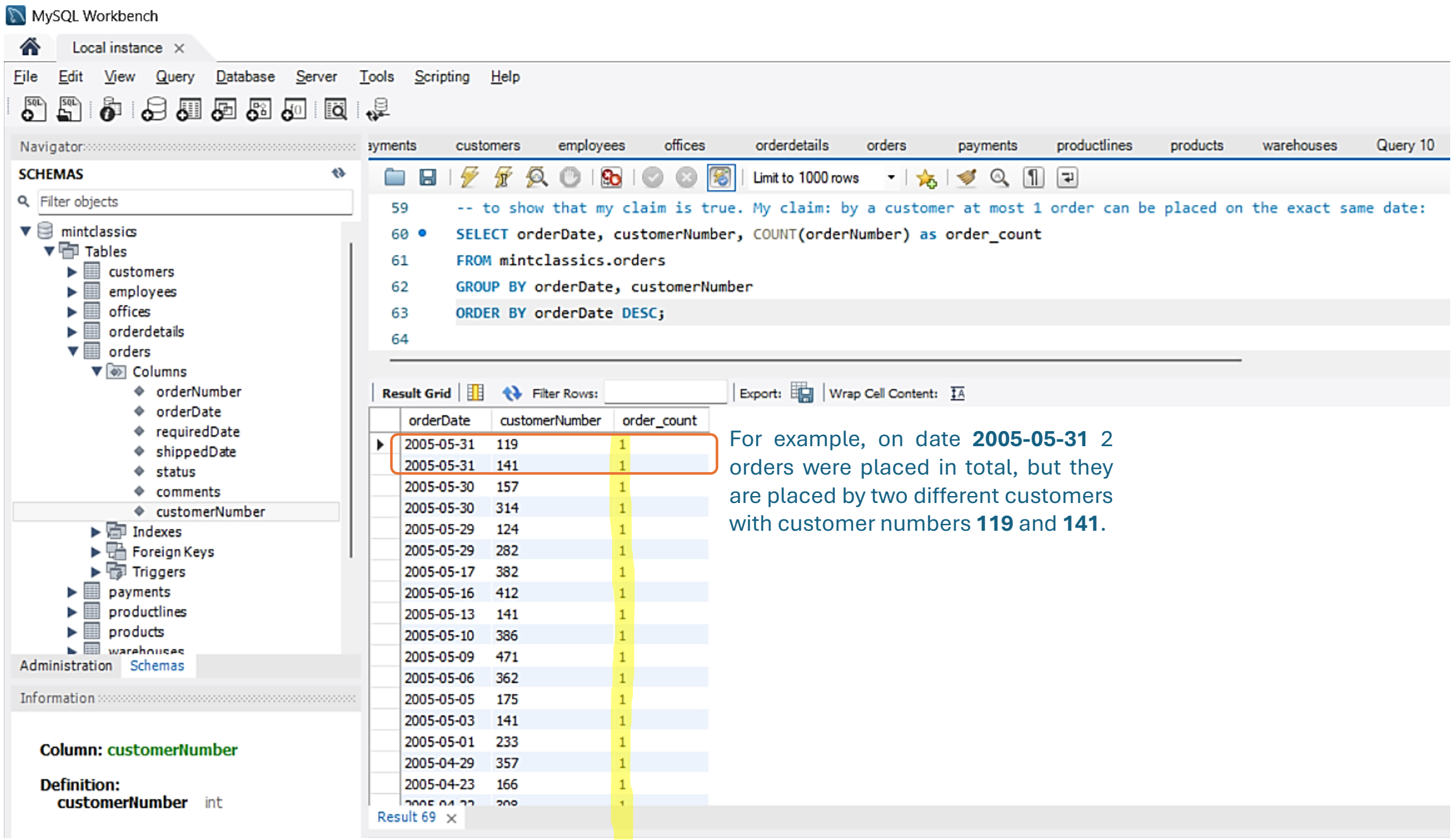
Description of the example tree for a.: what you see here is nothing, but the simple structure of how **customer-order-product** relationship happens. We have a customer called CUSTOMER A who placed three orders (ORDER 1,2,3) with one or more products. Each order placed on different dates.

- On 2003-02-23, they ordered two products (product1 & product 2) under ORDER 1.
- About a month later on 2003-03-26, they placed another order, ORDER 2, with one product (product2).
- Finally, the last order, ORDER 3, is placed on 2003-04-15 with three products (product3, product2 & product1).

So, we showed that the statement **a.** makes sense, and a customer has placed three orders over three months period.

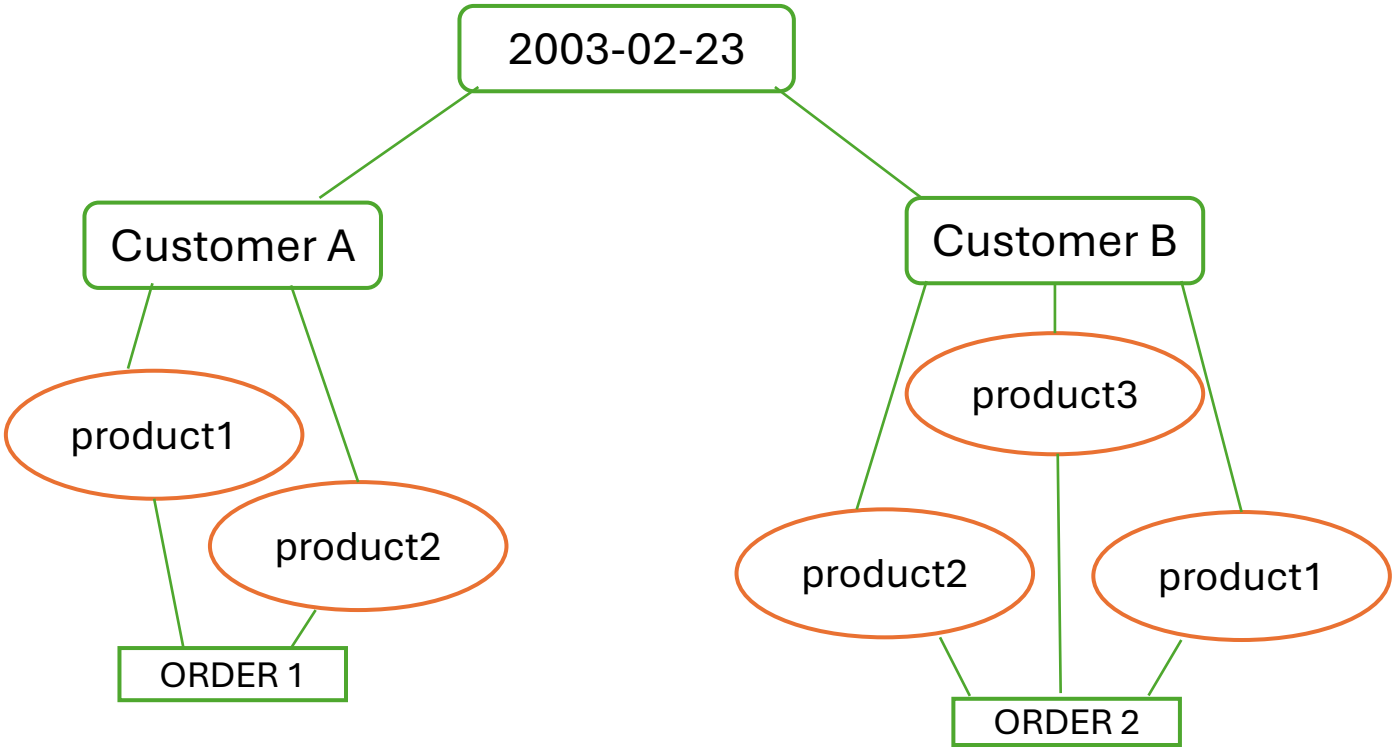
Observations:

- A customer can order one specific product multiple times over a period of time. One possible reason for this could be that the customer actually likes this product a lot. In my example, it looks like the customer prefer **product2** over the other products; they ordered **product2** 3 times over time, whereas the others are ordered less.
- By one customer at most 1 order can be placed on each date (i.e., on the exact same time). But the orders can have multiple products. Is this true? Let's check it.



For example, on date **2005-05-31** 2 orders were placed in total, but they are placed by two different customers with customer numbers **119** and **141**.

As claimed, only 1 order on each date by a customer. Note that this claim is not true when we consider the order count on the same date by different customers, because as we have shown, on the same day multiple orders can be placed by multiple/different customers (Statement b.).



b. multiple orders can be placed (most probably by different customers) on each date.

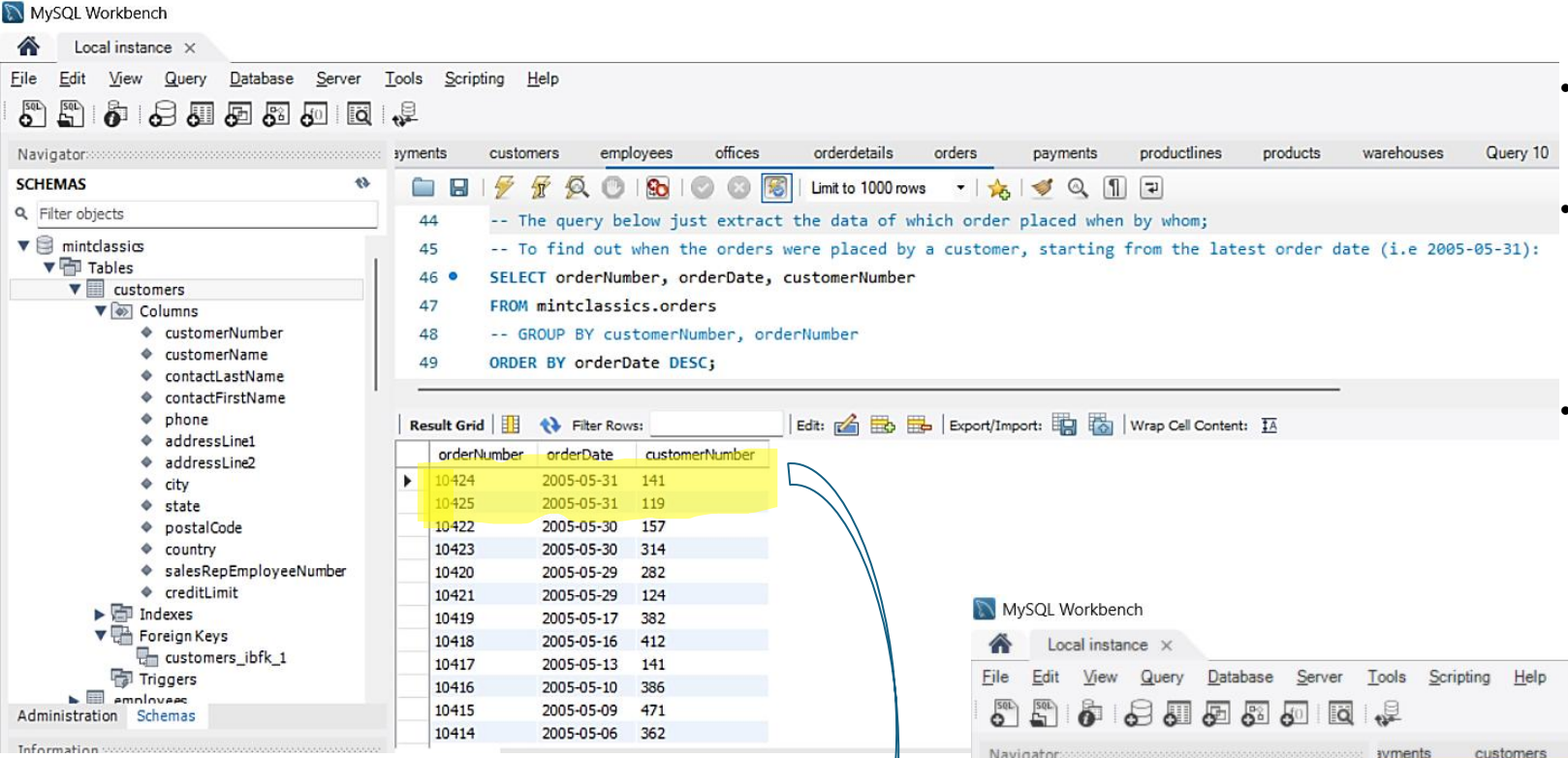
Now, we can move into the explanation of statement **b.** with a similar example.

Description of the example tree for b.: what you see here is nothing but the simple structure of **customer-order-product** relationship. Now the tree is slightly different, because what we investigate is different here. We have customers, Customer A & Customer B, who placed one order for each (ORDER 1 & 2, respectively) with one or more products. Here, each order by two different customers are placed on the same date.

- Customer A ordered two products (product1 & product2) under ORDER 1 on 2003-02-23.
- Customer B ordered three products (product1 & product2 & product3) under ORDER 2 on 2003-02-23.

So, 2 orders are placed on the same date, on 2003-02-23, but by two different customers. This proves our statement **b.**

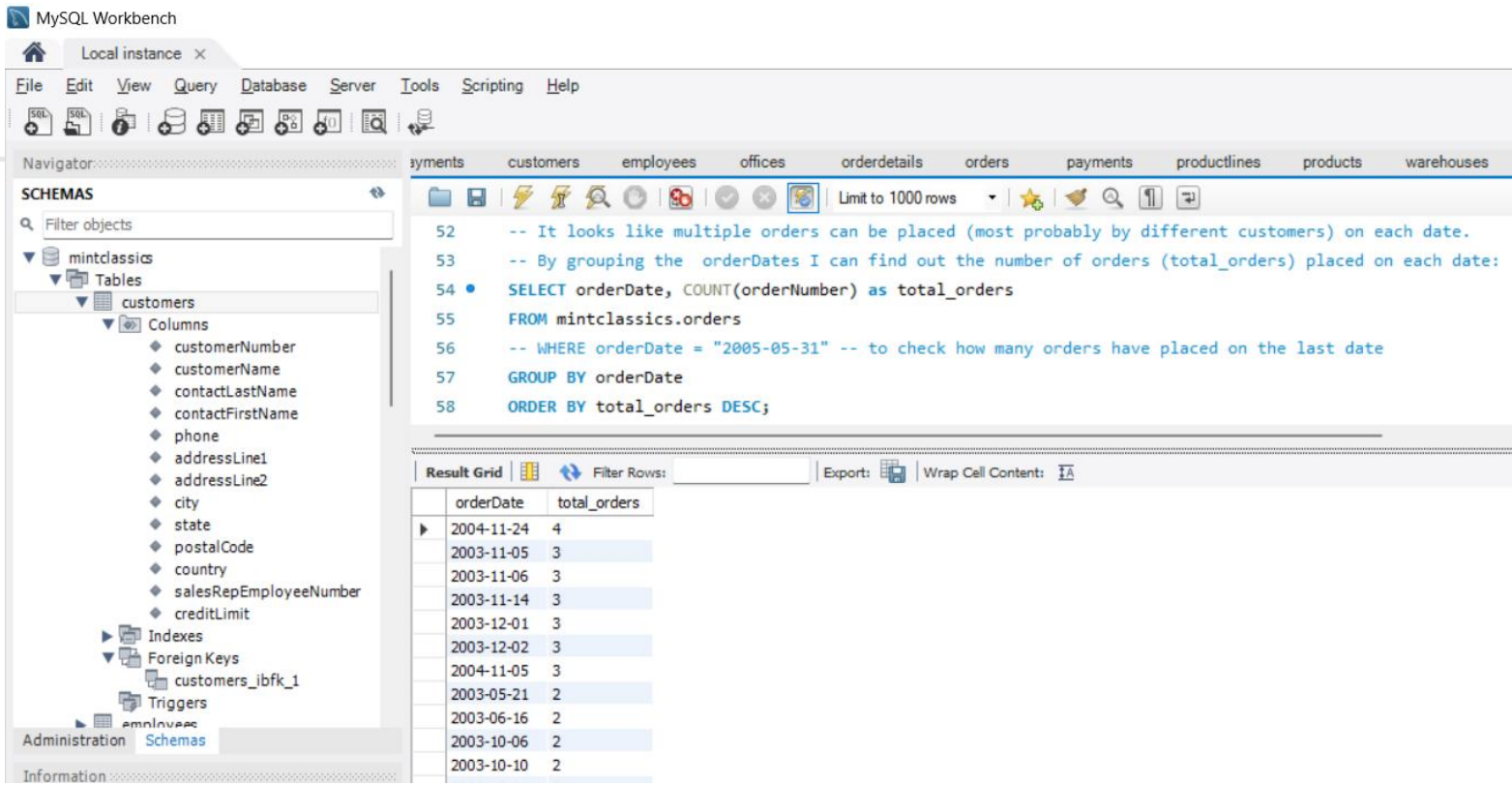
Let's extract this information from our dataset with SQL query. See the next slide.



- The snapshot on the left shows which order (orderNumber) placed when (orderDate) by whom (customerNumber).
- It can be observed that multiple orders can be placed (most probably by different customers) on each date. I can find out the number of orders (total_orders) placed on each date (check the second snapshot below right corner).
- Moreover, instead of checking manually, you can find out how many orders have placed on the last date of our dataset by commenting out the WHERE clause in the query below (the answer is, 2 orders have been placed on 2005-05-31).

For example: multiple orders (10424 and 10425) on the same date (2005-05-31) by different customers (141 and 119). Others also continue accordingly.

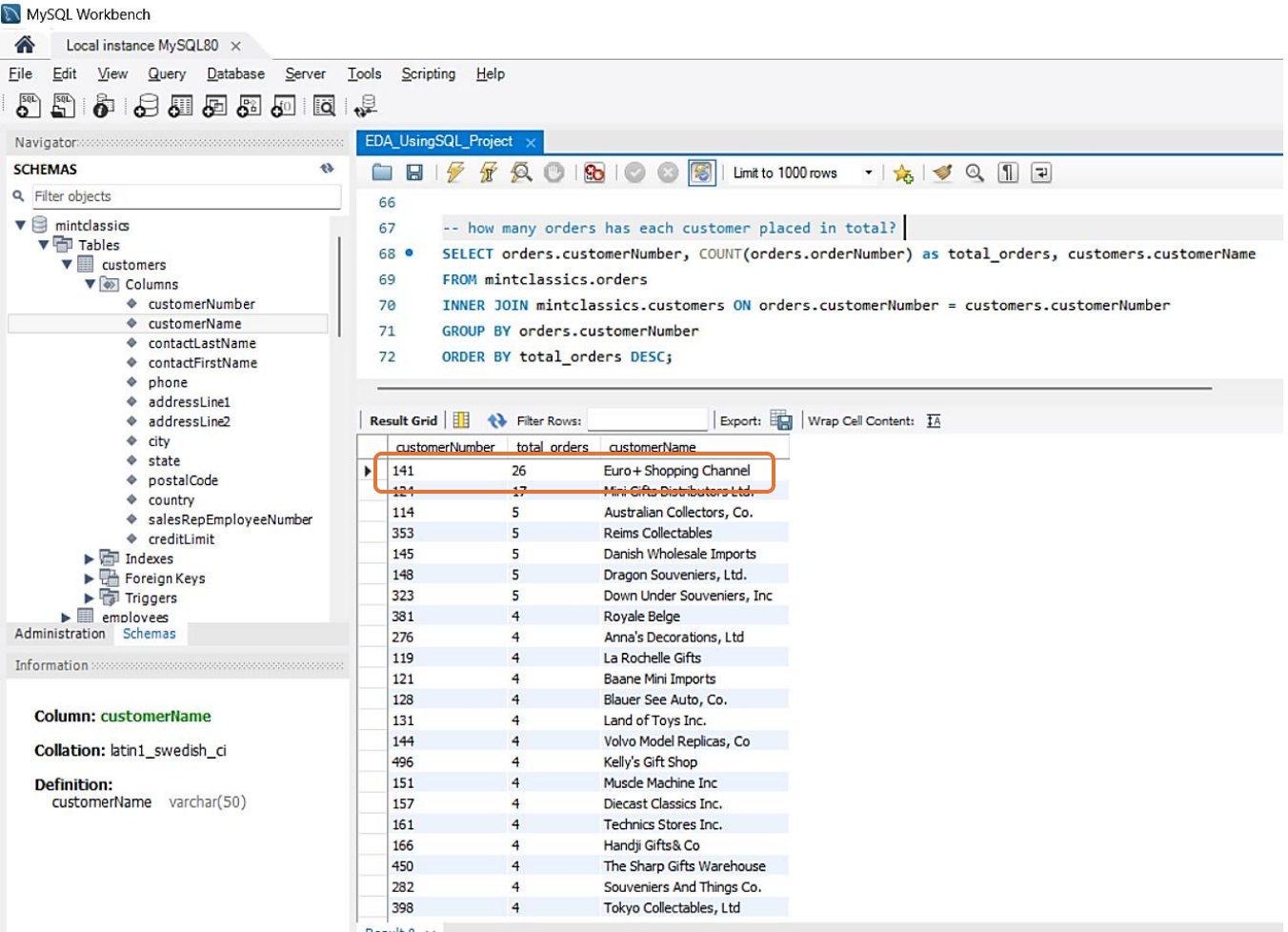
This is the same what I stated above(in result1 with SQL slide), the only difference is this result contains information about order numbers (orderNumber) that placed by each customer, but the one above contains order dates (orderDate) of each customer.



I would like to check our database using SQL queries? I could:

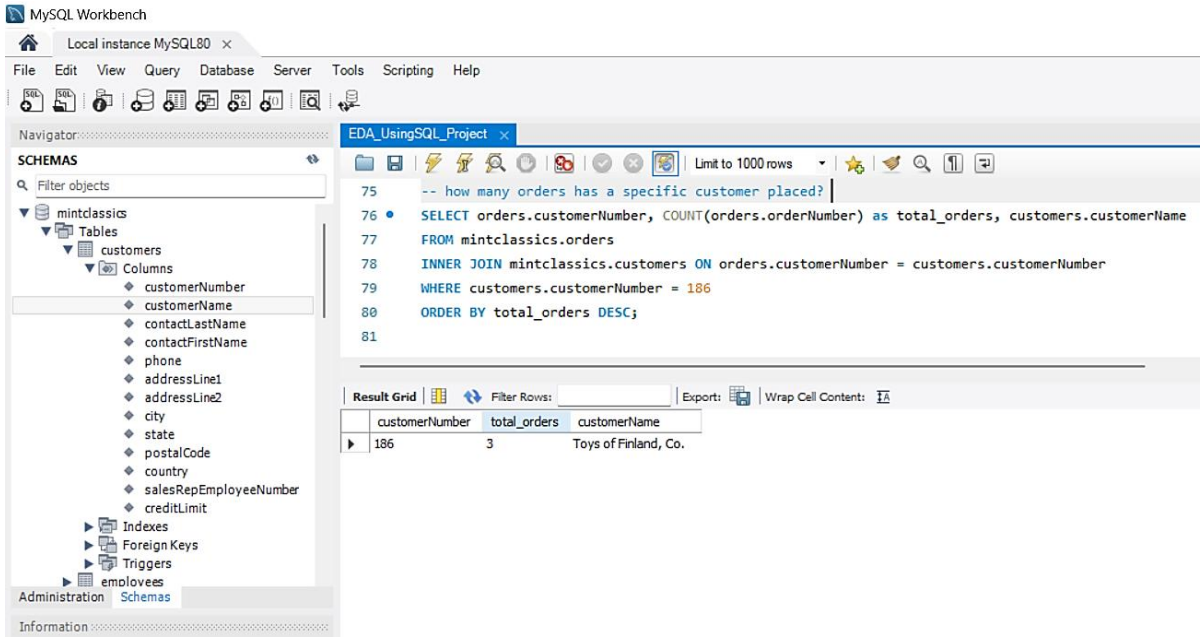
1. ~~**Check how many orders each customer has placed.**~~
2. **Check how many orders a specific customer has placed.**
3. **Check how many products are included in an order.**
4. **See if any customers placed multiple orders on the same day.**

1. I already found out the total number of orders placed on each date. Now I want to know the total number of orders has been placed by each customer:



1. **Check how many orders each customer has placed.**
2. ~~**Check how many orders a specific customer has placed.**~~
3. **Check how many products are included in an order.**
4. **See if any customers placed multiple orders on the same day.**

2. Check how many orders a specific customer has placed:



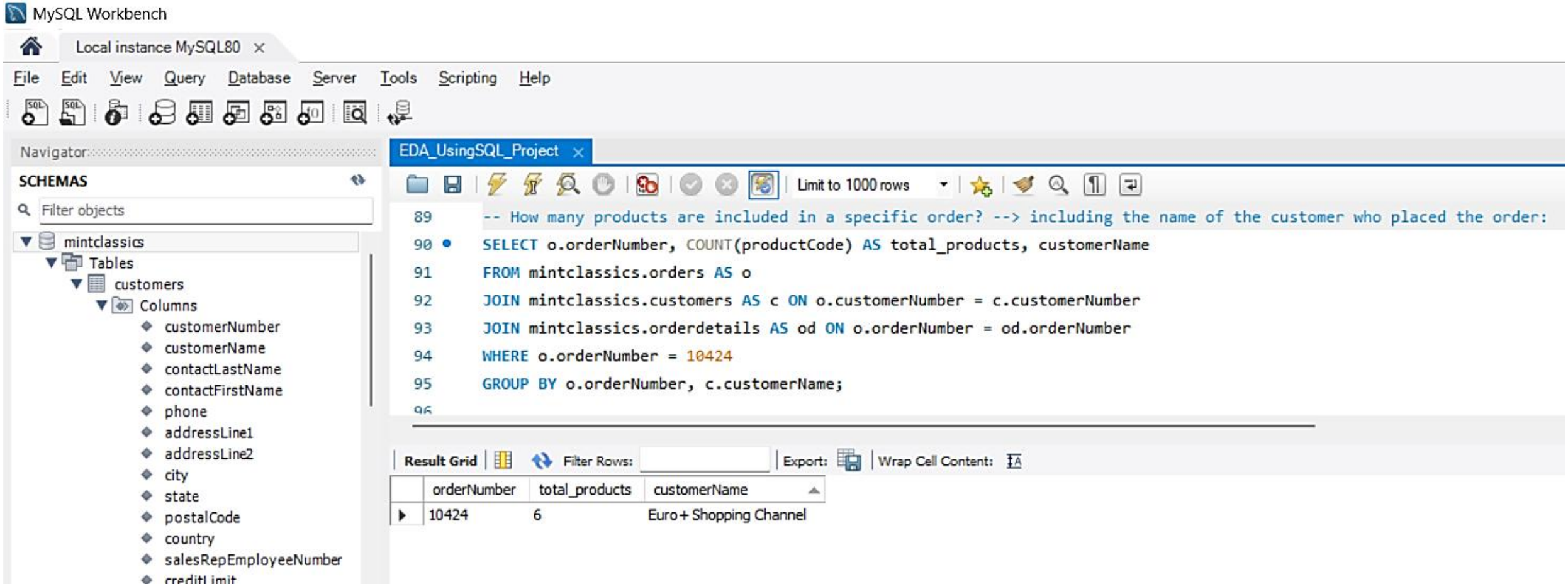
Observations/Insights:

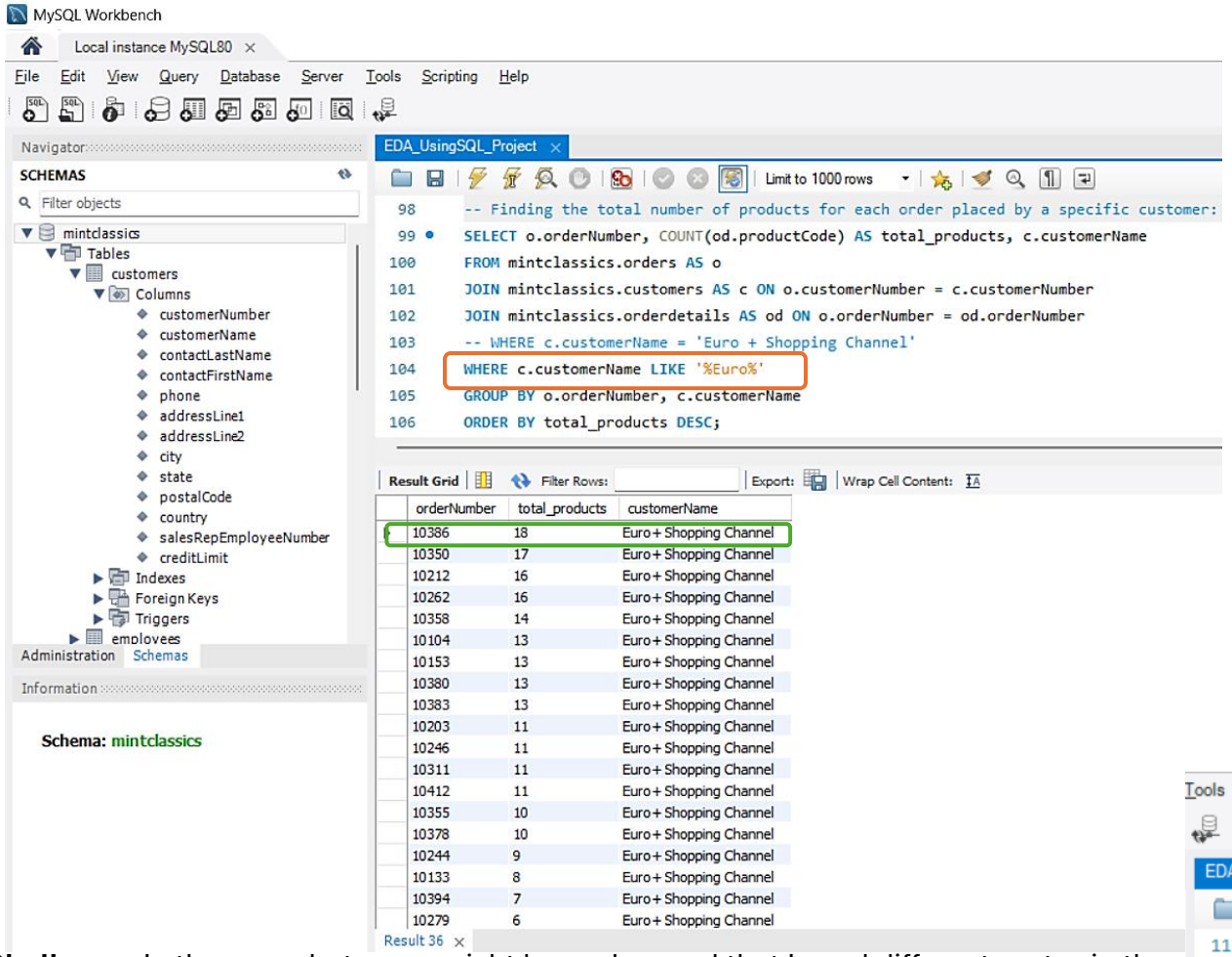
- In **1.**, we can observe that the **Euro + Shopping Channel** is the customer that placed the highest number of orders - in total 26 order.

⇒ We can check what products are ordered by this customer frequently. Where those products that are in demand by this customer are stored, etc.

1. **Check how many orders each customer has placed.**
2. **Check how many orders a specific customer has placed.**
3. ~~**Check how many products are included in a specific order.**~~
4. **See if any customers placed multiple orders on the same day.**

3. Check how many products are included in a specific order (including the name of the customer). To find how many products are included in a specific order **along with the name of the customer who placed the order**, we need to join the **orders, orderdetails**, and **customers** tables. The result is shown in the snapshot below. Why I wanted to include a customer name too? Because I observed when analysing the **1.** in the previous slide that the **Euro + Shopping Channel** has the highest total number of order. Which can be an interesting customer to analyse later on. That is why here, I chose an order number that belongs to that very customer. It looks like, this customer buy more than 1 product in each order. I will investigate it in next slide.

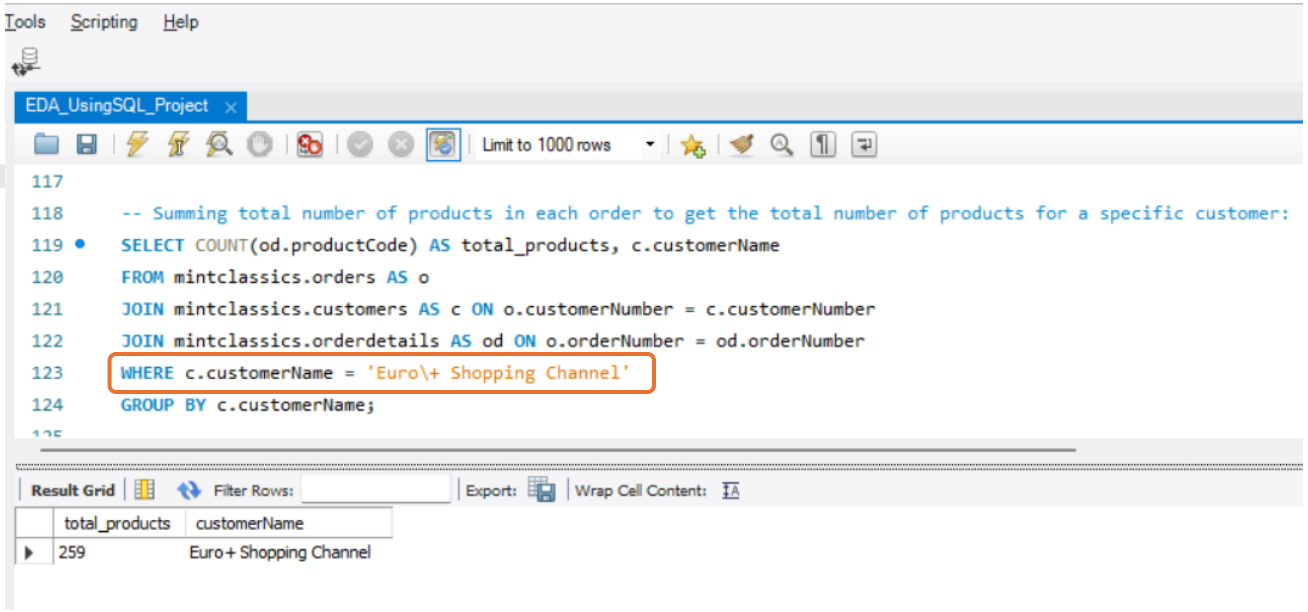




Q: What if I want to know the total number of products for each order placed by a specific customer?

As I stated, I found out that the customer **Euro + Shopping Channel** has placed the highest number of orders, with which I can claim that it orders more products than the others. To find out if this claim is true, I did the analysis considering only the name of this customer. This snapshot on the left shows the total number of products for each order (separately) by **Euro + Shopping Channel.**

I can also check, what is the sum of all products, to find out total number of products that is ordered by **Euro + Shopping Channel.** The snapshot below shows the result – it turned out the number of products that **Euro + Shopping Channel** ordered in total is 259.



Challenge: In the snapshots, you might have observed that I used different syntax in the WHERE clause to filter the result based on a customer name **Euro + Shopping Channel.** But the query wasn't running. The issue is most likely caused by the **+** sign in the name. In SQL, special characters (like **+**) can sometimes cause problems in queries, depending on the database system. I fixed the problem using two different methods:

1. LIKE with wildcards **('%'). % -** represents any sequence of characters before or after the specified word: WHERE **c.customerName** LIKE '%Euro%'

2. escaping the **+** sign using backslashes (****)**:**

WHERE **c.customerName =** 'Euro \+ Shopping Channel'

Pro tip: If there are multiple customers with "Euro" in their name, this method could return unintended results. But if we're sure that there's only **one** match, it's a quick and effective solution!

EDA_UsingSQL_Project* | orders

Limit to 1000 rows

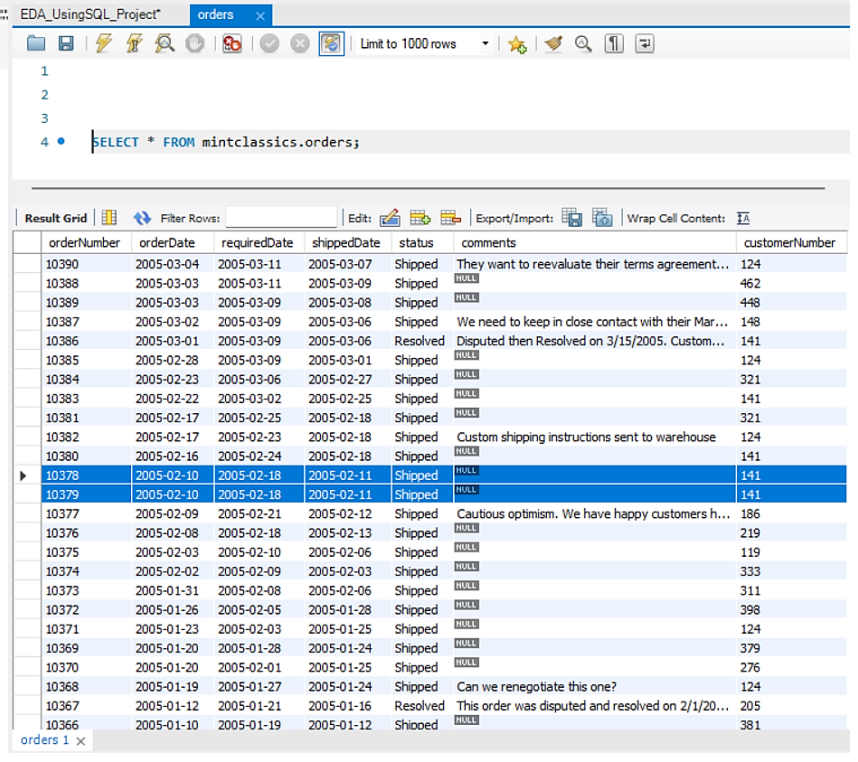
```sql
117
118     -- see if there is any customer who placed multiple orders on the same day:
119 •   SELECT orderDate, customerNumber, COUNT(orderNumber) AS order_count
120     FROM mintclassics.orders
121     GROUP BY orderDate, customerNumber
122     HAVING COUNT(orderNumber) > 1;
123
124     -- The SQL HAVING clause is used to filter the results of a GROUP BY query based on the result of an aggregate function.
125     -- It is similar to the WHERE clause but is specifically applied after grouping and aggregation,
126     -- allowing you to filter on the results of aggregate functions like COUNT, SUM, AVG, and others.
127
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

orderDate	customerNumber	order_count
2005-02-10	141	2

1. **Check how many orders each customer has placed.**
2. **Check how many orders a specific customer has placed.**
3. **Check how many products are included in an order.**
4. ~~**See if any customers placed multiple orders on the same day.**~~

4. See if any customers placed multiple orders on the same day:
Interestiiinngg..The result of this analysis shows that we have one customer (i.e. 141) that placed 2 orders on 2005-02-10. Interestingly, this customer is **Euro + Shopping Channel.** Now, the question is, why only this company placed multiple orders on the same date?



About the Customer with Multiple Orders on the Same Day

That's an interesting insight! Here are a few possible explanations for why this customer placed multiple orders on the same day:

1. **Large or Bulk Orders Split into Multiple Transactions**
    1. The customer might be ordering in large quantities, and the system (or the company's policy) might split orders into smaller chunks.
    2. Maybe there's a limit per order, so they had to place multiple ones.
2. **Separate Orders for Different Products or Departments**
    1. If this customer is a retailer or wholesaler, they might order different product categories separately.
    2. Some companies have internal policies that require separate purchases for different items or departments.
3. **Back-to-Back Orders Due to Stock Availability**
    1. Maybe they initially placed an order, then realized they needed more items and placed another one on the same day.
4. **Technical or System Issues**
    1. There's also a small chance that duplicate or redundant orders were placed accidentally due to a system issue.

What to Do Next?

a) Check **which products** were included in these multiple orders. Are they different, or are there duplicates?

b) Compare **order amounts**—are they similar in size, or is one much larger?

c) Check **time differences**—were the orders placed minutes apart or hours apart?

d) Check the amount of money paid for each product.

e) See if this customer is a **wholesaler or retailer** (if such information is available).

If this pattern is **consistent** (happens regularly), it could indicate a trend worth analysing further!

EDA_UsingSQL_Project | orders

Limit to 1000 rows

```sql
-- What to do next: a) Check which products were included in these multiple orders.
-- Are the products different, or are there duplicates?
SELECT od.productCode, od.quantityOrdered AS orderQuantity,
       c.customerName, o.customerNumber,
       o.orderNumber, o.orderDate,
       p.productName
FROM mintclassics.orders AS o
JOIN mintclassics.customers AS c ON o.customerNumber = c.customerNumber
JOIN mintclassics.orderdetails AS od ON o.orderNumber = od.orderNumber
JOIN mintclassics.products AS p ON p.productCode = od.productCode
WHERE o.customerNumber = 141 AND o.orderDate = '2005-02-10'
ORDER BY o.orderNumber;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productCode	orderQuantity	customerName	customerNumber	orderNumber	orderDate	productName
S18_1589	34	Euro+ Shopping Channel	141	10378	2005-02-10	1965 Aston Martin DB5
S18_3278	22	Euro+ Shopping Channel	141	10378	2005-02-10	1969 Dodge Super Bee
S18_3482	43	Euro+ Shopping Channel	141	10378	2005-02-10	1976 Ford Gran Torino
S18_3782	28	Euro+ Shopping Channel	141	10378	2005-02-10	1957 Vespa GS150
S18_4721	49	Euro+ Shopping Channel	141	10378	2005-02-10	1957 Corvette Convertible
S24_2972	41	Euro+ Shopping Channel	141	10378	2005-02-10	1982 Lamborghini Diablo
S24_3371	46	Euro+ Shopping Channel	141	10378	2005-02-10	1971 Alpine Renault 1600s
S24_3856	33	Euro+ Shopping Channel	141	10378	2005-02-10	1956 Porsche 356A Coupe
S24_4620	41	Euro+ Shopping Channel	141	10378	2005-02-10	1961 Chevrolet Impala
S32_2206	40	Euro+ Shopping Channel	141	10378	2005-02-10	1982 Ducati 996 R
S18_1749	39	Euro+ Shopping Channel	141	10379	2005-02-10	1917 Grand Touring Sedan
S18_2248	27	Euro+ Shopping Channel	141	10379	2005-02-10	1911 Ford Town Car
S18_2870	29	Euro+ Shopping Channel	141	10379	2005-02-10	1999 Indy 500 Monte Carlo SS
S18_3685	32	Euro+ Shopping Channel	141	10379	2005-02-10	1948 Porsche Type 356 Roadster
S24_1628	32	Euro+ Shopping Channel	141	10379	2005-02-10	1966 Shelby Cobra 427 S/C

Result 34

What to Do Next?

Let's analyse the points mentioned above from a) to e):

a) Check **which products** were included in these multiple orders. Are they different, or are there duplicates?

⇒ In the **WHERE** clause, I filtered the date and the customer number because, while analyzing whether multiple orders were placed on the same day, I discovered that a specific customer (customer number **141**) had placed **two** orders on **2005-02-10** (see the slide *Multiple Orders on the Same Day*). To efficiently analyze the **products** ordered by this customer on that date, I incorporated this information into my query.

⇒ Observations: Customer **Euro + Shopping Channel (141)** placed **2 orders** on **2005-02-10.** In total **15 products** are ordered. All products are distinct. Of these, **10 products** were included in order **#10378**, while the remaining **5 products** were part of order **#10379**.

⇒ So, maybe there's a limit in the number of products per order, that's why, they had to place second one **(#10379)** when this limit is exceeded**.**

EDA_UsingSQL_Project* | orders

Limit to 1000 rows

```sql
-- What to Do Next: b) Compare order amounts (total order value)—are they similar in size, or is one much larger?
SELECT o.orderNumber, o.orderDate,
       SUM(od.quantityOrdered * od.priceEach) AS totalOrderValue
FROM mintclassics.orders AS o
JOIN mintclassics.customers AS c ON o.customerNumber = c.customerNumber
JOIN mintclassics.orderdetails AS od ON o.orderNumber = od.orderNumber
WHERE c.customerName = 'Euro\+ Shopping Channel'
GROUP BY o.orderNumber
ORDER BY totalOrderValue;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

orderNumber	orderDate	totalOrderValue
10380	2005-02-16	34404.21
10379	2005-02-10	16621.27
10378	2005-02-10	32289.12
10358	2004-12-10	44185.46
10355	2004-12-07	25529.78
10350	2004-12-02	46493.16
10311	2004-10-16	36140.38
10279	2004-08-09	20009.53
10262	2004-06-24	47065.36
10246	2004-05-05	35420.74
10244	2004-04-29	26155.91
10212	2004-01-16	59830.55
10205	2003-12-03	13059.16
10203	2003-12-02	40062.53
10190	2003-11-19	10721.86
10156	2003-10-08	4599.52
10153	2003-09-28	44939.85
10133	2003-06-27	22366.04

Result 60

b) Compare **order amounts -** are they similar in size, or is one much larger?

Now, let's check the total value of each order by **Euro + Shopping Channel**. If the amounts are similar, it might indicate systematic multiple orders. If one is much larger, it could be a separate large purchase.

EDA_UsingSQL_Project*

Limit to 1000 rows

```sql
-- This query shows the price per product:
SELECT c.customerName, o.customerNumber,
       o.orderNumber, o.orderDate, p.productCode,
       od.priceEach, od.quantityOrdered
FROM mintclassics.orders AS o
JOIN mintclassics.customers AS c ON o.customerNumber = c.customerNumber
JOIN mintclassics.orderdetails AS od ON o.orderNumber = od.orderNumber
JOIN mintclassics.products AS p ON p.productCode = od.productCode
GROUP BY o.orderNumber, c.customerNumber, o.orderDate, c.customerName,
         od.priceEach, od.quantityOrdered, p.productCode
ORDER BY o.orderDate;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customerName	customerNumber	orderNumber	orderDate	productCode	priceEach	quantityOrdered
Land of Toys Inc.	131	10107	2003-02-24	S10_1678	81.35	30
Reims Collectables	353	10121	2003-05-07	S10_1678	86.13	34
Lyon Souveniers	250	10134	2003-07-01	S10_1678	90.92	41
Toys4GrownUps.com	205	10145	2003-08-25	S10_1678	76.56	45
Corporate Gift Ideas Co.	321	10159	2003-10-10	S10_1678	81.35	49
Technics Stores Inc.	161	10168	2003-10-28	S10_1678	94.74	36
Daedalus Designs Imports	171	10180	2003-11-11	S10_1678	76.56	29
Herkku Gifts	167	10188	2003-11-18	S10_1678	95.70	48
Mini Wheels Co.	129	10201	2003-12-01	S10_1678	82.30	22
Baane Mini Imports	121	10103	2003-01-29	S10_1949	214.30	26
Volvo Model Replicas, Co	144	10112	2003-03-24	S10_1949	197.16	29
Corrida Auto Replicas, Ltd	458	10126	2003-05-28	S10_1949	205.73	38
Technics Stores Inc.	161	10140	2003-07-24	S10_1949	186.44	37
Dragon Souveniers, Ltd.	148	10150	2003-09-19	S10_1949	182.16	45
Classic Legends Inc.	424	10163	2003-10-20	S10_1949	212.16	21
Australian Gift Network, Co	333	10174	2003-11-06	S10_1949	207.87	34
Classic Gift Ideas, Inc	339	10183	2003-11-13	S10_1949	180.01	23

Result 6

d) Check the amount of money paid for each product.

Observation:

The price of each product fluctuates across different orders. Since each order is placed by a different customer, this suggests that product prices may vary depending on the customer. Two possible factors influencing these price variations could be the customer's location/country or the date of the order.

For instance, the product with the code **S10_1678** was priced at **$81.35** on **February 24, 2003**. However, by **May 4, 2003**, its price had increased by approximately **$5**, reaching **$86.13**. While the product was ordered by different customers, the most likely factors driving the price change are the purchase date and possibly the region in which it was sold.

EDA_UsingSQL_Project

Limit to 1000 rows

Find - price per product

```sql
242    -- This query shows the price per product:
243    SELECT c.customerName, o.customerNumber,
244           o.orderNumber, o.orderDate, p.productCode, p.productName,
245           od.priceEach -- , od.quantityOrdered
246          -- (od.quantityOrdered * od.priceEach) AS orderPricePerProduct,
247           -- c.country, c.city
248    FROM mintclassics.orders AS o
249    JOIN mintclassics.customers AS c ON o.customerNumber = c.customerNumber
250    JOIN mintclassics.orderdetails AS od ON o.orderNumber = od.orderNumber
251    JOIN mintclassics.products AS p ON p.productCode = od.productCode
252    GROUP BY o.orderNumber, c.customerNumber, o.orderDate, c.customerName,
253             od.priceEach, p.productCode
254    ORDER BY od.priceEach DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customerName	customerNumber	orderNumber	orderDate	productCode	productName	priceEach
Baane Mini Imports	121	10103	2003-01-29	S10_1949	1952 Alpine Renault 1300	214.30
Cambridge Collectables Co.	173	10228	2004-03-10	S10_1949	1952 Alpine Renault 1300	214.30
Mini Gifts Distributors Ltd.	124	10312	2004-10-21	S10_1949	1952 Alpine Renault 1300	214.30
Classic Legends Inc.	424	10163	2003-10-20	S10_1949	1952 Alpine Renault 1300	212.16
Scandinavian Gift Ideas	448	10291	2004-09-08	S10_1949	1952 Alpine Renault 1300	210.01
Australian Gift Network, Co	333	10174	2003-11-06	S10_1949	1952 Alpine Renault 1300	207.87
Corrida Auto Replicas, Ltd	458	10348	2004-11-01	S12_1108	2001 Ferrari Enzo	207.80
Corrida Auto Replicas, Ltd	458	10126	2003-05-28	S10_1949	1952 Alpine Renault 1300	205.73
Québec Home Shopping Network	233	10411	2005-05-01	S10_1949	1952 Alpine Renault 1300	205.73
Amica Models & Co.	249	10280	2004-08-17	S10_1949	1952 Alpine Renault 1300	205.73
West Coast Collectables Co.	475	10215	2004-01-29	S10_1949	1952 Alpine Renault 1300	205.73
Danish Wholesale Imports	145	10105	2003-02-11	S12_1108	2001 Ferrari Enzo	205.72
Lyon Souveniers	250	10395	2005-03-17	S12_1108	2001 Ferrari Enzo	205.72
Super Scale Inc.	455	10196	2003-11-26	S12_1108	2001 Ferrari Enzo	203.64

Result 10

c) Check Time Differences Between Orders on the Same Day

If we have timestamps for orders, we could check the exact times, but since we only have dates (orderDate), we can count the number of orders per day for this customer.

⇒ Uncommenting **HAVING** clause will show only the dates when more than 1 order has been placed by **Euro + Shopping Channel.**

⇒ As we can see, on other dates the customer placed only 1 order (with probably multiple products).

⇒ To show # of orders by each customer on each date, I just needed to uncomment **WHERE** clause where I filtered only one customer.

EDA_UsingSQL_Project* | orders

Limit to 1000 rows

```
230  -- we can count the number of orders per day for this customer:
231  SELECT o.customerNumber, c.customerName, o.orderDate,
232         COUNT(o.orderNumber) AS numberOfOrders
233  FROM mintclassics.orders o
234  JOIN mintclassics.customers c ON o.customerNumber = c.customerNumber
235  WHERE c.customerName = 'Euro\+ Shopping Channel'
236  GROUP BY o.customerNumber, o.orderDate
237  -- HAVING COUNT(o.orderNumber) > 1
238  ORDER BY numberOfOrders DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customerNumber	customerName	orderDate	numberOfOrders
141	Euro+ Shopping Channel	2005-02-10	2
141	Euro+ Shopping Channel	2003-01-31	1
141	Euro+ Shopping Channel	2003-06-06	1
141	Euro+ Shopping Channel	2003-06-27	1
141	Euro+ Shopping Channel	2003-09-28	1
141	Euro+ Shopping Channel	2003-10-08	1
141	Euro+ Shopping Channel	2003-11-19	1
141	Euro+ Shopping Channel	2003-12-02	1
141	Euro+ Shopping Channel	2003-12-03	1
141	Euro+ Shopping Channel	2004-01-16	1
141	Euro+ Shopping Channel	2004-04-29	1
141	Euro+ Shopping Channel	2004-05-05	1
141	Euro+ Shopping Channel	2004-06-24	1
141	Euro+ Shopping Channel	2004-08-09	1
141	Euro+ Shopping Channel	2004-10-16	1
141	Euro+ Shopping Channel	2004-12-02	1
141	Euro+ Shopping Channel	2004-12-07	1
141	Euro+ Shopping Channel	2004-12-10	1
141	Euro+ Shopping Channel	2005-02-16	1
141	Euro+ Shopping Channel	2005-02-22	1

Result 70

EDA_UsingSQL_Project* | orders

Limit to 1000 rows

```
229
230  -- we can count the number of orders per day for this customer:
231  SELECT o.customerNumber, c.customerName, o.orderDate,
232         COUNT(o.orderNumber) AS numberOfOrders
233  FROM mintclassics.orders o
234  JOIN mintclassics.customers c ON o.customerNumber = c.customerNumber
235  -- WHERE c.customerName = 'Euro\+ Shopping Channel'
236  GROUP BY o.customerNumber, o.orderDate
237  -- HAVING COUNT(o.orderNumber) > 1
238  ORDER BY numberOfOrders DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customerNumber	customerName	orderDate	numberOfOrders
141	Euro+ Shopping Channel	2005-02-10	2
363	Online Diecast Creations Co.	2003-01-06	1
128	Blauer See Auto, Co.	2003-01-09	1
181	Vitachrome Inc.	2003-01-10	1
121	Baane Mini Imports	2003-01-29	1
141	Euro+ Shopping Channel	2003-01-31	1
145	Danish Wholesale Imports	2003-02-11	1
278	Rovelli Gifts	2003-02-17	1
131	Land of Toys Inc.	2003-02-24	1
385	Cruz & Sons Co.	2003-03-03	1
486	Motor Mint Distributors Inc.	2003-03-10	1
187	AV Stores, Co.	2003-03-18	1
129	Mini Wheels Co.	2003-03-25	1
144	Volvo Model Replicas, Co	2003-03-24	1
124	Mini Gifts Distributors Ltd.	2003-03-26	1
172	La Corne D'abondance, Co.	2003-04-01	1
424	Classic Legends Inc.	2003-04-04	1
381	Royale Belge	2003-04-11	1
148	Dragon Souveniers, Ltd.	2003-04-16	1

Result 71

Observation:
All the customer have 1 order on each date, except **Euro + Shopping Channel** which has 2, and we already showed this in the previous analysis.

EDA_UsingSQL_Project* | orders | orderdetails

Limit to 1000 rows

```sql
-- What To Do Next: e) See if this customer is a wholesaler or retailer (if such information is available):
SELECT c.customerNumber, c.customerName,
    COUNT(DISTINCT o.orderNumber) AS totalOrders,
    SUM(od.quantityOrdered) AS totalProductsOrdered,
    SUM(od.quantityOrdered * od.priceEach) AS totalSpent,
    c.creditLimit
FROM mintclassics.customers c
JOIN mintclassics.orders o ON c.customerNumber = o.customerNumber
JOIN mintclassics.orderdetails od ON o.orderNumber = od.orderNumber
-- WHERE c.customerName = 'Euro\+ Shopping Channel'
GROUP BY c.customerNumber, c.customerName, c.creditLimit
ORDER BY totalOrders DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customerNumber	customerName	totalOrders	totalProductsOrdered	totalSpent	creditLimit
141	Euro+ Shopping Channel	26	9327	820689.54	227600.00
124	Mini Gifts Distributors Ltd.	17	6366	591827.34	210500.00
114	Australian Collectors, Co.	5	1926	180585.07	117300.00
353	Reims Collectables	5	1433	126983.19	81100.00
145	Danish Wholesale Imports	5	1315	129085.12	83400.00
148	Dragon Souveniers, Ltd.	5	1524	156251.03	103800.00
323	Down Under Souveniers, Inc	5	1691	154622.08	88000.00
381	Royale Belge	4	278	29217.18	23500.00
276	Anna's Decorations, Ltd	4	1469	137034.22	107800.00
119	La Rochelle Gifts	4	1832	158573.12	118200.00
121	Baane Mini Imports	4	1082	104224.79	81700.00
128	Blauer See Auto, Co.	4	811	75937.76	59700.00
131	Land of Toys Inc.	4	1631	149085.15	114900.00
144	Volvo Model Replicas, Co	4	647	66694.82	53100.00
496	Kelly's Gift Shop	4	1647	137460.79	110000.00
151	Muscle Machine Inc	4	1775	177913.95	138500.00
157	Diecast Classics Inc.	4	1111	104358.69	100600.00
161	Technics Stores Inc	4	1179	104545.22	84600.00

Result 64

e) See if this customer is a **wholesaler or retailer** (if such information is available).

Check If This Customer is a Wholesaler or Retailer

We don't have a specific column that tells us whether a customer is a wholesaler, but we can check:

- If their **total order volume** is much higher than other customers.
- If they frequently place **large orders** (high **quantityOrdered**).
- If they have **a high credit limit** (usually assigned to big buyers).

Observation:

⇒ Interestingly, customer with the highest credit limit is **141 – Euro + Shopping Channel**

⇒ The **COUNT** function in Power BI is an aggregation method that calculates the total number of occurrences of a specific value or data point in a given dataset. This feature is very beneficial when understanding the frequency or distribution of certain values within your data.

EDA_UsingSQL_Project

Limit to 1000 rows

```sql
-- There is multiple/different products under a single order(number):
SELECT c.customerName,
       o.orderNumber,
       p.productCode
FROM mintclassics.customers c
JOIN mintclassics.orders o ON c.customerNumber = o.customerNumber
JOIN mintclassics.orderdetails od ON o.orderNumber = od.orderNumber
JOIN mintclassics.products p ON p.productCode = od.productCode
ORDER BY c.customerName DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customerName	orderNumber	productCode
Euro+ Shopping Channel	10104	S12_3148
Euro+ Shopping Channel	10104	S12_4473
Euro+ Shopping Channel	10104	S18_2238
Euro+ Shopping Channel	10104	S18_2319
Euro+ Shopping Channel	10104	S18_3232
Euro+ Shopping Channel	10104	S18_4027
Euro+ Shopping Channel	10104	S24_1444
Euro+ Shopping Channel	10104	S24_2840
Euro+ Shopping Channel	10104	S24_4048
Euro+ Shopping Channel	10104	S32_2509
Euro+ Shopping Channel	10104	S32_3207
Euro+ Shopping Channel	10104	S50_1392
Euro+ Shopping Channel	10104	S50_1514
Euro+ Shopping Channel	10128	S18_3140
Euro+ Shopping Channel	10128	S18_3259
Euro+ Shopping Channel	10128	S18_4522
Euro+ Shopping Channel	10128	S700_1938
Euro+ Shopping Channel	10133	S18_2581
Euro+ Shopping Channel	10133	S24_1785

Result 2

Challenge

Without DISTINCT, just because one order can contain multiple products, the corresponding order numbers appear multiple times (in the table) for each product. This cause them to be counted multiple times. Solution: Using DISTINCT helps us count the number of different orders placed by each customer.

EDA_UsingSQL_Project* | orders | orderdetails

Limit to 1000 rows

```sql
-- Using DISTINCT help us to count the number of different orders placed by each customer.
SELECT c.customerName,
       COUNT(o.orderNumber) AS totalOrder
FROM mintclassics.customers c
JOIN mintclassics.orders o ON c.customerNumber = o.customerNumber
JOIN mintclassics.orderdetails od ON o.orderNumber = od.orderNumber
GROUP BY c.customerName
ORDER BY totalOrder DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customerName	totalOrder
Euro+ Shopping Channel	259
Mini Gifts Distributors Ltd.	180
Australian Collectors, Co.	55
La Rochelle Gifts	53
AV Stores, Co.	51
Land of Toys Inc.	49
Muscle Machine Inc	48
Kelly's Gift Shop	48
Rovelli Gifts	48
Souveniers And Things Co.	46
Anna's Decorations, Ltd	46
Down Under Souveniers, Inc	46
Dragon Souveniers, Ltd.	43
Reims Collectables	41
Saveley & Henriot, Co.	41
Corporate Gift Ideas Co.	41
Salzburg Collectables	40
The Sharp Gifts Warehouse	40
L'ordine Souveniers	39
Scandinavian Gift Ideas	38
Danish Wholesale Imports	36
Handji Gifts& Co	36

Result 56

EDA_UsingSQL_Project* | orders | orderdetails

Limit to 1000 rows

```
309     -- Using DISTINCT help us to count the number of different orders placed by each customer.
310 •   SELECT c.customerName,
311            COUNT(DISTINCT o.orderNumber) AS totalOrder
312     FROM mintclassics.customers c
313     JOIN mintclassics.orders o ON c.customerNumber = o.customerNumber
314     JOIN mintclassics.orderdetails od ON o.orderNumber = od.orderNumber
315     GROUP BY c.customerName
316     ORDER BY totalOrder DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customerName	totalOrder
Euro+ Shopping Channel	26
Mini Gifts Distributors Ltd.	17
Danish Wholesale Imports	5
Australian Collectors, Co.	5
Down Under Souveniers, Inc	5
Dragon Souveniers, Ltd.	5
Reims Collectables	5
Technics Stores Inc.	4
Land of Toys Inc.	4
Anna's Decorations, Ltd	4
Souveniers And Things Co.	4
Diecast Classics Inc.	4
Salzburg Collectables	4
Royale Belge	4
Baane Mini Imports	4
Muscle Machine Inc	4
Blauer See Auto, Co.	4
Kelly's Gift Shop	4
Handji Gifts& Co	4
Volvo Model Replicas, Co	4
Tokyo Collectables, Ltd	4
The Sharp Gifts Warehouse	4

Result 57

I used DISTINCT here to solve problem which might have led us to wrong insights about the business.

What to Visualize in Power BI?

Now, let's move on to visualizing these insights!

Total Orders per Customer

- This will highlight how **Euro + Shopping Channel** compares to other customers.

Total Products Ordered per Customer

- This will help confirm if Euro + Shopping Channel orders **way more products** than others.

Orders per Day for Euro + Shopping Channel

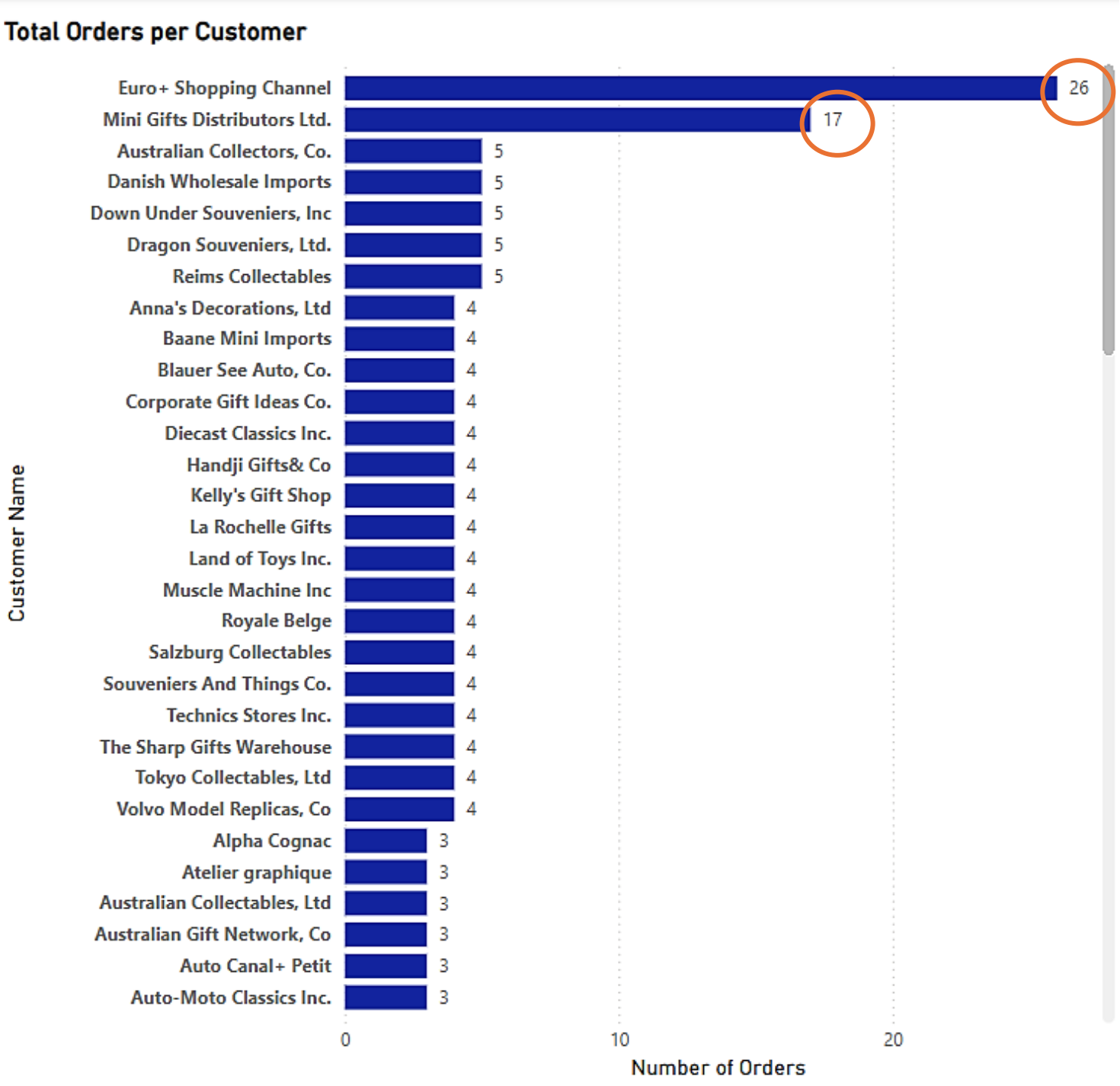
- To visualize the **ordering pattern over time**.

Most Ordered Products by Euro + Shopping Channel

- To check **which products they frequently purchase**.
- This could reveal if they focus on specific items.

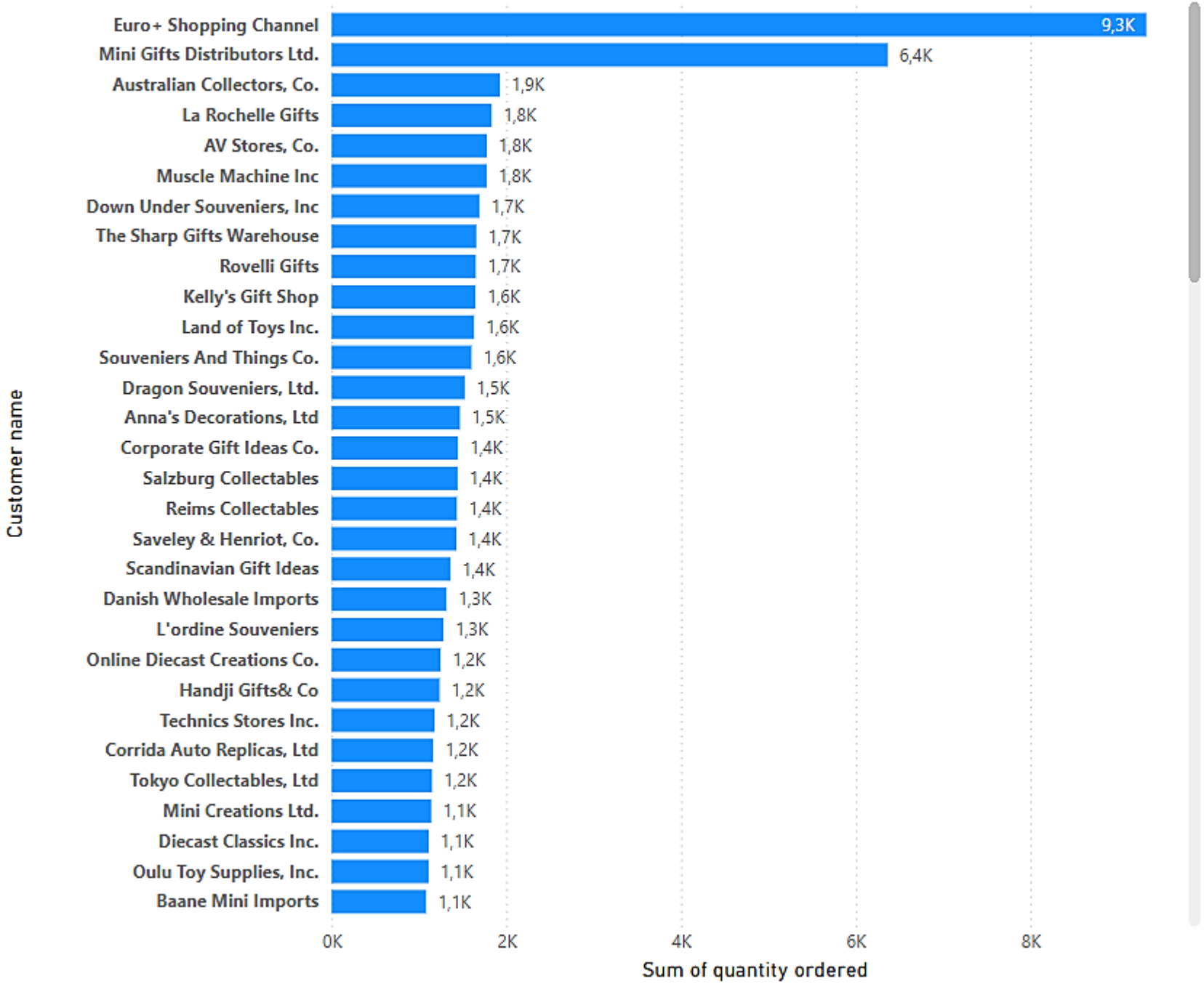
Customer Credit Limit vs. Total Orders

- This will show whether customers with **higher credit limits tend to order more**.



This will highlight how **Euro + Shopping Channel** compares to other customers.
Sorted in **descending order** to highlight the customers with the most orders.
It looks like **Mini Gifts Distributors Ltd.** might be a potential wholesaler, too.

Total Product Quantity per Customer



customerName	Sum of quantityOrdered
Euro+ Shopping Channel	9327
Mini Gifts Distributors Ltd.	6366
Australian Collectors, Co.	1926
La Rochelle Gifts	1832
AV Stores, Co.	1778
Muscle Machine Inc	1775
Down Under Souveniers, Inc	1691
The Sharp Gifts Warehouse	1656
Rovelli Gifts	1650
Kelly's Gift Shop	1647
Land of Toys Inc.	1631
Souveniers And Things Co.	1601
Dragon Souveniers, Ltd.	1524
Anna's Decorations, Ltd	1469
Corporate Gift Ideas Co.	1447
Salzburg Collectables	1442
Reims Collectables	1433
Saveley & Henriot, Co.	1428
Scandinavian Gift Ideas	1359
Danish Wholesale Imports	1315
L'ordine Souveniers	1280
Online Diecast Creations Co.	1248
Handji Gifts& Co	1236
Technics Stores Inc.	1179
Corrida Auto Replicas, Ltd	1163
Tokyo Collectables, Ltd	1150
Mini Creations Ltd.	1140
Diecast Classics Inc.	1111
Oulu Toy Supplies, Inc.	1110
Baane Mini Imports	1082
Total	**105516**

This visualization and the corresponding table will help confirm if **Euro + Shopping Channel** orders **way more products** than the others.

EDA_UsingSQL_Project* | orders | orderdetails

Limit to 1000 rows

```sql
-- Most ordered products in general, i.e by each customer. To check which products they frequently purchase.
-- This way we can also find out top-selling product:
SELECT p.productName, SUM(od.quantityOrdered) AS totalQuantitySold
FROM mintclassics.orders AS o
JOIN mintclassics.customers AS c ON o.customerNumber = c.customerNumber
JOIN mintclassics.orderdetails AS od ON o.orderNumber = od.orderNumber
JOIN mintclassics.products AS p ON p.productCode = od.productCode
-- WHERE c.customerName = 'Euro\+ Shopping Channel'
GROUP BY  p.productName
ORDER BY totalQuantitySold DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productName	totalQuantitySold
1992 Ferrari 360 Spider red	1808
1937 Lincoln Berline	1111
American Airlines: MD-11S	1085
1941 Chevrolet Special Deluxe Cabriolet	1076
1930 Buick Marquette Phaeton	1074
1940s Ford truck	1061
1969 Harley Davidson Ultimate Chopper	1057
1957 Chevy Pickup	1056
1964 Mercedes Tour Bus	1053
1956 Porsche 356A Coupe	1052
Corsair F4U (Bird Cage)	1051
F/A 18 Hornet 1/72	1047
1980s Black Hawk Helicopter	1040
1913 Ford Model T Speedster	1038
1997 BMW R 1100 S	1033
1972 Alfa Romeo GTA	1030
1962 Volkswagen Microbus	1029
2002 Suzuki XREO	1028
The USS Constitution Ship	1020
2001 Ferrari Enzo	1019

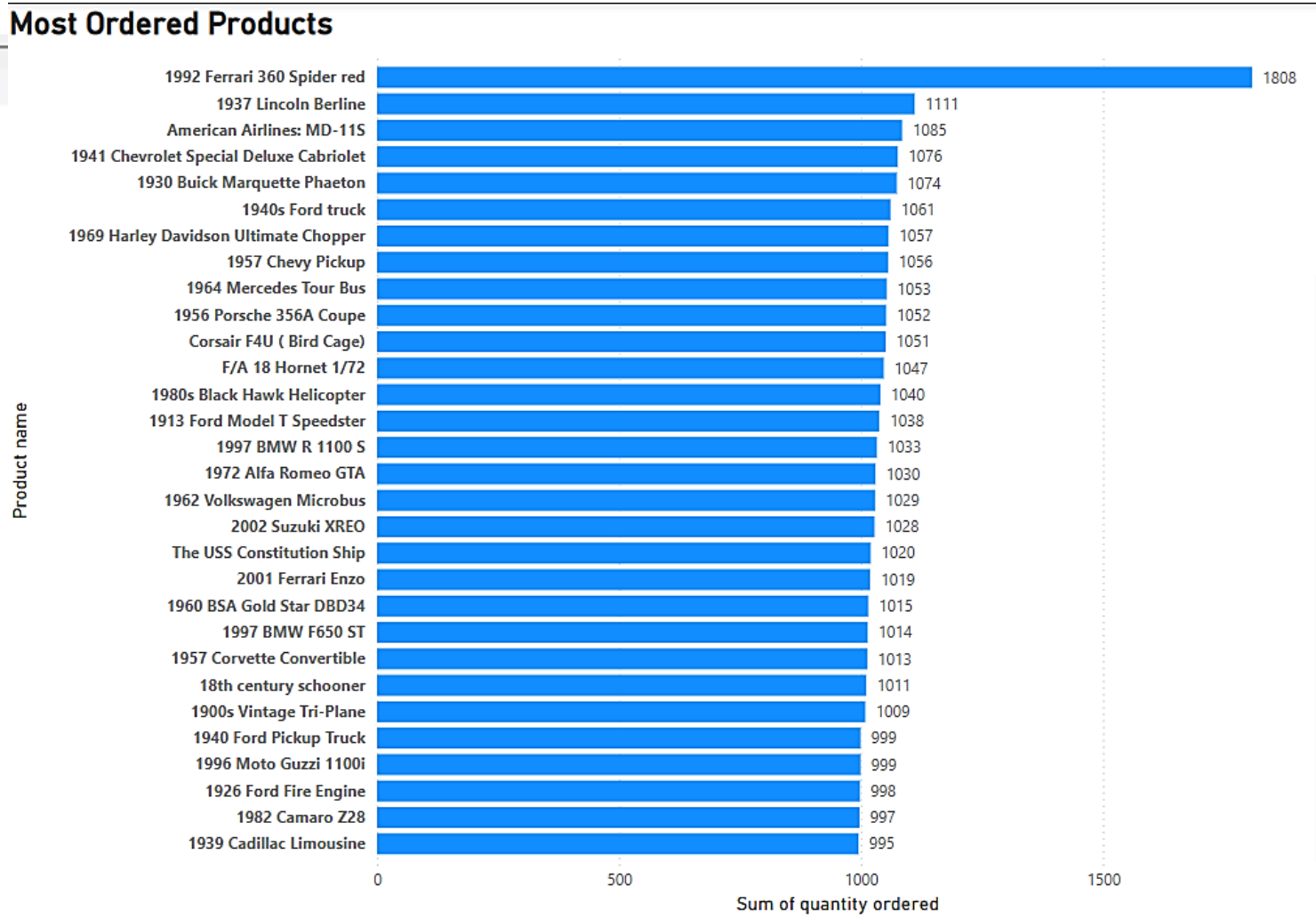
Note: to see the most ordered products by **Euro+ Shopping Channel,** simple uncomment the **WHERE** clause I've written above.

Most Ordered Products in general/by each customer

- To check **which products they frequently purchase**.
- This could reveal if they focus on specific items.

They indeed focus on a specific product: **1992 Ferrari 360 Spider red**

So, we can tell that this product is the most in-demand product among all customers.



EDA_UsingSQL_Project | orders | orderdetails

Limit to 1000 rows

```sql
278  -- How many spesific product is ordered by a specific customer over time?
279  SELECT p.productName, SUM(od.quantityOrdered) AS orderQuantity, o.orderDate
280  FROM mintclassics.orders AS o
281  JOIN mintclassics.customers AS c ON o.customerNumber = c.customerNumber
282  JOIN mintclassics.orderdetails AS od ON o.orderNumber = od.orderNumber
283  JOIN mintclassics.products AS p ON p.productCode = od.productCode
284  WHERE c.customerName = 'Euro\+ Shopping Channel' AND
285          p.productName LIKE '%Ferrari%'
286  GROUP BY  p.productName, o.orderDate
287  ORDER BY orderQuantity DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productName	orderQuantity	orderDate
1992 Ferrari 360 Spider red	60	2005-05-03
1992 Ferrari 360 Spider red	48	2003-12-02
1992 Ferrari 360 Spider red	47	2005-02-22
1992 Ferrari 360 Spider red	40	2004-01-16
1992 Ferrari 360 Spider red	36	2004-05-05
1992 Ferrari 360 Spider red	32	2004-12-10
1992 Ferrari 360 Spider red	23	2003-01-31
1992 Ferrari 360 Spider red	22	2005-03-15
2001 Ferrari Enzo	20	2003-09-28

Check:

I also wanted to check the total number of most in-demand product, **1992 Ferrari 360 Spider red,** that is ordered by **Euro + Shopping Channel** on each **date.**

Another thing to observe in the SQL query is that I filtered the name of the product using **LIKE** with wildcards (%), which is quick and simple, but problem can occur if there is other product names with '**Ferrari**' in it, for example, **2001 Ferrari Enzo**. I had that problem. It turned out there are two product names with 'Ferrari'. So, it might not be a perfect way of filtering in general, but in my situation, it did not hurt, so I left it as it is.

EDA_UsingSQL_Project* | orders | orderdetails

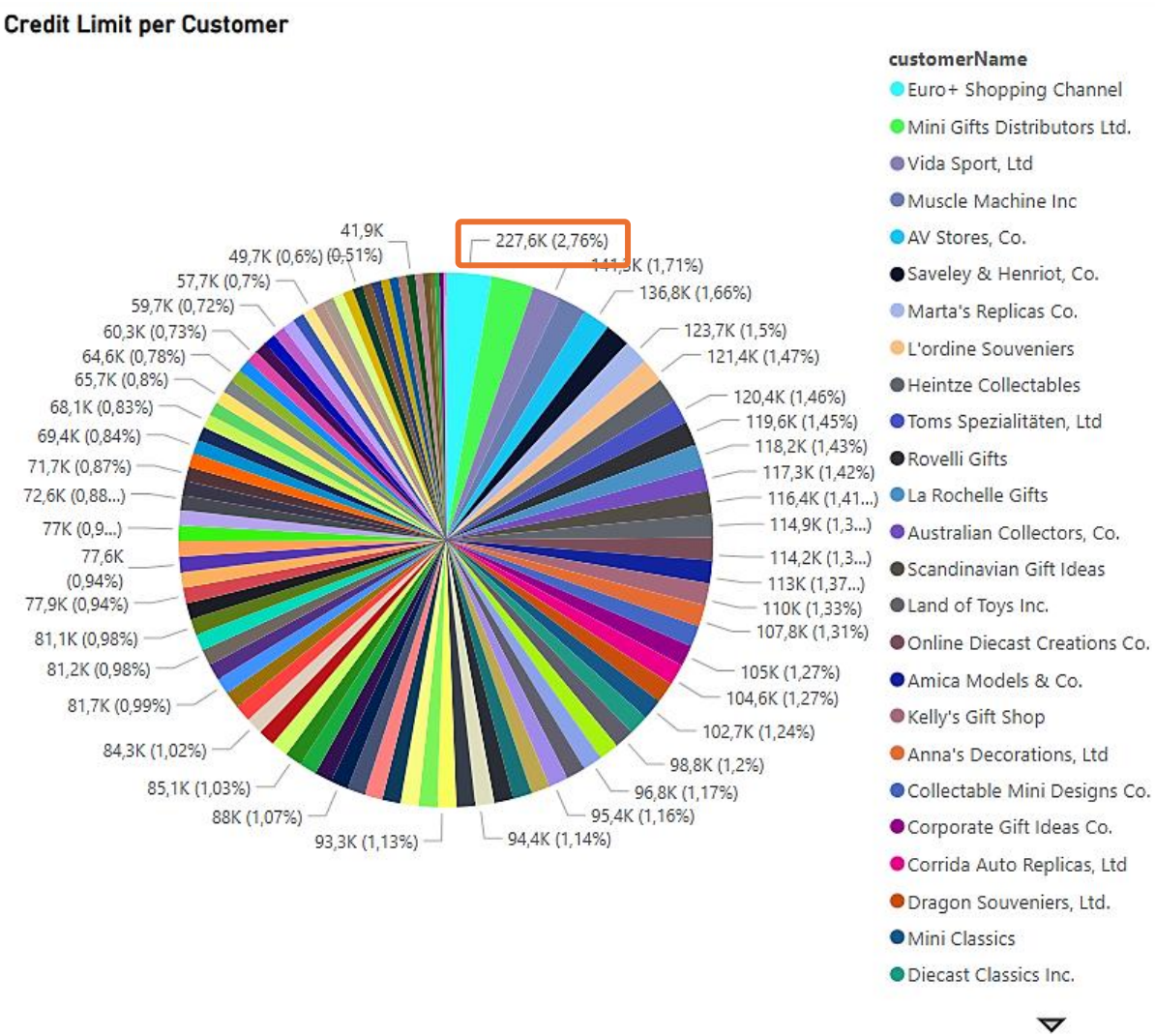
Limit to 1000 rows

```sql
288  -- Credit limit vs Total orders:
289  SELECT c.customerName, COUNT(o.orderNumber) AS totalOrders, c.creditLimit
290  FROM mintclassics.orders AS o
291  JOIN mintclassics.customers AS c ON c.customerNumber = o.customerNumber
292  GROUP BY c.customerName, c.creditLimit
293  ORDER BY c.creditLimit DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customerName	totalOrders	creditLimit
Euro+ Shopping Channel	26	227600.00
Mini Gifts Distributors Ltd.	17	210500.00
Vida Sport, Ltd	2	141300.00
Muscle Machine Inc	4	138500.00
AV Stores, Co.	3	136800.00
Saveley & Henriot, Co.	3	123900.00
Marta's Replicas Co.	2	123700.00
L'ordine Souveniers	3	121400.00
Heintze Collectables	2	120800.00
Toms Spezialitäten, Ltd	2	120400.00
Rovelli Gifts	3	119600.00
La Rochelle Gifts	4	118200.00
Australian Collectors, Co.	5	117300.00
Scandinavian Gift Ideas	3	116400.00
Land of Toys Inc.	4	114900.00
Online Diecast Creations ...	3	114200.00
Amica Models & Co.	2	113000.00
Kelly's Gift Shop	4	110000.00
Anna's Decorations, Ltd	4	107800.00
Corporate Gift Ideas Co.	4	105000.00
Collectable Mini Designs ...	2	105000.00
Corrida Auto Replicas, Ltd	3	104600.00
Dragon Souveniers, Ltd.	5	103800.00

Result 32



Customer Credit Limit vs. Total Orders

This will show whether customers with **higher credit limits tend to order more**.

Two customers with the highest credit limit tend to place more orders than the other customers with less credit limit. See the pie chart.

EDA_UsingSQL_Project* | orders | orderdetails

Limit to 1000 rows

```sql
-- Total Revenue per Customer: This query shows the total order amount/price per order of each customer.
SELECT c.customerName,
       SUM(od.quantityOrdered * od.priceEach) AS totalOrderPrice
FROM mintclassics.orders AS o
JOIN mintclassics.customers AS c ON o.customerNumber = c.customerNumber
JOIN mintclassics.orderdetails AS od ON o.orderNumber = od.orderNumber
GROUP BY c.customerName
ORDER BY totalOrderPrice DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customerName	totalOrderPrice
Euro+ Shopping Channel	820689.54
Mini Gifts Distributors Ltd.	591827.34
Australian Collectors, Co.	180585.07
Muscle Machine Inc	177913.95
La Rochelle Gifts	158573.12
Dragon Souveniers, Ltd.	156251.03
Down Under Souveniers, Inc	154622.08
Land of Toys Inc.	149085.15
AV Stores, Co.	148410.09
The Sharp Gifts Warehouse	143536.27
Salzburg Collectables	137480.07
Kelly's Gift Shop	137460.79
Anna's Decorations, Ltd	137034.22
Souveniers And Things Co.	133907.12
Corporate Gift Ideas Co.	132340.78
Saveley & Henriot, Co.	130305.35
Danish Wholesale Imports	129085.12
Rovelli Gifts	127529.69
Reims Collectables	126983.19
L'ordine Souveniers	125505.57

Result 35

What this query does:

- Joins **customers**, **orders**, and **orderdetails** tables.
- Calculates **total revenue,** (quantityOrdered * priceEach), for each customer.
- Groups the data by customer name.
- Sorts the result in **descending order** (highest revenue first)

Observations: as expected, **Euro Shopping Channel** and **Mini Gifts Distributors Ltd.** have the highest total revenue out of all customers. Our business earns a lot of money especially through these two customers. Because the total order placed by these very customers is also the highest.

EDA_UsingSQL_Project | orders | orderdetails

Limit to 1000 rows

```
328
329     -- Total Revenue per Product:
330 •   SELECT
331         p.productName,
332         SUM(od.quantityOrdered * od.priceEach) AS totalRevenue
333     FROM mintclassics.products p
334     JOIN mintclassics.orderdetails od ON p.productCode = od.productCode
335     GROUP BY p.productName
336     ORDER BY totalRevenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productName	totalRevenue
1992 Ferrari 360 Spider red	276839.98
2001 Ferrari Enzo	190755.86
1952 Alpine Renault 1300	190017.96
2003 Harley-Davidson Eagle Drag Bike	170686.00
1968 Ford Mustang	161531.48
1969 Ford Falcon	152543.02
1980s Black Hawk Helicopter	144959.91
1998 Chrysler Plymouth Prowler	142530.63
1917 Grand Touring Sedan	140535.60
2002 Suzuki XREO	135767.03
1956 Porsche 356A Coupe	134240.71
1969 Corvair Monza	132363.79
1928 Mercedes-Benz SSK	132275.98
1957 Corvette Convertible	130749.31
1972 Alfa Romeo GTA	127924.32
1962 LanciaA Delta 16V	123123.01
1970 Triumph Spitfire	122254.75
1976 Ford Gran Torino	121890.60
1948 Porsche Type 356 Roadster	121653.46
1958 Setra Bus	119085.25
1995 Honda Civic	119050.95

Result 65

1992 Ferrari 360 Spider red is the product that our business receives the most revenue from. I was expecting this, because this very product is also the most frequently ordered product.

EDA_UsingSQL_Project | orders | orderdetails

Limit to 1000 rows

```
283     -- Most Profitable Products:
284 •   SELECT p.productName,
285            SUM(od.quantityOrdered * (od.priceEach - p.buyPrice)) AS totalProfit
286     FROM mintclassics.products p
287     JOIN mintclassics.orderdetails od ON p.productCode = od.productCode
288     GROUP BY p.productName
289     ORDER BY totalProfit DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productName	totalProfit
1992 Ferrari 360 Spider red	135996.78
1952 Alpine Renault 1300	95282.58
2001 Ferrari Enzo	93349.65
2003 Harley-Davidson Eagle Drag Bike	81031.30
1968 Ford Mustang	72579.26
1969 Ford Falcon	72399.77
1928 Mercedes-Benz SSK	68423.18
2002 Suzuki XREO	67641.47
1980s Black Hawk Helicopter	64599.11
1948 Porsche Type 356 Roadster	62725.78
1917 Grand Touring Sedan	60945.00
1957 Corvette Convertible	59910.22
1962 Volkswagen Microbus	55655.47
1976 Ford Gran Torino	54647.25
1932 Model A Ford J-Coupe	54026.65
1999 Indy 500 Monte Carlo SS	52240.32
1903 Ford Model A	51219.92
1957 Chevy Pickup	51127.01
1940 Ford Pickup Truck	47291.45
1969 Corvair Monza	46521.97
1969 Dodge Charger	45950.46
ATA: B757-300	45058.29
1958 Setra Bus	43366.45

Result 68

Most Profitable products

The query I wrote calculates the **total profit** for each product.

To determine the **most profitable products**, we need:
Total Profit → (quantityOrdered * (priceEach - buyPrice))
Observation: Again, as expected, **1992 Ferrari 360 Spider red** is the most profitable product for Mint Classics. To check the least profitable one, just remove DESC so we have ascending order; then the least profitable will appear first in the list.

EDA_UsingSQL_Project | orders | orderdetails | productlines | warehouses | products

Limit to 1000 rows

```sql
385  -- How many product lines does each warehouse have:
386  SELECT  w.warehouseName,
387          p.productLine,
388          COUNT(p.productLine) AS productLineCount
389  FROM mintclassics.products p
390  JOIN mintclassics.warehouses w ON p.warehouseCode = w.warehouseCode
391  GROUP BY w.warehouseName, p.productLine
392  ORDER BY productLineCount DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

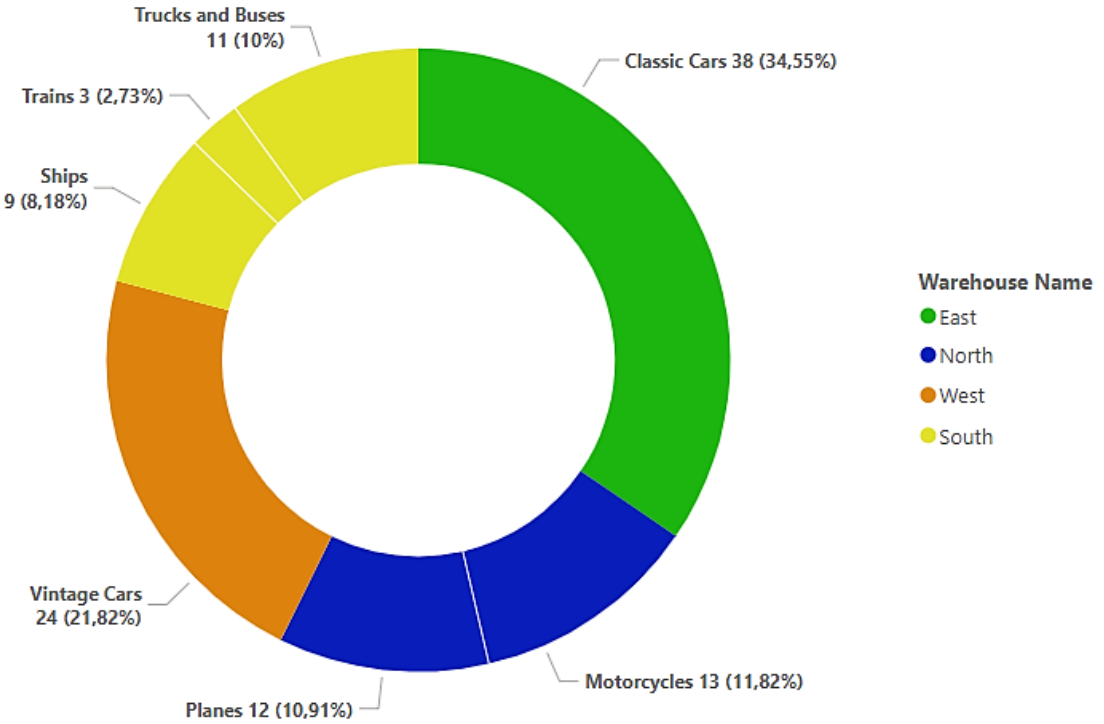
warehouseName	productLine	productLineCount
East	Classic Cars	38
West	Vintage Cars	24
North	Motorcycles	13
North	Planes	12
South	Trucks and Buses	11
South	Ships	9
South	Trains	3

With the SQL query got what I wanted – information about the amount of product lines in our warehouses. The aim was to understand the warehouse system of Mint Classics. I visualized this using Donut chart in Power BI. Each slice here represents the number of each product line and its percentage of total.

To get familiar with the dataset I want to analyse warehouses:

- How to analyse warehouses?
- What do warehouses contain?
- What kind of products?
- What information might be useful to solve the business problem we have?
- What insight we can potentially gain by knowing the number of product lines in each warehouse?

Number of Product Lines per Warehouse



EDA_UsingSQL_Project | orders | orderdetails

```
290
291     -- Warehouse Utilization:
292 •   SELECT warehouseName, warehousePctCap
293     FROM mintclassics.warehouses;
294
```

Result Grid | Filter Rows: | Export:

warehouseName	warehousePctCap
South	75
North	72
East	67
West	50

Warehouse Utilization

To analyse warehouse utilization, we need:

- **Percentage Capacity Used** → warehousePctCap (already available in warehouses table).

What this query does:

- Retrieves warehouse names and their **percentage capacity usage.**

EDA_UsingSQL_Project | orders | orderdetails

Limit to 1000 rows

```sql
-- Stock levels: this query shows how many stocks each product line have. In total, there is 7 product lines.
-- Classic cars are the one with the highest stock level:
SELECT  w.warehouseName,
        p.productLine,
        SUM(p.quantityInStock) AS totalStock
FROM mintclassics.products p
JOIN mintclassics.warehouses w ON p.warehouseCode = w.warehouseCode
GROUP BY w.warehouseName, p.productLine
ORDER BY totalStock DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouseName	productLine	totalStock
East	Classic Cars	219183
West	Vintage Cars	124880
North	Motorcycles	69401
North	Planes	62287
South	Trucks and Buses	35851
South	Ships	26833
South	Trains	16696

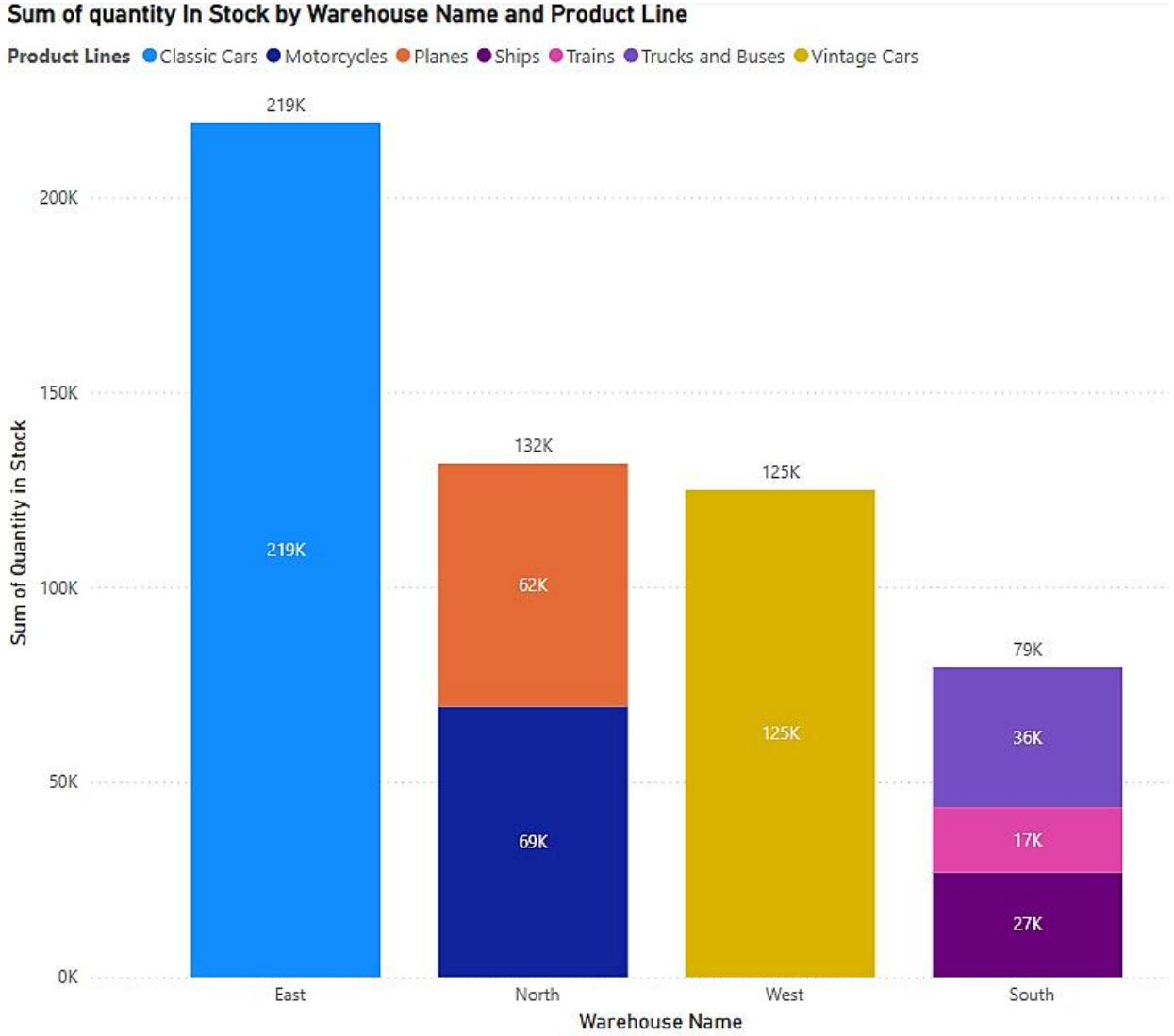
What this query does:

- Retrieves stock levels for each **product line** in each **warehouse**.
- Groups data by **warehouse** and **product line**.
- Sorts by **warehouse name** and **total stock (descending order)**.

Stock levels: this query shows how many stocks each product line have. In total, there is 7 product lines. **Classic cars** are the one with the highest stock levels and based only in **East Warehouse**. The column chart on the right visualize what I wrote in the SQL query. **Trains** has the least stock level that is based in **South Warehouse.**

To analyze stock levels, we need:

- warehouseName (from warehouses)
- productLine (from productlines)
- quantityInStock (from products)



EDA_UsingSQL_Project* | orders | orderdetails | productlines | warehouses | products | SQL File 4*

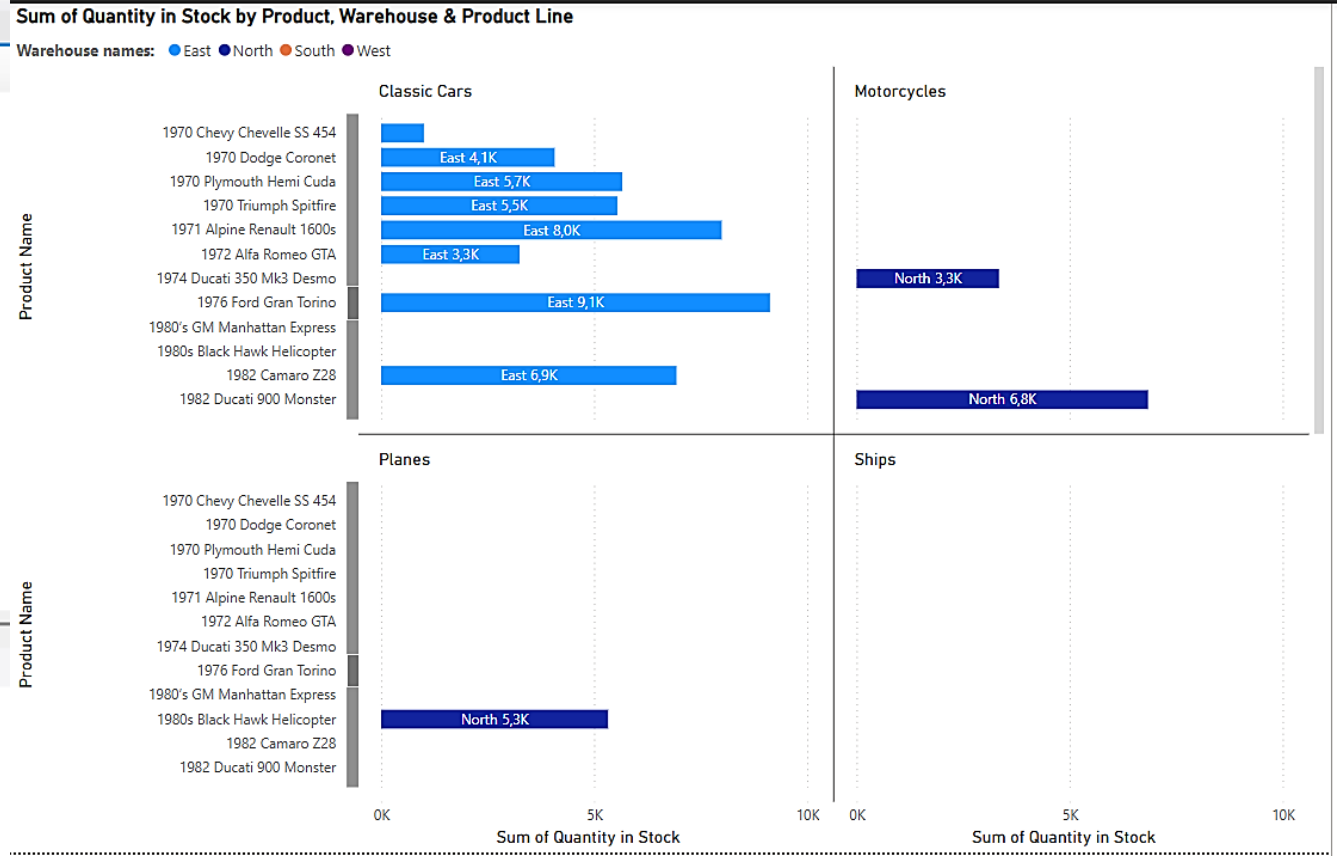
Limit to 1000 rows

```sql
-- This query shows which product belongs to which product line,
-- where the products are stored, and how many stocks of each product we have in total:
SELECT  w.warehouseName,
        w.warehouseCode,
        w.warehousePctCap,
        p.productLine,
        p.productName,
        SUM(p.quantityInStock) AS totalStock
FROM mintclassics.products p
JOIN mintclassics.warehouses w ON p.warehouseCode = w.warehouseCode
JOIN mintclassics.productlines pl ON p.productLine = pl.productLine
-- WHERE productName LIKE '%red%'
-- WHERE p.productLine = 'Classic Cars'
GROUP BY w.warehouseName, p.productLine, p.productName, w.warehousePctCap, w.warehouseCode
ORDER BY totalStock DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouseName	warehouseCode	warehousePctCap	productLine	productName	totalStock
North	a	72	Motorcycles	2002 Suzuki XREO	9997
East	b	67	Classic Cars	1995 Honda Civic	9772
North	a	72	Planes	America West Airlines B757-200	9653
East	b	67	Classic Cars	2002 Chevy Corvette	9446
West	c	50	Vintage Cars	1932 Model A Ford J-Coupe	9354
North	a	72	Motorcycles	1982 Ducati 996 R	9241
West	c	50	Vintage Cars	1912 Ford Model T Delivery Wagon	9173
East	b	67	Classic Cars	1976 Ford Gran Torino	9127
East	b	67	Classic Cars	1968 Dodge Charger	9123
East	b	67	Classic Cars	1965 Aston Martin DB5	9042
East	b	67	Classic Cars	1948 Porsche Type 356 Roadster	8990
East	b	67	Classic Cars	1948 Porsche 356-A Roadster	8826
North	a	72	Planes	American Airlines: MD-11S	8820

Result 95



Tip: To see the low stock products, basically, remove DESC in the last line of the query.

Result: **2002 Suzuki XREO** is the product with high stock level of **9997**, whereas the **1960 BSA Gold Star DBD34** has the lowest stock level of **15**. **What should this mean? ->check it.**

Visualization: I preferred Stacked bar chart from Power BI to have information I want to analyse altogether. Only part of the chart is shown in the snapshot, the whole chart is available in my project folder. Basically, scroll down/up to see the details for each product.

EDA_UsingSQL_Project | orders | orderdetails | productlin

Limit to 1000

```
409  -- To find 10 Most Stocked Products:
410  SELECT  productName, quantityInStock
411  FROM mintclassics.products
412  ORDER BY quantityInStock DESC
413  LIMIT 10;
```

Result Grid | Filter Rows: | Export:

productName	quantityInStock
2002 Suzuki XREO	9997
1995 Honda Civic	9772
America West Airlines B757-200	9653
2002 Chevy Corvette	9446
1932 Model A Ford J-Coupe	9354
1982 Ducati 996 R	9241
1912 Ford Model T Delivery Wagon	9173
1976 Ford Gran Torino	9127
1968 Dodge Charger	9123
1965 Aston Martin DB5	9042

EDA_UsingSQL_Project* | orders | orderdetails | productlin

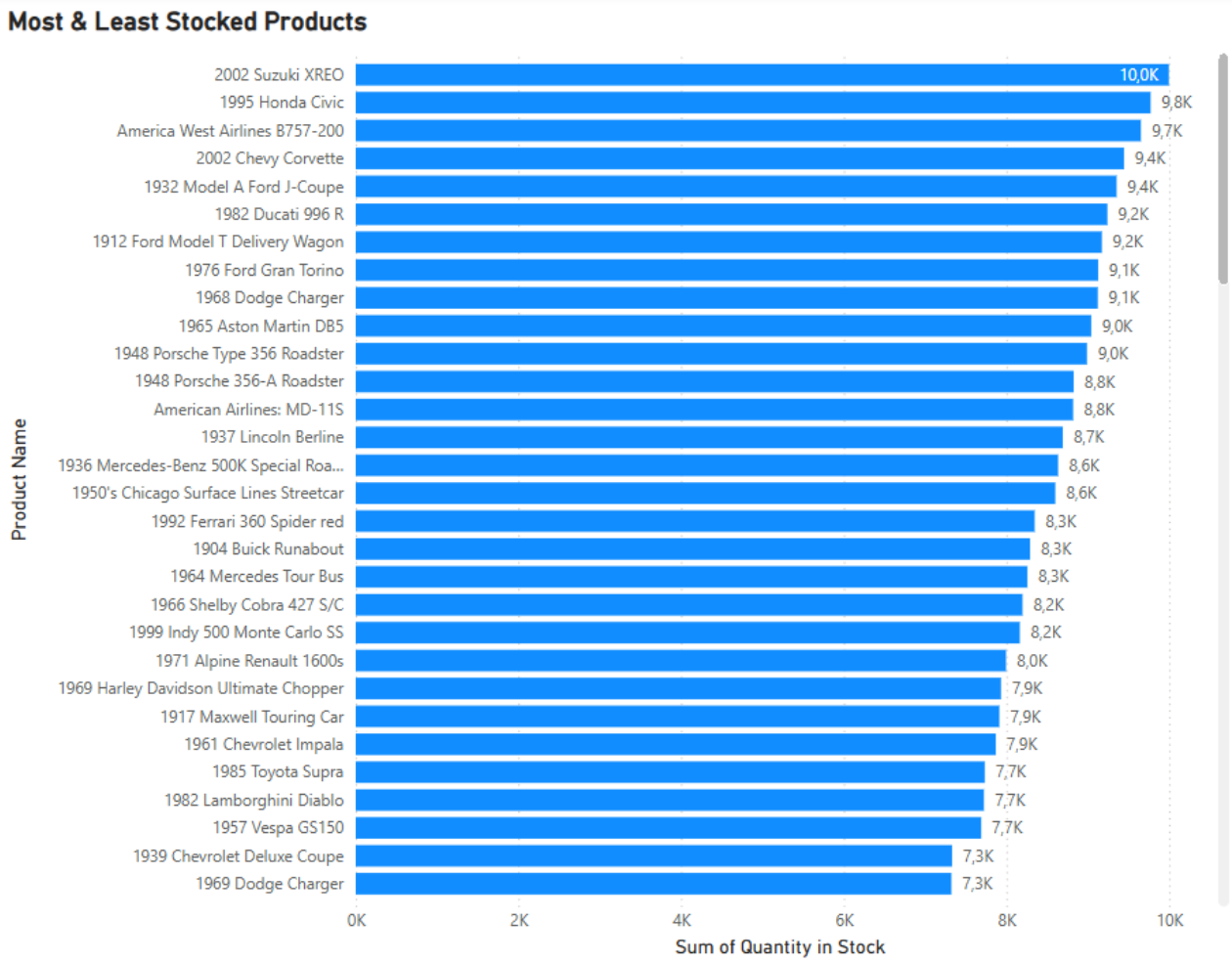
Limit to 1000

```
409  -- To find 10 Least Stocked Products:
410  SELECT  productName, quantityInStock
411  FROM mintclassics.products
412  ORDER BY quantityInStock
413  LIMIT 10;
```

Result Grid | Filter Rows: | Export:

productName	quantityInStock
1960 BSA Gold Star DBD34	15
1968 Ford Mustang	68
1928 Ford Phaeton Deluxe	136
1997 BMW F650 ST	178
Pont Yacht	414
1911 Ford Town Car	540
1928 Mercedes-Benz SSK	548
F/A 18 Hornet 1/72	551
2002 Yamaha YZR M1	600
The Mayflower	737

Visualization: to see the low stock product in the clustered bar chart, basically, scroll down.



What this query does:

- Retrieves the **top 10 products** with the **highest and the lowest stock levels**.
- Orders them **from highest to lowest** and **lowest to highest** stock quantity.

MSRP, Profit Margin & Markup Percentage

What Does MSRP Mean?

- **MSRP = Manufacturer's Suggested Retail Price**
- It's the price that the manufacturer recommends selling the product for.
- The difference between **buyPrice** and **MSRP** could show **profit margins**.
- You can also calculate the **markup percentage** if you know cost price (buyPrice) and MSRP:

Formula for Markup Percentage

$$\text{Markup Percentage} = \left(\frac{\text{MSRP} - \text{buyPrice}}{\text{buyPrice}}\right) \times 100$$

This shows the percentage increase from **buyPrice** to **MSRP**.

Example Calculation

Let's say we have a product with:

- **buyPrice = $50**
- **MSRP = $80**

$$\text{Markup Percentage} = \left(\frac{80 - 50}{50}\right) \times 100 = 60\%$$

This means the **MSRP is 60% higher than the buy price.**

We can also check which warehouse the products with higher markup percentage belong to => this way we can get an idea about which warehouse is important and should not to be closed. We can also check the products with the less markup percentage, then we can see what can be done to increase the profit, or it can also give idea about if we can close the warehouses they belong to.

The **markup percentage** tells us how much a product's price is increased from its cost price (**buyPrice**) to its selling price (**MSRP**). We can calculate the markup percentage per product using SQL queries easily. In the query, I also included the warehouse code to know where these products belong to:

EDA_UsingSQL_Project* | orders | orderdetails | productlines | warehouses | products | SQL File 4*

Limit to 1000 rows

```
415    -- Calculating markup percentage
416    -- to see how much a product's price is increased from its cost price (buyPrice) to its selling price (MSRP):
417 •  SELECT  productName, warehouseCode, buyPrice, MSRP,
418            (((MSRP - buyPrice)/ buyPrice)*100) AS markupPercentage
419    FROM mintclassics.products
420    ORDER BY markupPercentage DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productName	warehouseCode	buyPrice	MSRP	markupPercentage
1961 Chevrolet Impala	b	32.33	80.84	150.046397
1970 Plymouth Hemi Cuda	b	31.92	79.80	150.000000
1937 Horch 930V Limousine	c	26.30	65.75	150.000000
1936 Harley Davidson El Knucklehead	a	24.23	60.57	149.979364
1926 Ford Fire Engine	d	24.92	60.77	143.860353
2002 Yamaha YZR M1	a	34.17	81.36	138.103600
Corsair F4U (Bird Cage)	a	29.34	68.24	132.583504
1928 Mercedes-Benz SSK	c	72.56	168.75	132.566152
1950's Chicago Surface Lines Streetcar	d	26.72	62.14	132.559880
1999 Indy 500 Monte Carlo SS	b	56.76	132.00	132.558140
1982 Lamborghini Diablo	b	16.24	37.76	132.512315
1948 Porsche Type 356 Roadster	b	62.16	141.28	127.284427
2002 Suzuki XREO	a	66.27	150.62	127.282330
HMS Bounty	d	39.83	90.52	127.265880
1958 Chevy Corvette Limited Edition	b	15.91	35.36	122.250157
1936 Mercedes-Benz 500K Special Roadster	c	24.26	53.91	122.217642
1939 Cadillac Limousine	c	23.14	50.31	117.415730
1932 Model A Ford J-Coupe	c	58.48	127.13	117.390561
1952 Alpine Renault 1300	b	98.58	214.30	117.386894
2001 Ferrari Enzo	b	95.59	207.80	117.386756
1982 Camaro Z28	b	46.53	101.15	117.386632
1992 Ferrari 360 Spider red	b	77.90	169.34	117.381258
1938 Cadillac V-16 Presidential Limousine	c	20.61	44.80	117.370209
1957 Corvette Convertible	b	69.93	148.80	112.784213

Result 101

EDA_UsingSQL_Project* | orders | orderdetails | productlines | warehouses | products | SQL File 4*

Limit to 1000 rows

```sql
-- Calculating markup percentage
-- to see how much a product's price is increased from its cost price (buyPrice) to its selling price (MSRP):
SELECT  productName, warehouseCode, buyPrice, MSRP,
        (MSRP - buyPrice) AS profitMargin,
        CEILING(((MSRP - buyPrice)/ buyPrice)*100) +'%' AS markupPercentage
FROM mintclassics.products
ORDER BY markupPercentage DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productName	warehouseCode	buyPrice	MSRP	profitMargin	markupPercentage
1961 Chevrolet Impala	b	32.33	80.84	48.51	151
1970 Plymouth Hemi Cuda	b	31.92	79.80	47.88	150
1936 Harley Davidson El Knucklehead	a	24.23	60.57	36.34	150
1937 Horch 930V Limousine	c	26.30	65.75	39.45	150
1926 Ford Fire Engine	d	24.92	60.77	35.85	144
2002 Yamaha YZR M1	a	34.17	81.36	47.19	139
1928 Mercedes-Benz SSK	c	72.56	168.75	96.19	133
1999 Indy 500 Monte Carlo SS	b	56.76	132.00	75.24	133
1982 Lamborghini Diablo	b	16.24	37.76	21.52	133
Corsair F4U (Bird Cage)	a	29.34	68.24	38.90	133
1950's Chicago Surface Lines Streetcar	d	26.72	62.14	35.42	133
2002 Suzuki XREO	a	66.27	150.62	84.35	128
1948 Porsche Type 356 Roadster	b	62.16	141.28	79.12	128
HMS Bounty	d	39.83	90.52	50.69	128
1936 Mercedes-Benz 500K Special Roadster	c	24.26	53.91	29.65	123
1958 Chevy Corvette Limited Edition	b	15.91	35.36	19.45	123
1952 Alpine Renault 1300	b	98.58	214.30	115.72	118

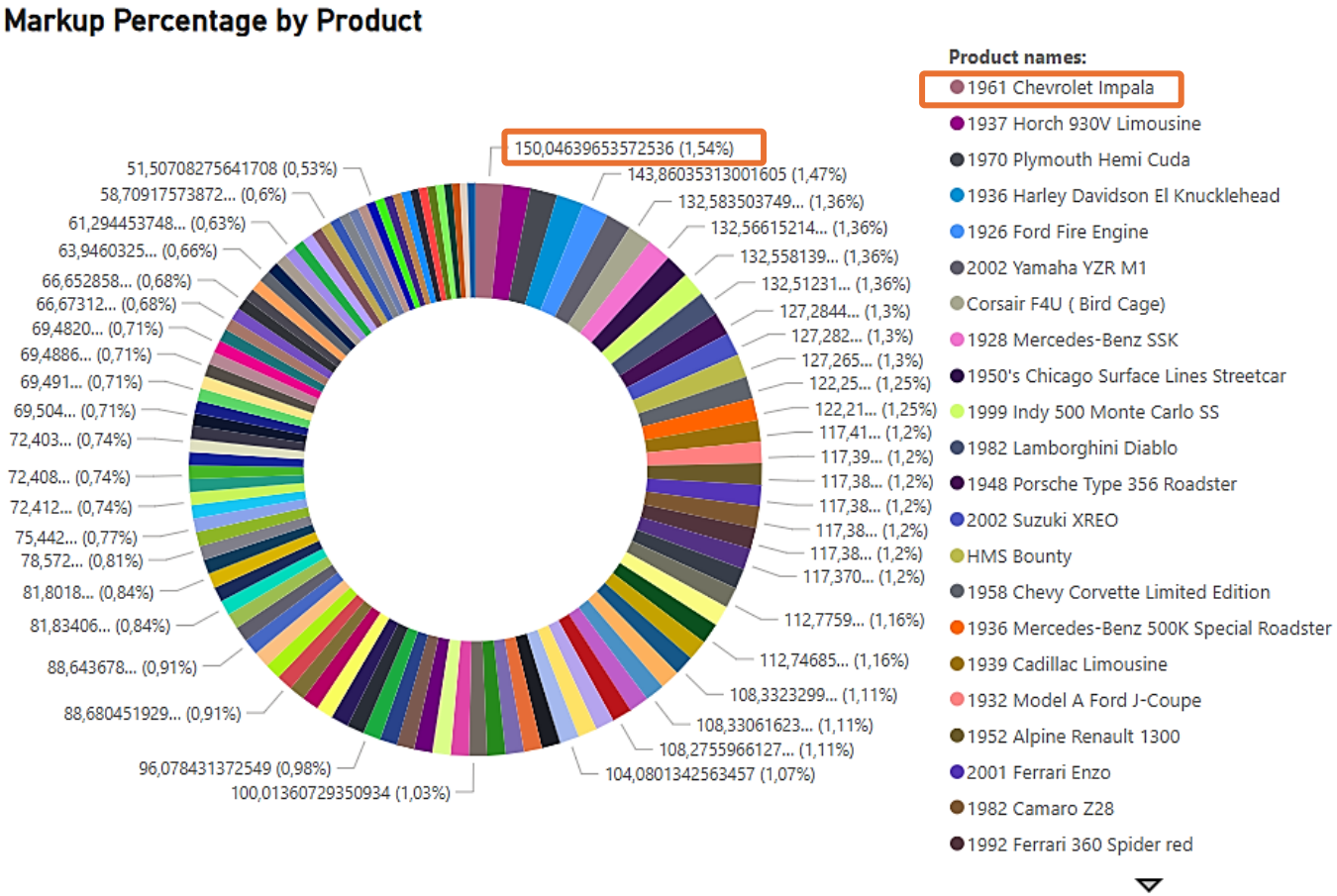
Result 118

Visualization: I used donut chart for the visualization of markup percentage per product.

Challenge: Calculating markup percentage. The formula I used in **DAX (Data Analysis Expressions)** for Power BI was: MarkupPercentage = ('mintclassics products'[MSRP] - 'mintclassics products'[buyPrice]) / 'mintclassics products'[buyPrice] * 100.

I tried to do this calculation inside a measure. But it did not work, turned out that the error occurs because **Power BI requires explicit aggregation** when performing calculations inside a measure. Since 'mintclassics products'[MSRP] and 'mintclassics products'[buyPrice] contain multiple rows, Power BI doesn't know which single value to use.

Solution: I solved the problem by using a **calculated column** instead of creating a **measure**. This works because calculated columns evaluate row by row and don't require aggregation.



Margins can never be more than 100 percent, but markups can be 200 percent, 500 percent, or 10,000 percent, depending on the price and the total cost of the offer. The higher your price and the lower your cost, the higher your markup.

For example, we marked the product **1961 Chevrolet Impala** up 151 %.

I couldn't make a percentage sign to appear in the result, will check it later.

CEILING aggregate function is used to round the results, because they were too long, and not reader friendly (it was containing too many number for stakeholders from different departments ☺).

EDA_UsingSQL_Project | orders | orderdetails | productlines | warehouses | products

Limit to 1000 rows

```
432
433     -- To analyze total stock per warehouse:
434 •   SELECT  w.warehouseName,
435             SUM(p.quantityInStock) AS totalStockPerWarehouse
436     FROM mintclassics.products p
437     JOIN mintclassics.warehouses w ON p.warehouseCode = w.warehouseCode
438     GROUP BY w.warehouseName
439     ORDER BY totalStockPerWarehouse DESC;
440
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouseName	totalStockPerWarehouse
East	219183
North	131688
West	124880
South	79380

What this query does:

- Joins the products table with warehouses to get the **warehouse names**.
- Sums up quantityInStock for each warehouse.
- Orders the results from **highest to lowest stock**.

Understanding **why** we're doing each analysis is just as important as knowing **how** to do it. Here's why this step - **Total Stock per Warehouse -** matters in our project.

Why Analyze Total Stock per Warehouse?

Project Goal Reminder:
Mint Classics Company is considering closing one of its warehouses. Our job is to **help them make a data-driven decision** on which warehouse (if any) should be closed or optimized.

How This Analysis Helps:
1. Identifies Stock Distribution:

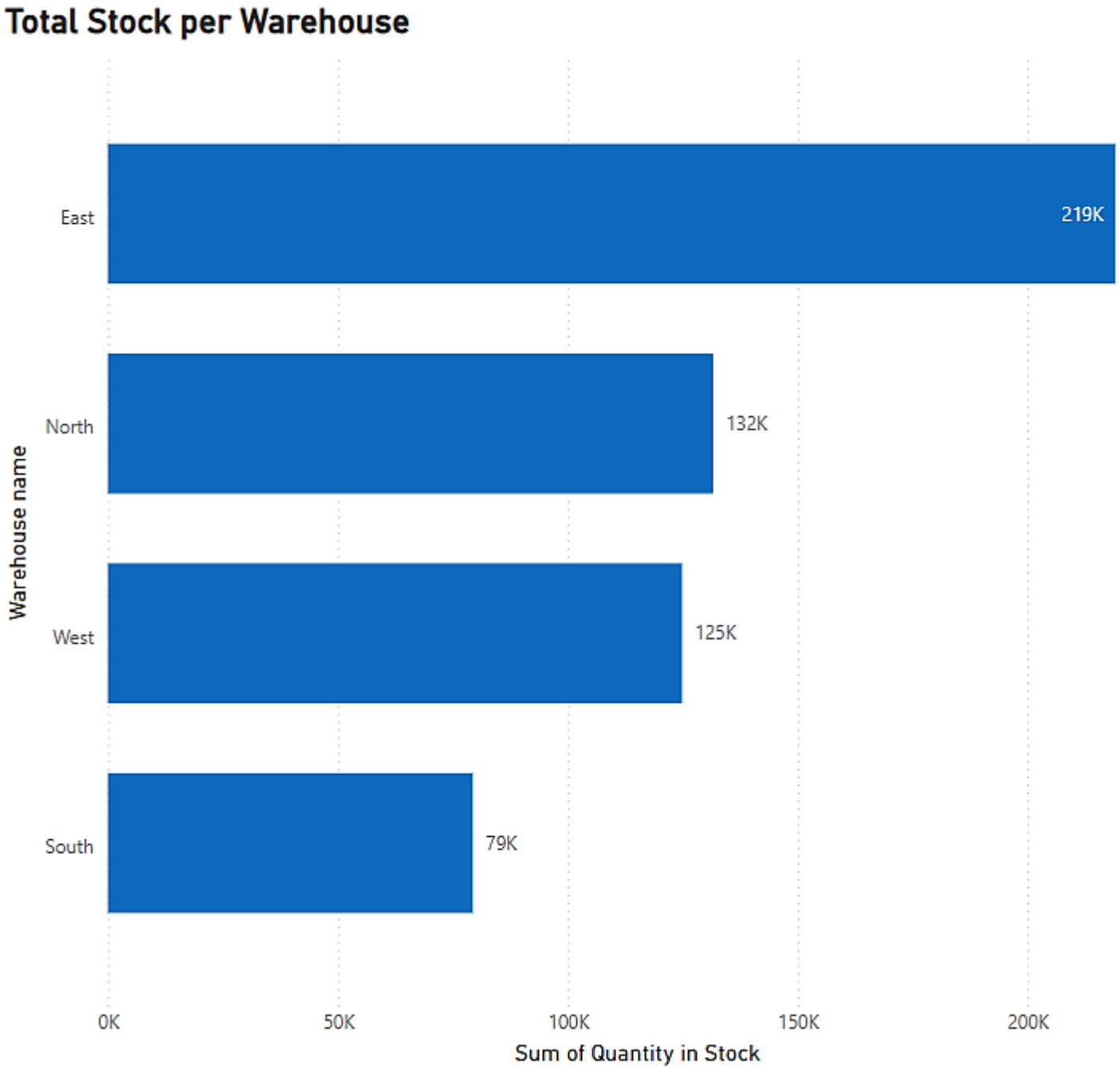
- Some warehouses may be **overloaded**, while others may be **underutilized**.
- If a warehouse has **very little stock**, it might be a **candidate for closure**.

2. Improves Inventory Management:

- If too much stock is concentrated in one warehouse, it can cause **logistics inefficiencies**.
- Finding a **balanced distribution** helps improve **storage and delivery** efficiency.

3. Ensures Customer Satisfaction:

- The company wants to **ship orders within 24 hours**.
- If a warehouse is **understocked**, it might not be able to **fulfil orders quickly**, leading to **delays**.



What We Should Look For in the Chart?

I created the **Total Stock per Warehouse** chart in Power BI. Now we need to consider these:

- **Which warehouse holds the most inventory?** (Is it overloaded?)
- **Which warehouse has the least inventory?** (Is it underutilized?)
- **Are stocks evenly distributed, or is there an imbalance?**

If a warehouse has **very little stock**, it could indicate:

- It's **not needed** anymore.
- It's **not receiving enough shipments**.
- Products should be **redistributed** to other warehouses.

This insight will help us **decide if closing or reorganizing a warehouse is a good move.**

Trends that I noticed:

It turns out that the East warehouse holds the highest stock, with 219K units, while the South warehouse has the lowest, at 79K. The North and West warehouses have 132K and 125K units, respectively.

Now, let's **interpret** these findings and figure out what they mean for the business decision.

What does this tell us?

Total stock per warehouse summary:

- **East Warehouse** → **219K** (Highest stock)
- **North Warehouse** → **132K**
- **West Warehouse** → **125K**
- **South Warehouse** → **79K** (Lowest stock)

Key observations:

East warehouse has the most stock (219K), which could mean:

- It is a **primary hub** for inventory storage.
- It might handle **higher-demand products**.
- It could be **overloaded**, making logistics harder.

South warehouse has the least stock (79K), which raises questions:

- Is it **underutilized**?
- Does it hold **low-demand products**?
- Could we **redistribute its stock to other warehouses** and close it?

North & West warehouses have nearly equal stock (~130K), suggesting:

- They **play a balanced role** in inventory storage.
- They might **support different regions/customers**.

What we need to investigate next

Before recommending any warehouse closures, we should explore:

- **Stock Turnover by Warehouse** → Are some warehouses selling their stock faster than others? => if yes, then this warehouse is very efficient, and there's no need to close it.
- **Most & Least Sold Products by Warehouse** → Does the South warehouse store low-demand products?
- **Warehouse Efficiency (Capacity Usage %)** → Does South warehouse operate far below capacity?

Interpreting the results

- **High Turnover Ratio** (efficient warehouse):
  → Products sell quickly, suggesting **high demand & good inventory management**.
- **Low Turnover Ratio** (potential issue):
  → Products move slowly, meaning the warehouse might be **overstocked or dealing with low-demand items**.

If **South Warehouse has a low stock turnover ratio**, this could be **a strong reason to close it**. But before coming to that decision, let's check if this is indeed true.

Stock Turnover by Warehouse

We'll analyze **how fast products are being sold from each warehouse**. This will help us determine if South warehouse is just **low in stock but efficient** or if it is **truly underperforming and should be closed**.

Analysis: We need to calculate **Stock Turnover**, which measures how quickly inventory is being sold and replaced. A higher turnover indicates **efficient inventory movement**, while a lower turnover might suggest **overstocking or low demand**.

Stock Turnover can be calculated as:

$$Stock\ Turnover = \frac{Total\ Products\ Sold\ from\ a\ Warehouse}{Total\ Stock\ in\ that\ Warehouse}$$

Where:

- **Total Products Sold from a Warehouse** → Sum of **quantityOrdered** from **orderdetails**, grouped by **warehouseCode**.
- **Total Stock in that Warehouse** → **quantityInStock** from **products**, grouped by **warehouseCode**.

We first extract the necessary data from MySQL:

EDA_UsingSQL_Project | orders | orderdetails | productlines | warehouses | products

Limit to 1000 rows

```
442     -- Stock Turnover by Warehouse:
443 •   SELECT
444         p.warehouseCode,
445         w.warehouseName,
446         SUM(od.quantityOrdered) AS totalProductsSold,
447         SUM(p.quantityInStock) AS totalStock,
448         (SUM(od.quantityOrdered) / NULLIF(SUM(p.quantityInStock), 0)) AS stockTurnoverRatio
449     FROM mintclassics.products p
450     JOIN mintclassics.orderdetails od ON p.productCode = od.productCode
451     JOIN mintclassics.warehouses w ON p.warehouseCode = w.warehouseCode
452     GROUP BY p.warehouseCode, w.warehouseName
453     ORDER BY stockTurnoverRatio DESC;
454
455
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouseCode	warehouseName	totalProductsSold	totalStock	stockTurnoverRatio
d	South	22351	2186871	0.0102
a	North	24650	3659553	0.0067
c	West	22933	3439570	0.0067
b	East	35582	5844033	0.0061

What to do next?

To make a final recommendation, we should investigate:

- **Shipping times** – Does closing a specific warehouse affect the 24-hour shipping goal?
- **Storage costs** – Which warehouse is the most expensive to operate?
- **Product demand** – Are some warehouses holding too much stock of low-demand products?

Key Observations:

1.Total Products Sold

a. The **East warehouse (b)** has sold the highest number of products (**35,582**).
b. The **South warehouse (d)** has sold the lowest number of products (**22,351**).

2.Total Stock

a. The **East warehouse (b)** holds the highest stock (**5.84M**).
b. The **South warehouse (d)** holds the lowest stock (**2.18M**).

3.Stock Turnover Ratio (Total Products Sold / Total Stock)

a. The **South warehouse (d)** has the highest turnover ratio (**0.0102**), meaning it is selling its stock the fastest relative to its inventory.
b. The **East warehouse (b)** has the lowest turnover ratio (**0.0061**), meaning it holds a lot of stock but sells at a slower rate.
c. The **North (a) and West (c) warehouses** have the same turnover ratio (**0.0067**), slightly better than East but not as good as South.

What this means:

- The **South warehouse** is the most efficient in selling stock, which suggests it might not be the best candidate for closure.
- The **East warehouse** holds the most stock but has the lowest turnover, but it's not dealing with the low-demand products, so this could indicate **overstocking or inefficient distribution**.
- **North and West warehouses** have similar performance but are still not as efficient as South.

Next Steps:

1. **Interpret the results** – Which warehouse has the fastest/slowest shipping?
2. **Visualize in Power BI** – A bar chart of **average shipping time per warehouse**.
3. **Decide how this affects our closure recommendation**.

EDA_UsingSQL_Project* | orders | orderdetails | productlines | warehouses | products

Limit to 1000 rows

```
456
457     -- Shipping time-Does closing a specific warehouse affect the 24-hour shipping goal?
458     -- This query calculates shipping time:
459     SELECT  p.warehouseCode, w.warehouseName,
460             ROUND(AVG(DATEDIFF(o.shippedDate, o.orderDate)), 2) AS avgShippingTime
461     FROM mintclassics.orders o
462     JOIN mintclassics.orderdetails od ON o.orderNumber = od.orderNumber
463     JOIN mintclassics.products p ON od.productCode = p.productCode
464     JOIN mintclassics.warehouses w ON p.warehouseCode = w.warehouseCode
465     WHERE o.shippedDate IS NOT NULL  -- Ensures we only count shipped orders
466     GROUP BY p.warehouseCode, w.warehouseName
467     ORDER BY avgShippingTime;
468
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouseCode	warehouseName	avgShippingTime
c	West	3.52
a	North	3.89
b	East	3.96
d	South	4.26

I can investigate why the South warehouse is slower → Does it have fewer employees, inefficient routes, or a bottleneck in stock?

Key takeaways from the results:

- **West warehouse (3.52 days)** has the **fastest shipping time**.
- **South warehouse (4.26 days)** has the **slowest shipping time**.
- The other two warehouses (**North: 3.89 days, East: 3.96 days**) are in the middle.

The differences between **West** and **South** aren't huge but could impact customer satisfaction.

What does this mean for the business?

Since **timely shipping is a priority**, closing a warehouse with a **longer shipping time** could be an option. However, we need to balance this with **stock levels and demand** before making a decision.

Next Steps:

1.Visualize the data in Power BI

1. Create a **bar chart** of **average shipping time per warehouse**.
2. This will help us clearly see the differences.

2.Compare with inventory & sales

1. We've already seen that **South has the lowest stock, the lowest number of products sold** and **one of the lowest turnover ratios**.
2. Now, it also has the **slowest shipping time**.
3. Could this be a strong case for **closing the South warehouse**?

NOTE: There are some orders with the **status** "on hold", "cancelled" and "in process" which have NULL shippedDate.

The DATEDIFF Issue (Negative Values)
Problem: The function calculates **shippedDate - orderDate**, but if shippedDate is before orderDate, you get negative values.
Why? This could happen if some orders haven't shipped yet (i.e., shippedDate is NULL) or if there are data entry errors. In our case, it is the status of the orders.

Analyse Order Volume by Warehouse

We already have **avgShippingTime per warehouse** from SQL. Let's move to **order volume per warehouse** next:

EDA_UsingSQL_Project* | totalInventoryPerWarehouse

Limit to 1000 rows

```
466
467     -- Analyzing order volume by warehouse - How many orders are handled by each warehouse?
468     SELECT  p.warehouseCode, w.warehouseName,
469             COUNT(DISTINCT o.orderNumber) AS totalOrders
470     FROM mintclassics.orders o
471     JOIN mintclassics.orderdetails od ON o.orderNumber = od.orderNumber
472     JOIN mintclassics.products p ON od.productCode = p.productCode
473     JOIN mintclassics.warehouses w ON p.warehouseCode = w.warehouseCode
474     GROUP BY p.warehouseCode, w.warehouseName
475     ORDER BY totalOrders DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouseCode	warehouseName	totalOrders
b	East	209
c	West	187
d	South	145
a	North	116

Compare Warehouse Capacity vs. Shipping Time

EDA_UsingSQL_Project* | totalInventoryPerWarehouse

Limit to 1000 rows

```
476
477     -- Retrieving warehouse capacity
478     -- this will give us the percentage of warehouse capacity used:
479     SELECT  w.warehouseCode,
480             w.warehouseName,
481             w.warehousePctCap AS capacityUsage
482     FROM mintclassics.warehouses w;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

warehouseCode	warehouseName	capacityUsage
a	North	72
b	East	67
c	West	50
d	South	75
NULL	NULL	NULL

Insights:

- Warehouses with higher capacity usage tend to have longer shipping times.
- West warehouse is operating efficiently with low capacity usage and faster shipping.
- East warehouse, despite handling the most orders, maintains moderate efficiency.
- South warehouse may need improvements (e.g., inventory redistribution, increased space, or process optimization).

Now we can analyze the **warehouse capacity vs. shipping time** to see if warehouses with higher capacity utilization have longer shipping times.

Now that we have **capacity usage** for each warehouse, let's analyze how it correlates with **average shipping time** and **total orders**.

Observations Based on the Data:

1.South warehouse has the highest capacity usage (75%) and the longest average shipping time (4.26 days).

⇒ This suggests that higher capacity usage might lead to delays.

2.North warehouse also has a high capacity usage (72%) and a relatively long shipping time (3.89 days).

⇒ This reinforces the idea that storage congestion could be slowing down shipping.

3.East warehouse has the highest number of orders (209) but a lower capacity usage (67%) and a moderate shipping time (3.96 days).

⇒ This suggests it handles high demand efficiently compared to others.

4.West warehouse has the lowest capacity usage (50%) and one of the shortest shipping times (3.52 days).

⇒ This indicates that having extra storage space may help speed up order processing.

EDA_UsingSQL_Project | totalInventoryPerWarehouse

Limit to 1000 rows

```sql
-- identifying slow-moving products per warehouse:
SELECT  p.warehouseCode, w.warehouseName, p.productCode,
        p.productName, p.quantityInStock,
        COALESCE(SUM(od.quantityOrdered), 0) AS totalSold,
        ROUND(COALESCE(SUM(od.quantityOrdered) / NULLIF(p.quantityInStock, 0), 0), 4) AS stockTurnoverRatio
FROM mintclassics.products p
LEFT JOIN mintclassics.orderdetails od ON p.productCode = od.productCode
LEFT JOIN mintclassics.orders o ON od.orderNumber = o.orderNumber AND o.status = 'Shipped'
JOIN mintclassics.warehouses w ON p.warehouseCode = w.warehouseCode
GROUP BY p.warehouseCode, w.warehouseName, p.productCode, p.productName, p.quantityInStock
ORDER BY stockTurnoverRatio ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouseCode	warehouseName	productCode	productName	quantityInStock	totalSold	stockTurnoverRatio
b	East	S18_3233	1985 Toyota Supra	7733	0	0.0000
b	East	S18_1984	1995 Honda Civic	9772	917	0.0938
b	East	S24_3432	2002 Chevy Corvette	9446	894	0.0946
a	North	S32_2206	1982 Ducati 996 R	9241	906	0.0980
b	East	S18_3482	1976 Ford Gran Torino	9127	915	0.1003
b	East	S18_1589	1965 Aston Martin DB5	9042	914	0.1011
b	East	S12_3380	1968 Dodge Charger	9123	925	0.1014
a	North	S700_2466	America West Airlines B757-200	9653	984	0.1019
c	West	S18_2325	1932 Model A Ford J-Coupe	9354	957	0.1023
a	North	S12_2823	2002 Suzuki XREO	9997	1028	0.1028
b	East	S18_2870	1999 Indy 500 Monte Carlo SS	8164	855	0.1047
b	East	S18_3685	1948 Porsche Type 356 Roadster	8990	948	0.1055
c	West	S24_3151	1912 Ford Model T Delivery Wagon	9173	991	0.1080
d	South	S32_3207	1950's Chicago Surface Lines Streetcar	8601	934	0.1086
b	East	S18_1889	1948 Porsche 356-A Roadster	8826	972	0.1101
c	West	S18_1367	1936 Mercedes-Benz 500K Special Roadster	8635	960	0.1112
b	East	S24_1628	1966 Shelby Cobra 427 S/C	8197	915	0.1116
b	East	S24_2972	1982 Lamborghini Diablo	7723	912	0.1181

Result 17

Explanation of the query:

This SQL query calculates the **stock turnover ratio** for each **product** in each **warehouse**. Here's what it does:

1.Retrieves Relevant Columns:

- **warehouseCode** and **warehouseName** (to identify the warehouse).
- **productCode** and **productName** (to identify the product).
- **quantityInStock** (current stock level).
- **totalSold** (total units sold of the product).

2.Calculates Stock Turnover Ratio:

- Formula:
  Stock Turnover Ratio=Total Sold/Quantity in Stock
- This metric measures how efficiently a product is being sold. A higher ratio means the product is selling quickly relative to its stock level.

3.Filters or Sorts Results:

- The query likely retrieves data from **products**, **orderdetails**, and **warehouses** tables.
- It orders results by stock turnover ratio (sorts by lowest turnover).

- **Low stock turnover ratio** (close to 0) → **Product is overstocked but not selling well**
- **High stock turnover ratio** (closer to 1) → **Product is selling efficiently**

Initial analysis of the results:

- **Low Turnover Ratio (<0.2)**: Many products have very low sales relative to their stock, indicating overstocking or slow-moving inventory.
- **Moderate Turnover (0.2-0.5)**: Some products sell at a steady pace, but stock levels may still be high.
- **High Turnover (>1.0)**: These products are selling faster than they are restocked, indicating potential stock shortages.

Interesting observations:

- Extremely Low Turnover (0.0000): Example: 'S18_3233' - 1985 Toyota Supra has **7,733** units in stock but **0 sales**. This product might be obsolete or unwanted.

- Very High Turnover (>5.0):

- 'S24_2000' - 1960 BSA Gold Star DBD34' has only **15 units left** but **1,015 sold** → Demand is much higher than supply.
- 'S12_1099' - 1968 Ford Mustang' turnover is **13.72**, meaning it's selling out extremely fast.

- Overstocked Products:

- Products like 'S32_2206' - 1982 Ducati 996 R' have **9,241 in stock** but only **906 sold**.
- 'S700_2466' - America West Airlines B757-200' also has a very high stock-to-sales ratio.

Next Steps:

1. **Identify slow-moving inventory** (low turnover) and consider **discounts or promotions** to sell excess stock.
2. **Ensure sufficient stock for high-turnover products** to avoid lost sales.
3. **Check seasonal trends** to see if some products sell better in certain months.
4. **Analyse warehouse efficiency**: Compare turnover across warehouses to find underperforming locations.

1. Identify Slow-Moving Inventory (Low Turnover)

Products with low turnover may indicate overstocking. A common threshold is a turnover ratio below **0.5:**

EDA_UsingSQL_Project* | totalInventoryPerWarehouse

Limit to 1000 rows

```
509     -- 1. Identifying slow-moving inventory (low stock turnover):
510 •   SELECT  p.productCode, p.productName, p.quantityInStock,
511             COALESCE(SUM(od.quantityOrdered), 0) AS totalSold,
512             ROUND(COALESCE(SUM(od.quantityOrdered), 0) / NULLIF(p.quantityInStock, 0), 4) AS stockTurnoverRatio
513     FROM mintclassics.products p
514     LEFT JOIN mintclassics.orderdetails od ON p.productCode = od.productCode
515     GROUP BY p.productCode, p.productName, p.quantityInStock
516     HAVING stockTurnoverRatio < 0.5
517     ORDER BY stockTurnoverRatio ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

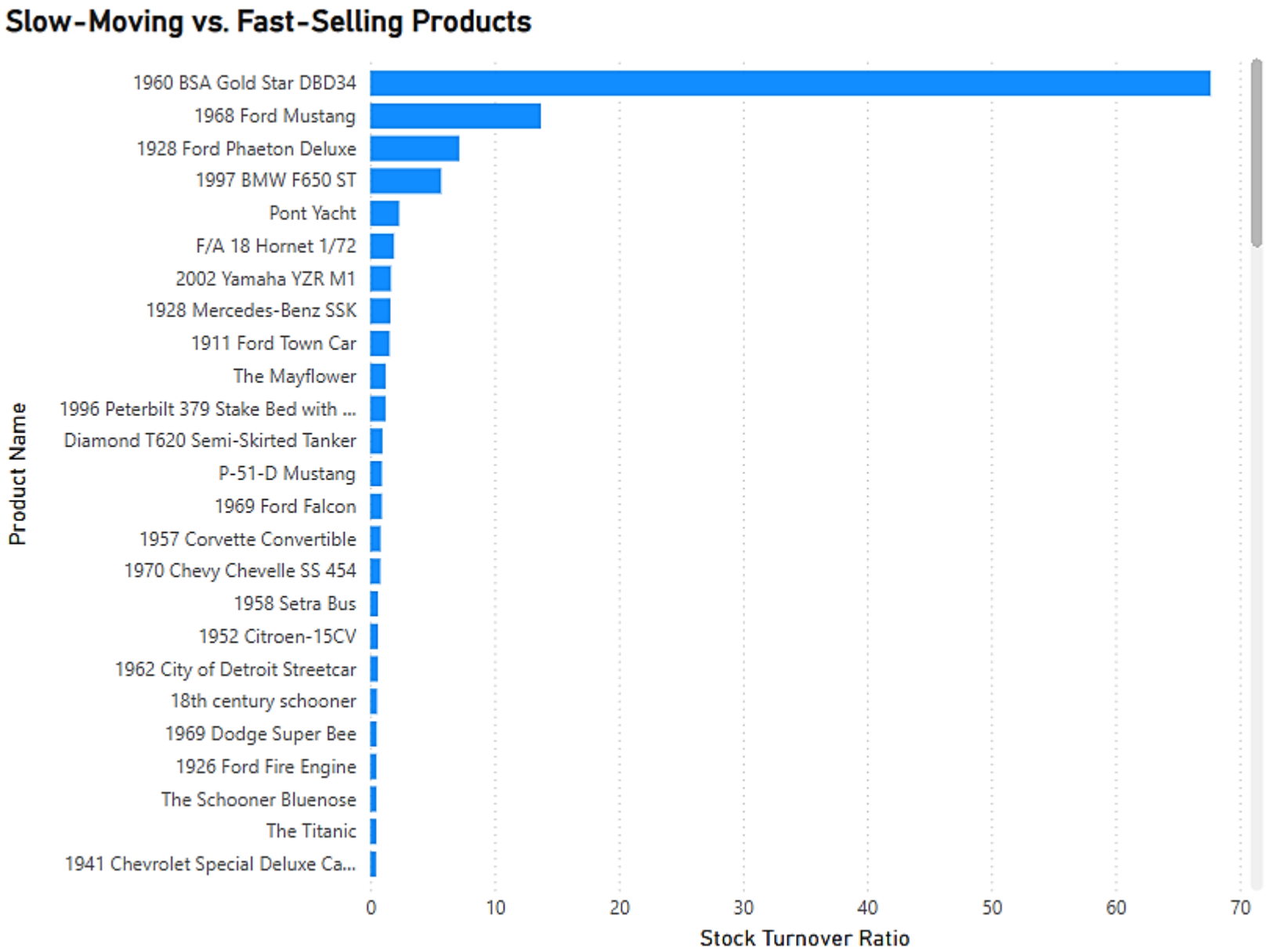
productCode	productName	quantityInStock	totalSold	stockTurnoverRatio
S18_3233	1985 Toyota Supra	7733	0	0.0000
S18_1984	1995 Honda Civic	9772	917	0.0938
S24_3432	2002 Chevy Corvette	9446	894	0.0946
S32_2206	1982 Ducati 996 R	9241	906	0.0980
S18_3482	1976 Ford Gran Torino	9127	915	0.1003
S18_1589	1965 Aston Martin DB5	9042	914	0.1011
S12_3380	1968 Dodge Charger	9123	925	0.1014
S700_2466	America West Airlines B757-200	9653	984	0.1019
S18_2325	1932 Model A Ford J-Coupe	9354	957	0.1023
S12_2823	2002 Suzuki XREO	9997	1028	0.1028
S18_2870	1999 Indy 500 Monte Carlo SS	8164	855	0.1047
S18_3685	1948 Porsche Type 356 Roadster	8990	948	0.1055
S24_3151	1912 Ford Model T Delivery Wagon	9173	991	0.1080
S32_3207	1950's Chicago Surface Lines Streetcar	8601	934	0.1086
S18_1889	1948 Porsche 356-A Roadster	8826	972	0.1101

Result 24

Analysis:
Products with **very low turnover** are candidates for promotions or discounts.

1. Identify Slow-Moving Inventory (Low Turnover)

Visualization: Bar Chart – Slow-Moving vs. Fast-Selling Products



Analysis:

- **Low-turnover products** (bars on the left) may need **discounts** or **promotions**.
- **High-turnover products** (bars on the right) might need **higher stock levels**.

Scroll down to see products with low-turnover ratio.

2. Ensure Sufficient Stock for High-Turnover Products

If a product sells quickly, maintaining adequate stock prevents lost sales.

EDA_UsingSQL_Project | totalInventoryPerWarehouse

Limit to 1000 rows

```
520     -- 2. Ensuring sufficient stock for high-turnover products:
521 •   SELECT  p.productCode, p.productName, p.quantityInStock,
522             COALESCE(SUM(od.quantityOrdered), 0) AS totalSold,
523             ROUND(COALESCE(SUM(od.quantityOrdered), 0) / NULLIF(p.quantityInStock, 0), 4) AS stockTurnoverRatio
524     FROM mintclassics.products p
525     LEFT JOIN mintclassics.orderdetails od ON p.productCode = od.productCode
526     GROUP BY p.productCode, p.productName, p.quantityInStock
527     HAVING stockTurnoverRatio > 2  -- Considered high turnover
528     ORDER BY stockTurnoverRatio DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productCode	productName	quantityInStock	totalSold	stockTurnoverRatio
S24_2000	1960 BSA Gold Star DBD34	15	1015	67.6667
S12_1099	1968 Ford Mustang	68	933	13.7206
S32_4289	1928 Ford Phaeton Deluxe	136	972	7.1471
S32_1374	1997 BMW F650 ST	178	1014	5.6966
S72_3212	Pont Yacht	414	958	2.3140

Analysis: These products might need **higher stock levels** to prevent shortages.

Sufficient Stock for High-Turnover Products

productName	Sum of quantityInStock	StockTurnoverRatio	HighTurnoverFlag
1928 Ford Phaeton Deluxe	136	7,15	High Turnover
1960 BSA Gold Star DBD34	15	67,67	High Turnover
1968 Ford Mustang	68	13,72	High Turnover
1997 BMW F650 ST	178	5,70	High Turnover
Pont Yacht	414	2,31	High Turnover
18th century schooner	1898	0,53	Normal
18th Century Vintage Horse Carriage	5992	0,15	Normal
1900s Vintage Bi-Plane	5942	0,16	Normal
1900s Vintage Tri-Plane	2756	0,37	Normal
1903 Ford Model A	3913	0,23	Normal
1904 Buick Runabout	8290	0,12	Normal
1911 Ford Town Car	540	1,54	Normal
1912 Ford Model T Delivery Wagon	9173	0,11	Normal
1913 Ford Model T Speedster	4189	0,25	Normal
1917 Grand Touring Sedan	2724	0,34	Normal
1917 Maxwell Touring Car	7913	0,13	Normal
1926 Ford Fire Engine	2018	0,49	Normal
1928 British Royal Navy Airplane	3627	0,27	Normal
1928 Mercedes-Benz SSK	548	1,61	Normal
1930 Buick Marquette Phaeton	7062	0,15	Normal
1932 Alfa Romeo 8C2300 Spider Sport	6553	0,13	Normal
1932 Model A Ford J-Coupe	9354	0,10	Normal
1934 Ford V8 Coupe	5649	0,17	Normal
1936 Chrysler Airflow	4710	0,21	Normal
1936 Harley Davidson El Knucklehead	4357	0,22	Normal
1936 Mercedes Benz 500k Roadster	2081	0,40	Normal
1936 Mercedes-Benz 500K Special Roadster	8635	0,11	Normal
1937 Horch 930V Limousine	2902	0,30	Normal
1937 Lincoln Berline	8693	0,13	Normal
1938 Cadillac V-16 Presidential Limousine	2847	0,34	Normal
1939 Cadillac Limousine	6645	0,15	Normal
Total	**555131**	**0,19**	**Normal**

2. Ensure Sufficient Stock for High-Turnover Products

Analysis:

- Products with **High Turnover** should be **monitored closely** to avoid stockouts.
- Check supplier lead-times to **avoid lost sales**.

3. Check Seasonal Trends for Products

Some products may have peak sales in specific months.

EDA_UsingSQL_Project | totalInventoryPerWarehouse

Limit to 1000 rows

```
531  -- 3. checking seasonal trends for products:
532  SELECT  MONTH(o.orderDate) AS orderMonth,
533          p.productCode, p.productName,
534          SUM(od.quantityOrdered) AS totalSold
535  FROM mintclassics.orders o
536  JOIN mintclassics.orderdetails od ON o.orderNumber = od.orderNumber
537  JOIN mintclassics.products p ON od.productCode = p.productCode
538  GROUP BY orderMonth, p.productCode, p.productName
539  ORDER BY totalSold DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

orderMonth	productCode	productName	totalSold
11	S18_3232	1992 Ferrari 360 Spider red	354
11	S700_4002	American Airlines: MD-11S	262
11	S18_2432	1926 Ford Fire Engine	249
11	S18_4721	1957 Corvette Convertible	242
11	S18_3856	1941 Chevrolet Special Deluxe Cabriolet	242
11	S18_4600	1940s Ford truck	241
11	S24_2841	1900s Vintage Bi-Plane	241
11	S12_3380	1968 Dodge Charger	240
11	S700_2610	The USS Constitution Ship	240
11	S18_1589	1965 Aston Martin DB5	237
11	S32_1374	1997 BMW F650 ST	236
11	S18_3685	1948 Porsche Type 356 Roadster	230
11	S32_3522	1996 Peterbilt 379 Stake Bed with Outrigger	229
11	S18_1129	1993 Mazda RX-7	228
11	S18_1342	1937 Lincoln Berline	227

Result 28

EDA_UsingSQL_Project* | totalInventoryPerWarehouse

Limit to 1000 rows

```
531  -- 3. checking seasonal trends for products:
532  SELECT  MONTH(o.orderDate) AS orderMonth,
533          p.productCode, p.productName,
534          SUM(od.quantityOrdered) AS totalSold
535  FROM mintclassics.orders o
536  JOIN mintclassics.orderdetails od ON o.orderNumber = od.orderNumber
537  JOIN mintclassics.products p ON od.productCode = p.productCode
538  GROUP BY orderMonth, p.productCode, p.productName
539  ORDER BY orderMonth, totalSold DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows

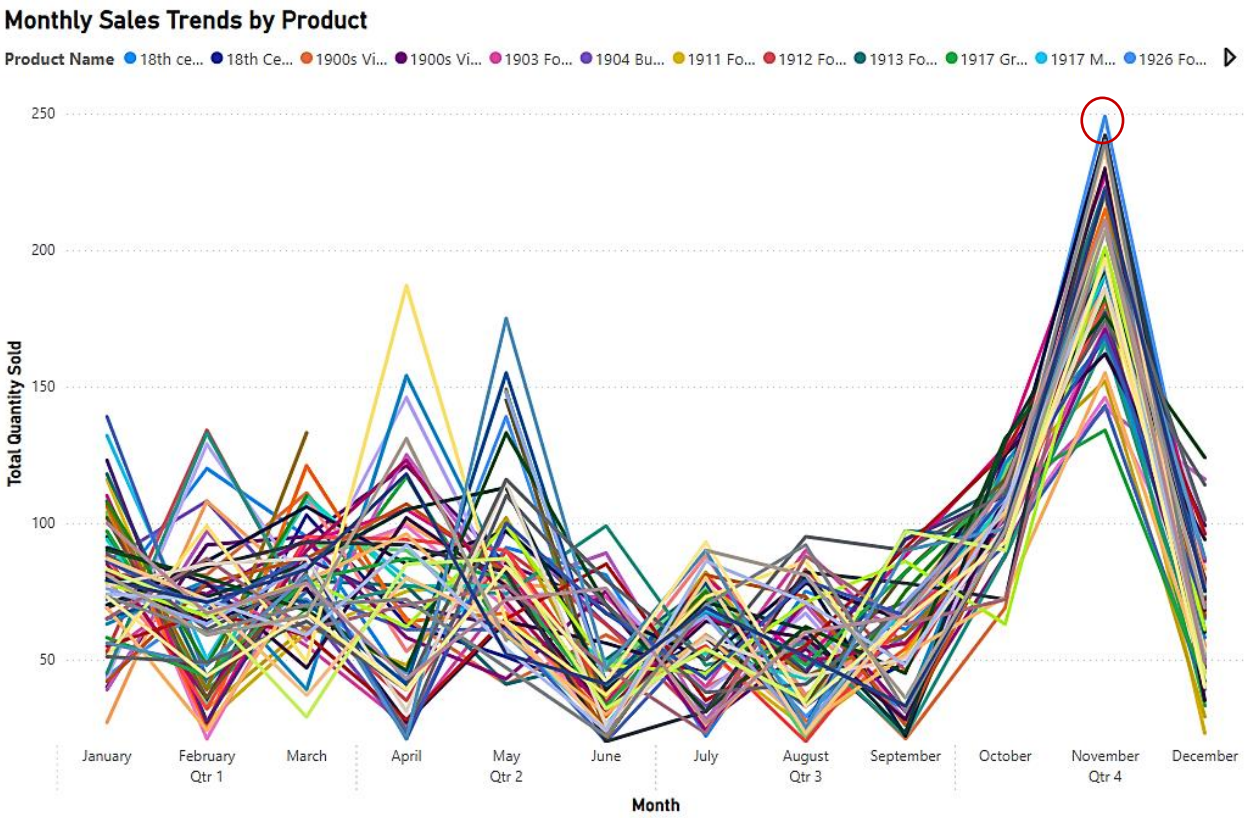
orderMonth	productCode	productName	totalSold
1	S24_3969	1936 Mercedes Benz 500k Roadster	139
1	S18_3320	1917 Maxwell Touring Car	132
1	S18_3232	1992 Ferrari 360 Spider red	124
1	S24_2022	1938 Cadillac V-16 Presidential Limousine	123
1	S18_2949	1913 Ford Model T Speedster	118
1	S18_2248	1911 Ford Town Car	116
1	S18_1342	1937 Lincoln Berline	110
1	S18_2795	1928 Mercedes-Benz SSK	108
1	S18_1097	1940 Ford Pickup Truck	108
1	S18_4668	1939 Cadillac Limousine	107
1	S18_1367	1936 Mercedes-Benz 500K Special Roadster	106
1	S24_4258	1936 Chrysler Airflow	104
1	S18_4409	1932 Alfa Romeo 8C2300 Spider Sport	103
1	S10_1949	1952 Alpine Renault 1300	102
1	S12_3148	1969 Corvair Monza	100

Result 31

My insights: I identified that product number S18_3232 sold well in November, which has the highest total sold of 354. Another example is, S24_3969 has the highest sell of 139 in January, etc. In general, trends show that November is the peak sales month (for most of the products).

Analysis:

- Identify products that **sell well in specific months** (e.g., holiday-related items).
- Plan **stock levels** accordingly to match peak demand periods.
- If possible, compare this **year-over-year** to confirm recurring seasonal trends.



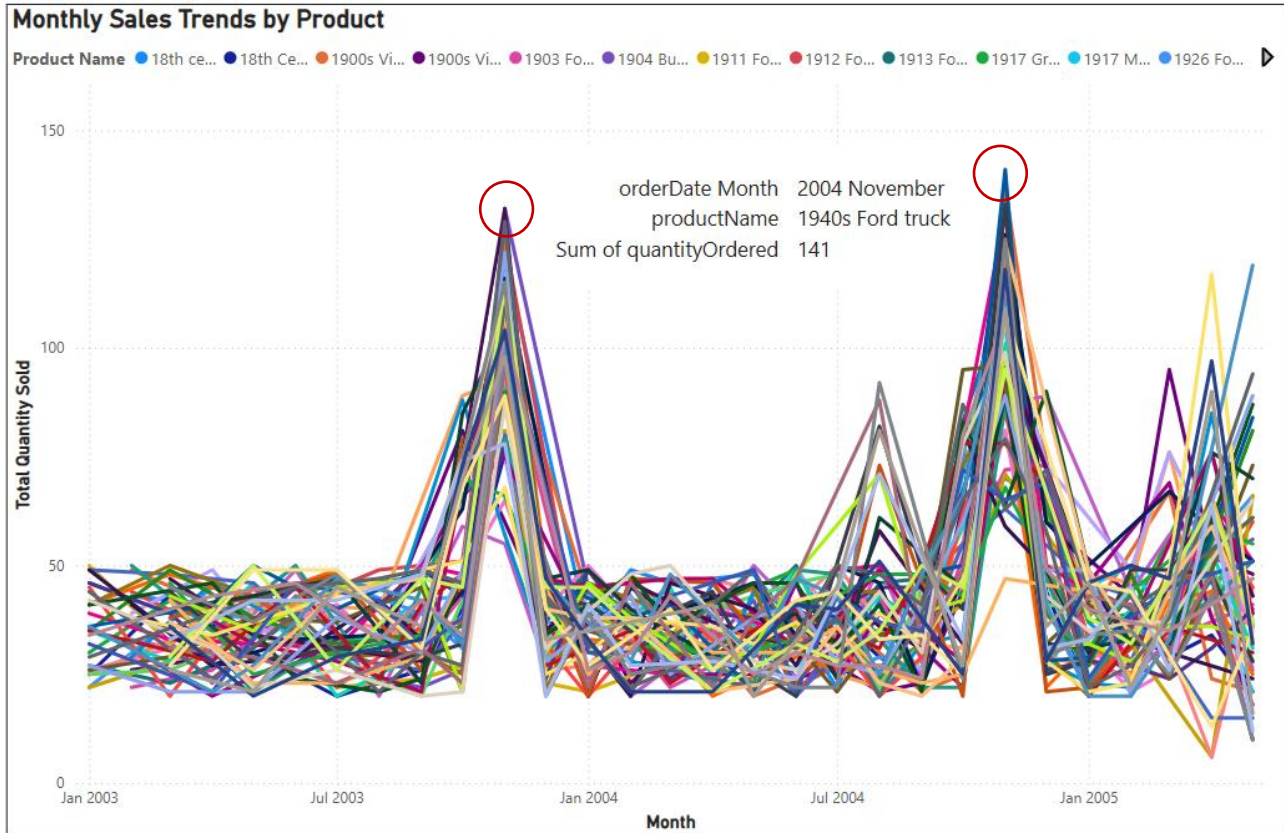
3. Check Seasonal Trends for Products

Visualization: Line Chart – Monthly Sales Trends

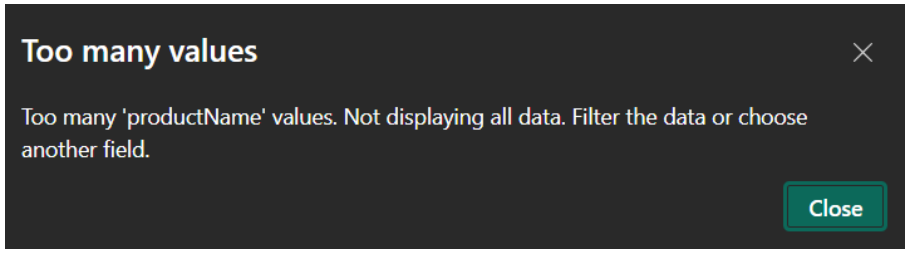
Analysis:

- Identify **peak sales months** for specific products.
- Plan **restocking strategies** for seasonal demand.
- Cross-check year-over-year trends for **long-term patterns.**

⇒ **when I checked this, it turned out November is the peak sales months for 2004 and 2005 (see chart below).**



Note: The message “Too many values” might pop up. Because number of products is too high, so Power BI is not showing all data.
However, overall trend is similar, since great number of products sold in **November**.



4. Analyse Warehouse Efficiency

Comparing turnover across warehouses can reveal inefficiencies.

EDA_UsingSQL_Project* | totalInventoryPerWarehouse

Limit to 1000 rows

```
542  -- 4. Analyzing warehouse efficiency.
543  -- Comparing turnover across warehouses can reveal inefficiencies:
544  SELECT  w.warehouseCode, w.warehouseName,
545          COUNT(DISTINCT p.productCode) AS totalProducts,
546          SUM(p.quantityInStock) AS totalStock,
547          COALESCE(SUM(od.quantityOrdered), 0) AS totalSold,
548          ROUND(COALESCE(SUM(od.quantityOrdered), 0) / NULLIF(SUM(p.quantityInStock), 0), 4) AS avgTurnoverRatio
549  FROM mintclassics.products p
550  JOIN mintclassics.warehouses w ON p.warehouseCode = w.warehouseCode
551  LEFT JOIN mintclassics.orderdetails od ON p.productCode = od.productCode
552  GROUP BY w.warehouseCode, w.warehouseName
553  ORDER BY avgTurnoverRatio DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouseCode	warehouseName	totalProducts	totalStock	totalSold	avgTurnoverRatio
d	South	23	2186871	22351	0.0102
a	North	25	3659553	24650	0.0067
c	West	24	3439570	22933	0.0067
b	East	38	5851766	35582	0.0061

Analysis:

- Warehouses with **low turnover ratios** may need optimization (e.g., stock redistribution).
- Check if underperforming warehouses have **longer shipping-times** that discourage purchases.
- Consider closing or repurposing low-efficiency storage locations (i.e., if a warehouse consistently has **low sales**) ⇒ South warehouse can be a candidate.

Final Step: Recommendations & Future Work

Now, let's summarize **recommendations** based on our findings and propose **future improvements** for Mint Classics' inventory management.

Final Conclusion & Decision (Updated with Shipping Time Insights)

Optimizing Warehouse Operations

- **The East Warehouse has the lowest efficiency** (low turnover and high inventory levels), making it a **potential candidate for closure or downsizing**.
- **However, the South Warehouse, while having the best turnover ratio, also has the longest average shipping time.** This suggests that it may not be the best option for absorbing inventory from a closed facility.
- Instead, the **North and West Warehouses**, which have similar turnover ratios but lower shipping times than the South Warehouse, should be considered for redistributing inventory.

Improving Inventory Management

- **Liquidate slow-moving stock** in warehouses with high inventory but low turnover (e.g., East Warehouse).
- **Ensure high-demand products** are stocked in warehouses that can fulfill orders quickly.

Balancing Shipping Efficiency vs. Inventory Performance

- The **South Warehouse has the highest stock turnover** but also the **longest average shipping time**.
- The company must **assess whether keeping South Warehouse fully operational** is worth the longer delivery time or whether **some inventory should be relocated** to a more central warehouse for faster shipping.

Warehouse Consolidation Strategy

- If a warehouse is to be closed, **the East Warehouse remains the most likely candidate**, but inventory must be strategically **reallocated to minimize shipping delays**.
- The **North and West Warehouses should be optimized** for absorbing stock without overloading their capacity.

Updated Next Steps & Future Work

- **Reevaluate the South Warehouse's role** – Despite its high efficiency, its **longer shipping times** may require **alternative logistics solutions**.
- **Optimize warehouse selection** – Instead of full closure, consider **redistributing stock based on both turnover and shipping efficiency**.
- **Customer Impact Analysis** – Investigate how **shipping times affect customer satisfaction and repeat purchases**.

Final Thought

By considering **both stock turnover and shipping performance**, Mint Classics can **close or optimize the right warehouse without negatively impacting delivery times**. This ensures a **cost-effective, efficient, and customer-friendly inventory strategy**

Insights from the analysis

Euro + Shopping Channel has the highest credit limit:

- A high credit limit is typically assigned to **big buyers**, often wholesalers or large retailers.
- This suggests that **Euro + Shopping Channel** is not an average customer but possibly a key business partner.

They placed multiple orders on the same day (while others didn’t):

- This could indicate **bulk ordering**, possibly in batches.
- Maybe they have an internal ordering system that splits large orders into separate transactions.
- Or, they could have **different departments** placing orders independently on the same day.

They ordered the highest number of products:

- This strengthens the idea that they might be **a wholesaler or a large distributor** rather than a regular retailer.
- It would be interesting to check **which products they order the most**—maybe they specialize in selling specific types of products.

Some Observations

MySQL Workbench

Local instance

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- Tables
  - customers
    - Columns
      - customerNumber
      - customerName
      - contactLastName
      - contactFirstName
      - phone
      - addressLine1
      - addressLine2
      - city
      - state
      - postalCode
      - country
      - salesRepEmployeeNumber
      - creditLimit
    - Indexes
    - Foreign Keys
      - customers_ibfk_1
    - Triggers
  - employees
  - offices

Administration Schemas

Information

Table: warehouses

Columns:

warehouseCode	char(1) PK
warehouseName	varchar(45)
warehousePctCap	varchar(50)

payments customers employees offices orderdetails orders payments productlines products

Limit to 1000 rows

```
37
38 •  SELECT orders.orderNumber, orders.orderDate, customers.customerName
39    FROM mintclassics.orders
40    INNER JOIN mintclassics.customers ON orders.customerNumber = customers.customerNumber
41    -- GROUP BY orders.orderDate
42    ORDER BY orderDate DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

orderNumber	orderDate	customerName
10424	2005-05-31	Euro+ Shopping Channel
10425	2005-05-31	La Rochelle Gifts
10422	2005-05-30	Diecast Classics Inc.
10423	2005-05-30	Petit Auto
10420	2005-05-29	Souveniers And Things Co.
10421	2005-05-29	Mini Gifts Distributors Ltd.
10419	2005-05-17	Salzburg Collectables
10418	2005-05-16	Extreme Desk Decorations, Ltd
10417	2005-05-13	Euro+ Shopping Channel
10416	2005-05-10	L'ordine Souveniers
10415	2005-05-09	Australian Collectables, Ltd
10414	2005-05-06	Gifts4AllAges.com
10413	2005-05-05	Gift Depot Inc.
10412	2005-05-03	Euro+ Shopping Channel
10411	2005-05-01	Québec Home Shopping Network
10410	2005-04-29	GiftsForHim.com
10409	2005-04-23	Handji Gifts& Co
10407	2005-04-22	The Sharp Gifts Warehouse
10408	2005-04-22	Tokyo Collectables, Ltd
10406	2005-04-15	Danish Wholesale Imports
10405	2005-04-14	Mini Caravy
10403	2005-04-08	UK Collectables. Ltd.

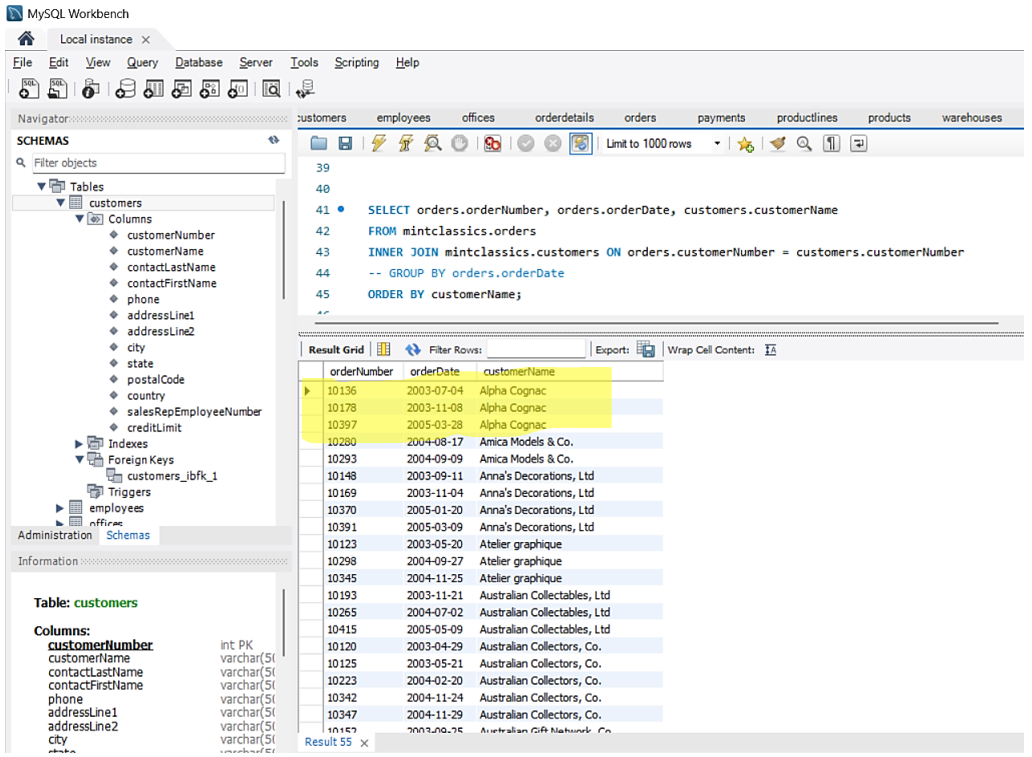
Result 52

What I did here: My main goal was to see which order is placed by which customer on what date. To achieve this result, I needed to gather information from two tables (customers and orders), I did that using INNER JOIN and got the result here.

Observations:

- When I listed the order dates in descending order, I found out that the date that the last order placed is 2005-05-31. However, then I noticed that on that very date 10424 was not the only order, the order 10425 was also placed on the same date (by different customer). So, on the same date multiple orders can be placed by different customers.

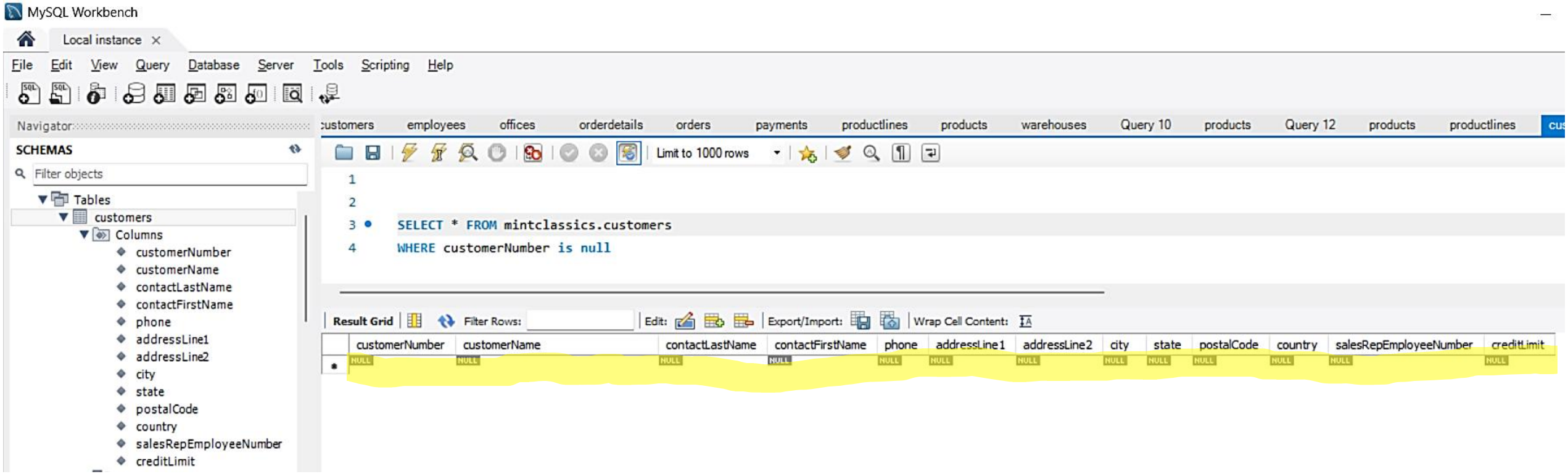
- When we order by customerName, we can observe that most customers placed more than one separate orders on different dates. E.g. Alpha Cognac placed 3 different orders between the year 2003 and 2005.



Note to myself: Understanding the dataset is indeed helpful. Through this process, I'm getting familiar with the data, and I feel like I actually understand what's going on here. Asking questions and then answering them with queries is surprisingly satisfying.

Important note:
It is good to be careful when using INNER JOIN, because you might lose some information as it matches only the values in the columns that appears in both tables. In my dataset, if one customer doesn't have a customer number, then that customer won't show up in the result, because we combine information from both tables by matching rows where the customerNumbers column in the customers table matches the customerNumbers column in the orders table.

For this reason, I checked if there is a null value in the customerNumber column, it turned out there is one row with full of null value, which might be removed.



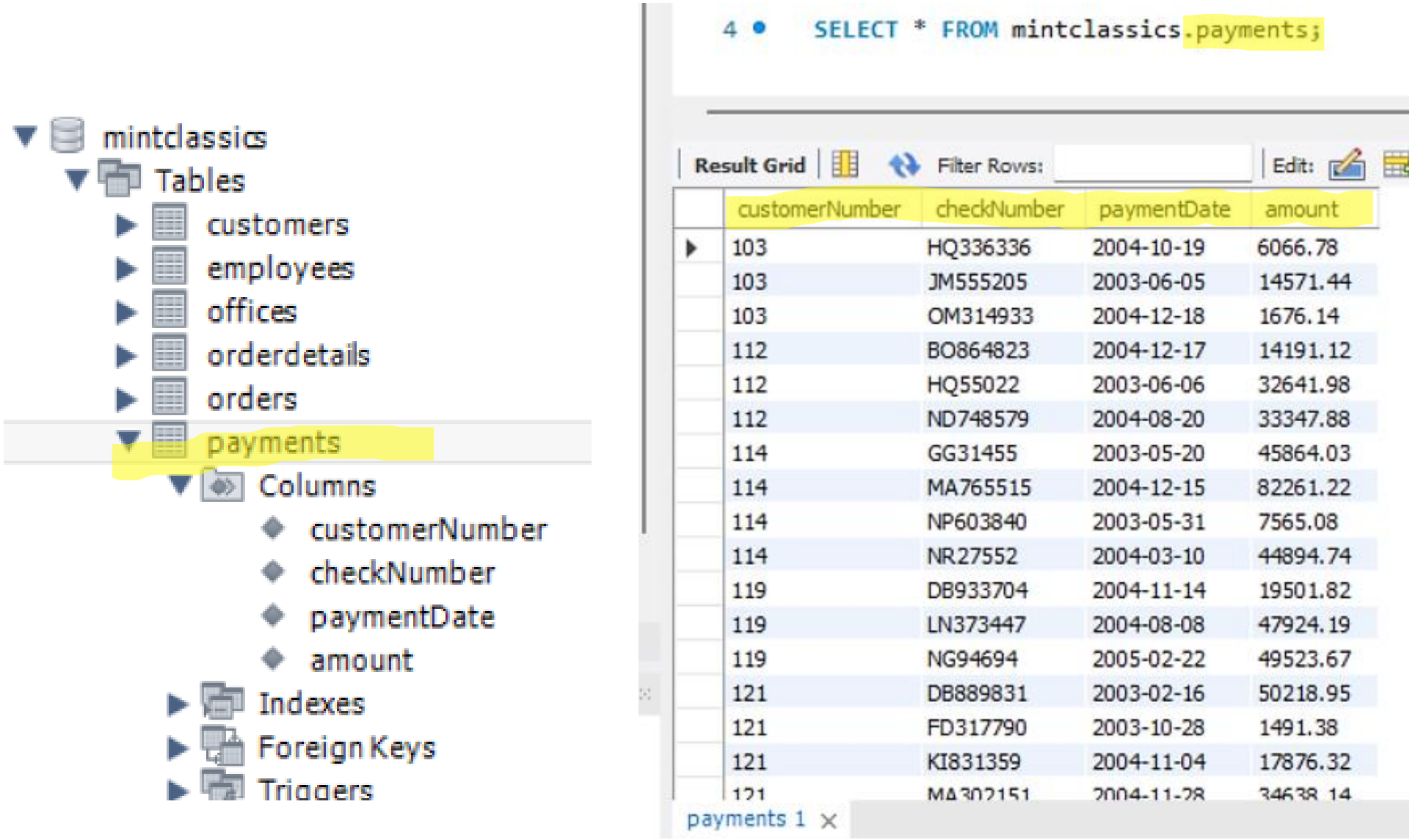
Key Takeaways (So Far)

My primary focus has been on analyzing the **orders, orderdetails, products, and customers** tables in the database and exploring the relationships between them. As a result, my key findings so far include:

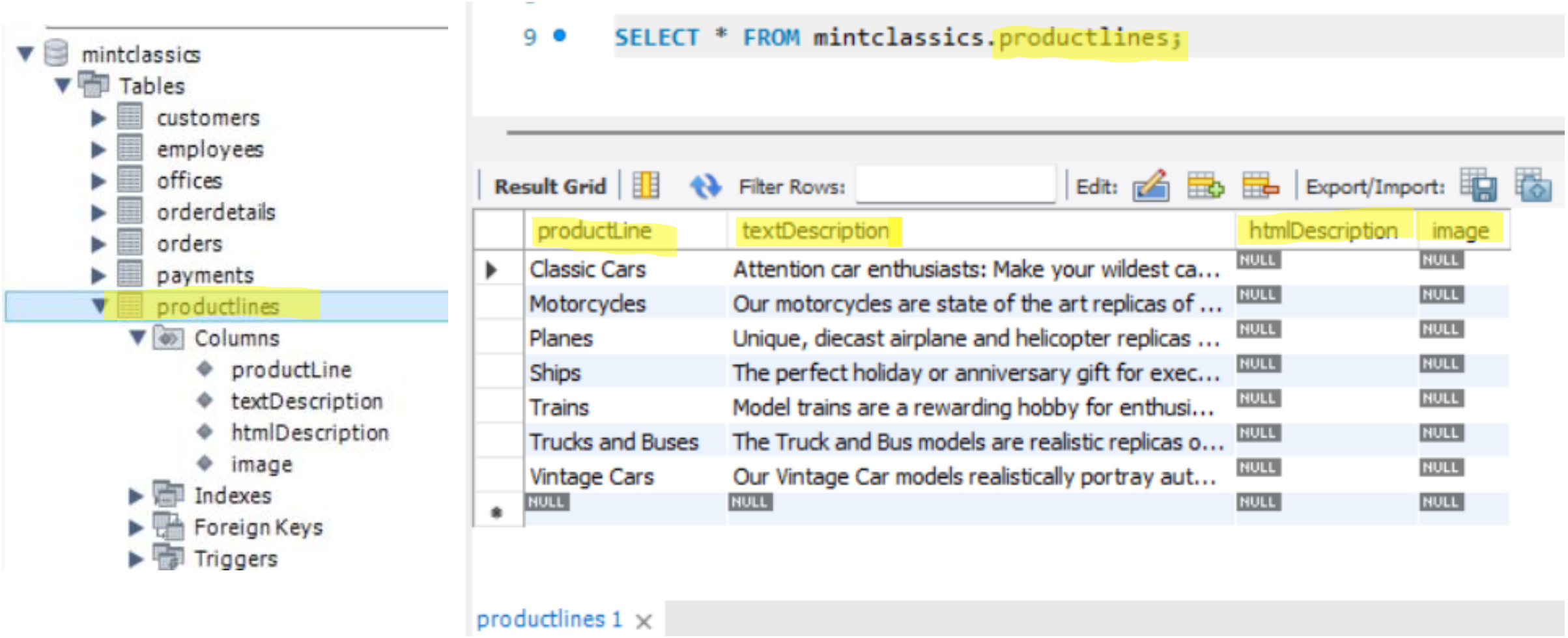
- A single order by a customer can include **multiple products.** In other words, when a customer places **one order**, they can buy **one or more products**. ⇒ The **orderdetails** table stores **which products** and **how many** of each product are included in the order.
- A customer can place **only one order per day** (i.e., on the same date). However, there is a single exception in our database - the customer with the highest number of orders. Why?
- Over time, a customer can make **multiple orders** containing different products.
- On the same date, **multiple orders** can be placed by different customers (only one exception, **Euro + Shopping Channel** with **customer number 141**).
- The **orderdetails** and **products** tables are linked via the **productCode** column. I used this relationship to join these tables for analysis.
- **orders** and **products** tables are connected through **orderdetails** table.
- **customers** and **orders** tables are connected with **customerNumber** column.

Challenges & Lessons Learned

One of the biggest challenges has been identifying relationships within the data, understanding how the business operates, and getting a clear picture of the database structure. What helped me most during this process was asking **a lot** of questions.
By running SQL queries (and occasionally using visualizations) to answer these questions, I became more familiar with the data, uncovered new insights, and even formulated **more** questions along the way. Having questions in mind is valuable, because in the process of solving them, you end up learning a lot along the way!


mintclassics
Tables
customers
employees
offices
orderdetails
orders
payments
Columns
customerNumber
checkNumber
paymentDate
amount
Indexes
Foreign Keys
Triggers
4 • SELECT * FROM mintclassics.payments;
Result Grid
Filter Rows:
Edit:
customerNumber checkNumber paymentDate amount
103 HQ336336 2004-10-19 6066.78
103 JM555205 2003-06-05 14571.44
103 OM314933 2004-12-18 1676.14
112 BO864823 2004-12-17 14191.12
112 HQ55022 2003-06-06 32641.98
112 ND748579 2004-08-20 33347.88
114 GG31455 2003-05-20 45864.03
114 MA765515 2004-12-15 82261.22
114 NP603840 2003-05-31 7565.08
114 NR27552 2004-03-10 44894.74
119 DB933704 2004-11-14 19501.82
119 LN373447 2004-08-08 47924.19
119 NG94694 2005-02-22 49523.67
121 DB889831 2003-02-16 50218.95
121 FD317790 2003-10-28 1491.38
121 KI831359 2004-11-04 17876.32
payments 1

Each product belongs to one product line and is stored in one warehouse.



```
4 •  SELECT * FROM mintclassics.products;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

productCode	productName	productLine	productScale	productVendor	productDescription	quantityInStock	warehouseCode	buyPrice	MSF
S10_1678	1969 Harley Davidson Ultimate Chopper	Motorcycles	1:10	Min Lin Diecast	This replica features working kickstand, front su...	7933	a	48.81	95.7
S10_1949	1952 Alpine Renault 1300	Classic Cars	1:10	Classic Metal Creations	Turnable front wheels; steering function; detail...	7305	b	98.58	214.
S10_2016	1996 Moto Guzzi 1100i	Motorcycles	1:10	Highway 66 Mini Classics	Official Moto Guzzi logos and insignias, saddle b...	6625	a	68.99	118.
S10_4698	2003 Harley-Davidson Eagle Drag Bike	Motorcycles	1:10	Red Start Diecast	Model features, official Harley Davidson logos a...	5582	a	91.02	193.
S10_4757	1972 Alfa Romeo GTA	Classic Cars	1:10	Motor City Art Classics	Features include: Turnable front wheels; steeri...	3252	b	85.68	136.
S10_4962	1962 LanciaA Delta 16V	Classic Cars	1:10	Second Gear Diecast	Features include: Turnable front wheels; steeri...	6791	b	103.42	147.
S12_1099	1968 Ford Mustang	Classic Cars	1:12	Autoart Studio Design	Hood, doors and trunk all open to reveal highly ...	68	b	95.34	194.
S12_1108	2001 Ferrari Enzo	Classic Cars	1:12	Second Gear Diecast	Turnable front wheels; steering function; detail...	3619	b	95.59	207.
S12_1666	1958 Setra Bus	Trucks and Buses	1:12	Welly Diecast Productions	Model features 30 windows, skylights & glare re...	1579	d	77.90	136.
S12_2823	2002 Suzuki XREO	Motorcycles	1:12	Unimax Art Galleries	Official logos and insignias, saddle bags located ...	9997	a	66.27	150.
S12_3148	1969 Corvair Monza	Classic Cars	1:18	Welly Diecast Productions	1:18 scale die-cast about 10" long doors open, ...	6906	b	89.14	151.
S12_3380	1968 Dodge Charger	Classic Cars	1:12	Welly Diecast Productions	1:12 scale model of a 1968 Dodge Charger. Ho...	9123	b	75.16	117.
S12_3891	1969 Ford Falcon	Classic Cars	1:12	Second Gear Diecast	Turnable front wheels; steering function; detail...	1049	b	83.05	173.
S12_3990	1970 Plymouth Hemi Cuda	Classic Cars	1:12	Studio M Art Models	Very detailed 1970 Plymouth Cuda model in 1:1...	5663	b	31.92	79.8
S12_4473	1957 Chevy Pickup	Trucks and Buses	1:12	Exoto Designs	1:12 scale die-cast about 20" long Hood opens, ...	6125	d	55.70	118.
S12_4675	1969 Dodge Charger	Classic Cars	1:12	Welly Diecast Productions	Detailed model of the 1969 Dodge Charger. Thi...	7323	b	58.73	115

products 1 × Apply

- mintclassics
  - Tables
    - customers
    - employees
    - offices
    - orderdetails
    - orders
    - payments
    - productlines
    - products
      - Columns
        - productCode
        - productName
        - productLine
        - productScale
        - productVendor
        - productDescription
        - quantityInStock
        - warehouseCode
        - buyPrice
        - MSRP

```
1
2
3 •  SELECT * FROM mintclassics.orderdetails;
```

Result Grid | Filter Rows: | Edit: | Export/Im

orderNumber	productCode	quantityOrdered	priceEach	orderLineNumber
10107	S10_1678	30	81.35	2
10121	S10_1678	34	86.13	5
10134	S10_1678	41	90.92	2
10145	S10_1678	45	76.56	6
10159	S10_1678	49	81.35	14
10168	S10_1678	36	94.74	1
10180	S10_1678	29	76.56	9
10188	S10_1678	48	95.70	1
10201	S10_1678	22	82.30	2
10103	S10_1949	26	214.30	11
10112	S10_1949	29	197.16	1
10126	S10_1949	38	205.73	11
10140	S10_1949	37	186.44	11
10150	S10_1949	45	182.16	8
10163	S10_1949	21	212.16	1
10174	S10_1949	34	207.87	4
10183	S10_1949	23	180.01	8
10194	S10_1949	42	203.59	11
10107	S10_2016	39	105.86	5
10120	S10_2016	29	118.94	3
10134	S10_2016	27	116.56	5

orderdetails 1 ×

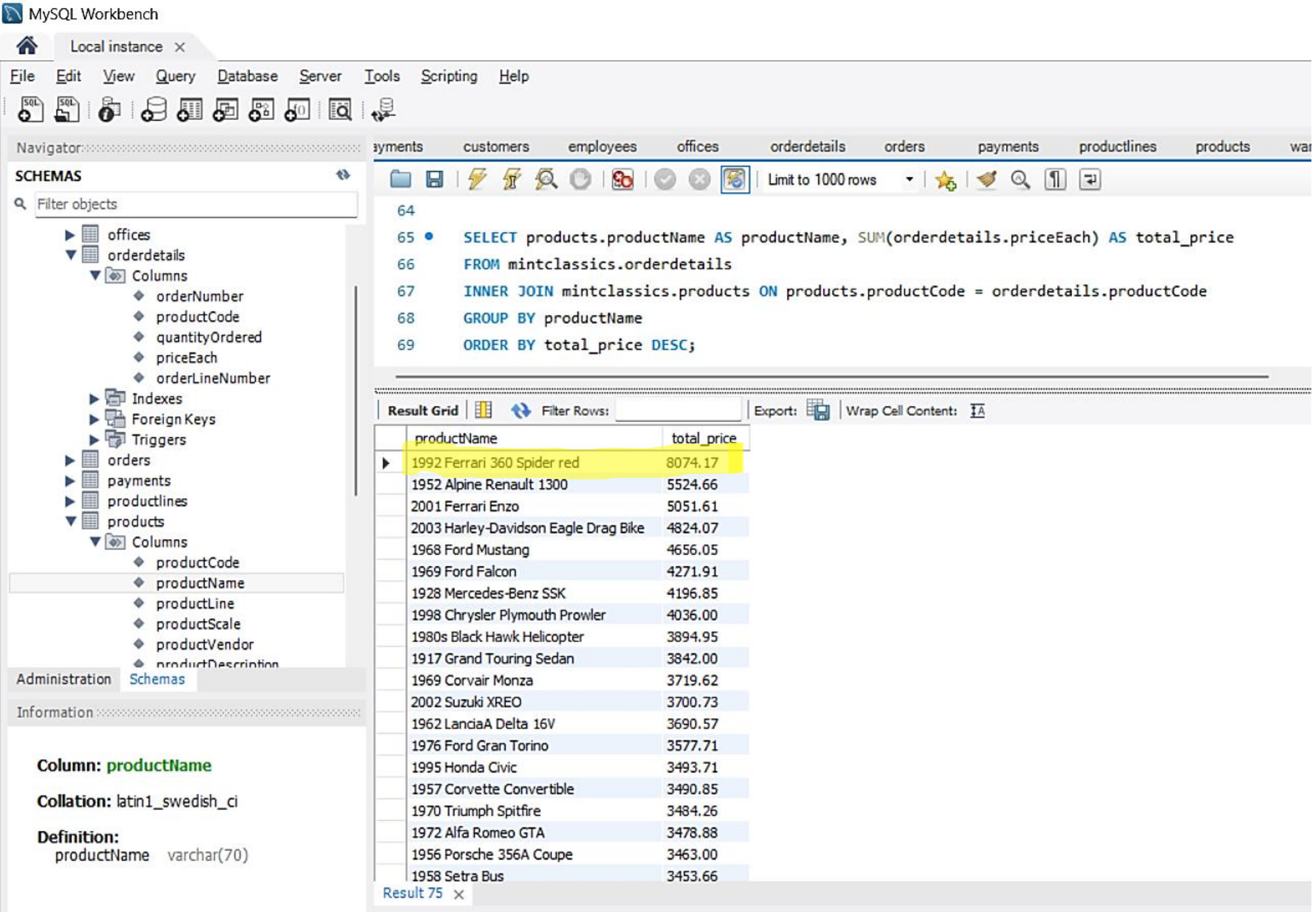
```
71  -- This query gives price for each product. But why each product has multiple price?
72 •  SELECT products.productName AS productName, orderdetails.priceEach
73  FROM mintclassics.orderdetails
74  INNER JOIN mintclassics.products ON products.productCode = orderdetails.productCode
75  ORDER BY productName;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

productName	priceEach
1952 Alpine Renault 1...	214.30
1952 Alpine Renault 1...	197.16
1952 Alpine Renault 1...	205.73
1952 Alpine Renault 1...	186.44
1952 Alpine Renault 1...	182.16
1952 Alpine Renault 1...	212.16
1952 Alpine Renault 1...	207.87
1952 Alpine Renault 1...	180.01
1952 Alpine Renault 1...	203.59
1952 Alpine Renault 1...	203.59
1952 Alpine Renault 1...	205.73
1952 Alpine Renault 1...	214.30
1952 Alpine Renault 1...	195.01
1952 Alpine Renault 1...	177.87
1952 Alpine Renault 1...	171.44
1952 Alpine Renault 1...	205.73
1950's Chicago Surfac...	56.55
1950's Chicago Surfac...	60.28
1950's Chicago Surfac...	60.90
1950's Chicago Surfac...	60.90
1950's Chicago Surfac...	57.17

Result 77 ×

We can observe in the snapshots of both orderdetails table and the combination of orderdetails and products table that each product has multiple price.
Q: Why each product has different price? What affects the variation of price? Does the price of a product change depending on the location (country/state/warehouse) it is sold?

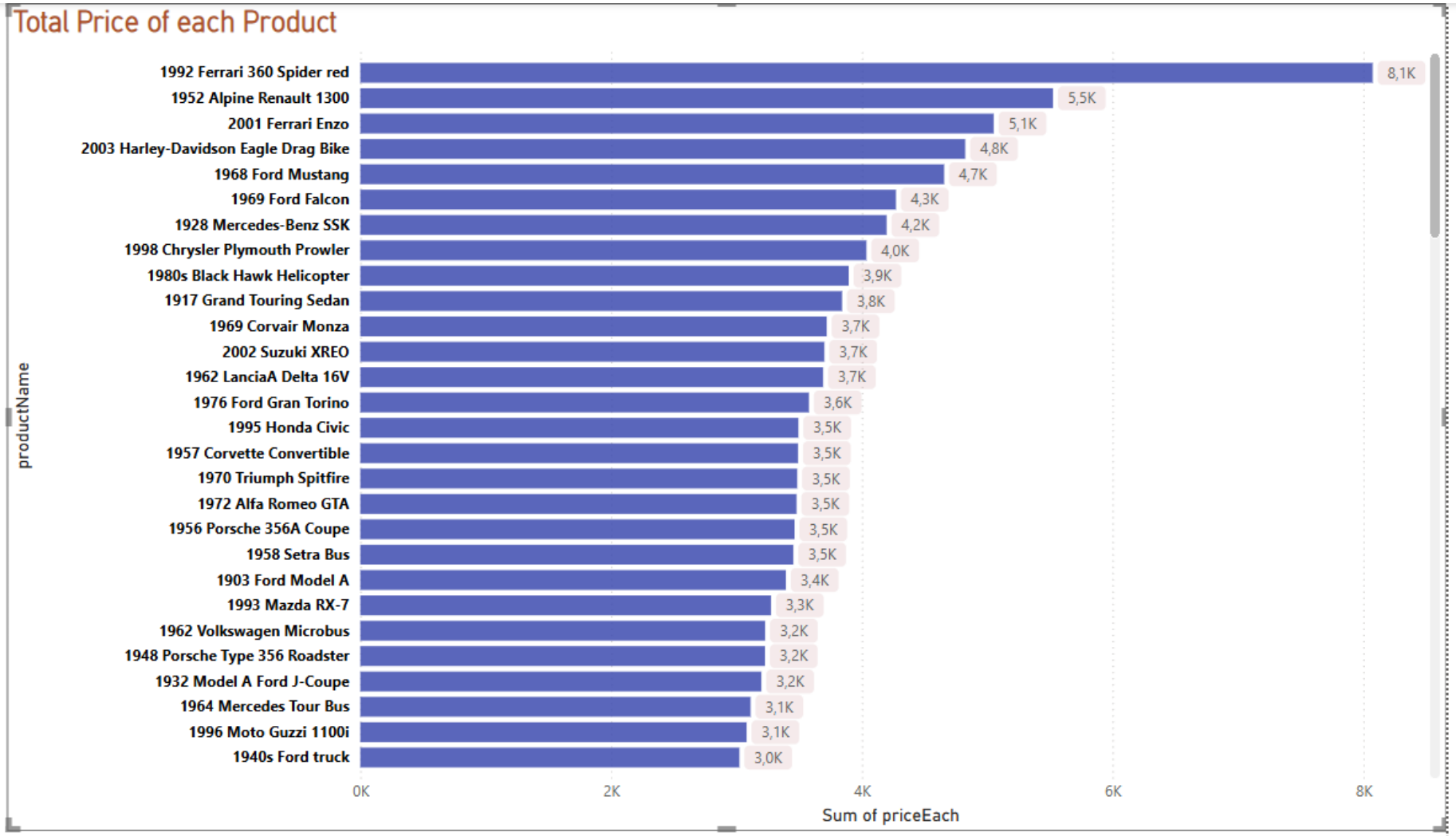


With this query I obtained the information about the total price of each product, this way I can find out which product is the most expensive and which is the least expensive one.

1992 Ferrari 360 Spider red is the product that makes the company the most money.

It might me interesting to check this very product further to get more insights.

Visualization on Power BI in the next slide demonstrate this with the help of bar chart.



With the help of this visualization we can easily detect which p most expensive and which is the least expensive.

Filters

Visualizations

Data

Search

- mintclassics offices
- mintclassics orderdetails
  - orderLineNumber
  - orderNumber
  - priceEach
  - productCode
  - quantityOrdered
- mintclassics orders
- mintclassics payments
- mintclassics productlines
- mintclassics products
  - buyPrice
  - MSRP
  - productCode
  - productDescription
  - productLine
  - productName
  - productScale
  - productVendor
  - quantityInStock
  - warehouseCode
- mintclassics warehouses

TotalPrice_perProduct | TotalOrder_perDate | +

```sql
-- This query show the prices for the very product which makes the company the most money:
SELECT products.productName AS productName, orderdetails.priceEach
FROM mintclassics.orderdetails
INNER JOIN mintclassics.products ON products.productCode = orderdetails.productCode
WHERE productName = '1992 Ferrari 360 Spider red'
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productName	priceEach
1992 Ferrari 360 Spider red	165.95
1992 Ferrari 360 Spider red	160.87
1992 Ferrari 360 Spider red	169.34
1992 Ferrari 360 Spider red	137.17
1992 Ferrari 360 Spider red	149.02
1992 Ferrari 360 Spider red	169.34
1992 Ferrari 360 Spider red	143.94
1992 Ferrari 360 Spider red	143.94
1992 Ferrari 360 Spider red	167.65
1992 Ferrari 360 Spider red	140.55
1992 Ferrari 360 Spider red	154.10
1992 Ferrari 360 Spider red	165.95
1992 Ferrari 360 Spider red	150.71
1992 Ferrari 360 Spider red	147.33
1992 Ferrari 360 Spider red	165.95
1992 Ferrari 360 Spider red	137.17
1992 Ferrari 360 Spider red	150.71
1992 Ferrari 360 Spider red	157.49

Investigating this product might give us some useful insights for the business problem we want to solve.

It is might be interesting to check:

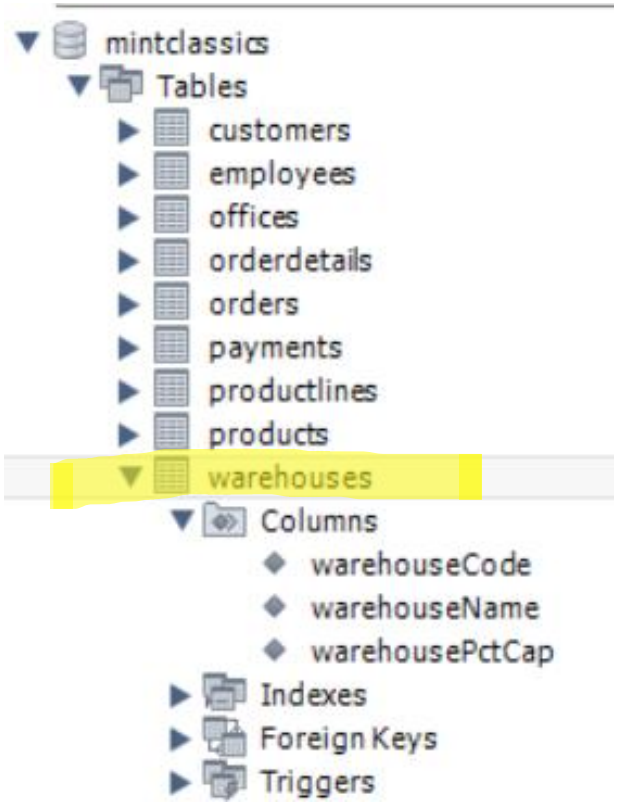
- Which warehouse stocks this product
- In which country this is ordered; price might change depending on the location.
- When it is ordered mostly
- Prices do not vary a lot. I can also find the average price for this product. See the snapshot below, **the average value for the price of 1992 Ferrari 360 Spider red is ~152.4**, which is plausible when you check the prices manually.

```sql
-- This query computes the average price for 1992 Ferrari 360 Spider red:
SELECT products.productName AS productName, AVG(orderdetails.priceEach) AS average_price
FROM mintclassics.orderdetails
INNER JOIN mintclassics.products ON products.productCode = orderdetails.productCode
WHERE productName = '1992 Ferrari 360 Spider red'
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

productName	average_price
1992 Ferrari 360 Spider red	152.342830

Warehouses - the smallest table of all; containing 3 columns and 5 rows.



Each product is stored in one warehouse.

Result Grid | Filter Rows: | Edit:

warehouseCode	warehouseName	warehousePctCap
a	North	72
b	East	67
c	West	50
d	South	75
NULL	NULL	NULL

Step 3: Questions to be asked

After understanding the data and the business problem, it is natural to ask questions in order to come up with the solution for the business. I will list the questions popped up when I was working, which I believe will be helpful later:

- Who are the regular customers? Where they locate?
- Who pays more money?

⇒ we can think of this as which customer helps our business to earn more money, this way we can also check which products they usually buy (i.e., which is their favourite products). From here we can ask another question: where these products are stored/stocked?

- Who does more sales?
- Which warehouse has more popular products?

⇒ Then we can decide not to close that very warehouse. I don't know yet how I can investigate this, but we will see.

- What **products** are popular, i.e. the products that are ordered frequently?

⇒ Which warehouses handling orders of those products?

⇒ Are some warehouses handling more **orders** than others?

- We can later investigate in which **country** does the **warehouse** we wan to close is located.

⇒ We need to check **offices** dataset for this analysis.

My **goal** is to analyse their **inventory, sales, and warehouse efficiency** to suggest:

- Whether the inventory/stock should be **moved to other location**?

⇒ How this will affect order fulfilment and customer satisfaction.

- Which warehouse(s) could be closed/reorganized?

Key Tables for the Analysis

Look specifically at values for common fields/columns that relate tables. For example, the warehouseCode field is present in both the warehouses table and in the products table.

To analyze **inventory management and warehouse efficiency**, these tables will be especially important:

- **products** → Contains quantityInStock and warehouseCode, which show where inventory is stored.
- **warehouses** → Shows the storage facilities and their capacity usage (warehousePctCap).
- **orders & orderdetails** → Help analyze sales trends and fulfillment times (shippedDate, orderDate, quantityOrdered).
- **customers** → Useful for understanding where demand is coming from (location-based analysis).

Business Recommendations

- Which products should be prioritized for storage?
- Can inventory be consolidated into fewer warehouses?
- Are there opportunities to reduce costs without affecting service time?